Reversible modular arithmetic in Shor's algorithm

What it must do, how it is built, and how it runs — with verified Qiskit code

What this is, and how to read it

Shor's circuit is three layers: Hadamards, a ladder of controlled modular multiplications, and an inverse QFT. The first and third are drawn gate by gate everywhere. The middle one is almost always a box labelled "$\times a^{2^j} \bmod N$" — and it is over 99% of the circuit. This note opens that box twice: once as mathematics, once as running code.

Parts I–III why the arithmetic is what it is: what phase estimation demands, why garbage is fatal, the tower from Toffoli up, and how a pile of permutations turns into a measurable phase.

Part IV a complete, minimal, *verified* implementation — about 150 lines of Qiskit, with the tests that make it believable.

Part V making it fast: ripple-carry arithmetic, windowing, unary-iteration lookups, measurement-based uncomputation, nested windows.

Part VI reading somebody else's implementation, and what its bugs teach.

Every listing in Parts IV–V is copied from a file that was executed, and every number quoted as measured was measured. The implementation factors 15, 21 and 33 end to end; 21 and 33 are the honest tests, because their orders do not divide 2^t and so force the continued-fraction step to do real work.

Notation: N is the modulus, $n = \lceil \log_2 N \rceil$, a the base with $\gcd(a, N) = 1$, r its order, $t = 2n$ the counting register. Qubit k of a register holds bit k of its value, LSB first (Qiskit's convention).

Contents

Part I — Orientation 4

1 Where the arithmetic sits 4
- 1.1 What phase estimation asks for, and what it costs . . . 4
- 1.2 The four demands on U_a . . . 5
- 1.3 One circuit, two descriptions . . . 5
- 1.4 Why garbage is fatal, not untidy . . . 6
- 1.5 The zoom . . . 6

2 How to read these circuits 6

Part II — The tower 7

3 Level 0: the gate set 7

4 Level 1: addition **8**
4.1 The recurrences . . . 8
4.2 The two blocks . . . 8
4.3 The matrices . . . 9
4.4 The CDKM ripple-carry adder . . . 9
4.5 Subtraction and comparison . . . 11

5 Level 2: modular addition by a classical constant **11**

6 Level 3: modular multiplication **12**

7 Level 4: the controlled multiplier, in place **14**

8 Level 5: the ladder **15**

Part III — Closing the loop **16**

9 From permutations to phases **16**

10 The bill **17**

Part IV — Building it: a verified implementation **17**

11 Level 0: the QFT **18**

12 Level 1: addition by a classical constant **19**

13 Level 2: modular addition — the hard one **19**

14 Levels 3–4: multiplication, in place **21**

15 Level 5: the ladder and the full circuit **22**

16 The classical half **23**

17 Verifying it — level by level **24**

Part V — Making it fast **27**

18 The other Level 1: ripple-carry arithmetic **28**
18.1 Two blocks that do everything . . . 28
18.2 Adding a classical constant . . . 29
18.3 What it costs, measured . . . 30

19 Windowing: trading lookups for additions **30**
19.1 The idea in three steps . . . 31
19.2 What has to be built . . . 31
19.3 What it costs, measured . . . 33
19.4 End to end . . . 33
19.5 Making the lookup cheap: unary iteration . . . 34
19.6 Uncomputing a lookup for free . . . 35
19.7 Nesting: windowing the exponent too . . . 36
19.8 What of the paper is actually here . . . 37

20 Emitting temporary ANDs **37**

21 One counting qubit: $4n + 2 \to 2n + 3$ **38**

22 Coset representation: deleting the modular adder **39**
22.1 The deviation law, measured . . . 40
22.2 What it saves . . . 41
22.3 Multiplying in the coset representation . . . 41

23 Where to go from here **42**

Part VI — Reading someone else's implementation 43

24 The package at a glance **43**

25 Level 1 in code, twice **43**
25.1 The ripple-carry backend . . . 43
25.2 The Fourier backend . . . 45

26 Levels 2–4: written once, above the interface **46**
26.1 The modular adder . . . 46
26.2 The multiplier, out of place and then in place . . . 47

27 Level 5 and the two order-finding circuits **48**
27.1 One control qubit: the $2n + 3$ circuit . . . 49
27.2 The bill, as the package pays it . . . 50

28 Cross-check: factoring 15 from scratch **50**

29 Running it **52**

Part VII — The other Shor: elliptic-curve discrete logarithms 54

30 What changes, and what does not **54**

31 Why it works **55**
31.1 The state, in four lines . . . 55
31.2 Two lattices . . . 56
31.3 The amplitude . . . 56
31.4 Reading off k, and what it costs . . . 57

32 The group law, and why it resists reversibility **57**

33 Level 2: modular arithmetic over $\mathbb{F}_p$ **58**

34 Level 3: inversion by Kaliski's algorithm **59**

35 Level 4: the eleven-step point addition **62**

36 Making it fast **63**
36.1 Level 1: which adder, and how to control it . . . 64
36.2 Level 0: temporary ANDs . . . 65
36.3 Level 2: approximate and pseudo-Mersenne reduction . . . 65
36.4 Level 3: Montgomery multiplication over a carry-save tree . . . 66
36.5 Level 3: dropping the counter and postponing the reduction . . . 67

36.6 Level 3, rebuilt: the Euclidean dialog . . . 68
36.7 Level 4, rebuilt: projective coordinates . . . 72
36.8 Level 5: windowing, zig-zag, and the semiclassical transform . . . 73

37 Everything at once **74**
37.1 Space at cryptographic size . . . 76

38 Verifying it **76**
38.1 The distribution, and the control that makes it mean something . . . 78

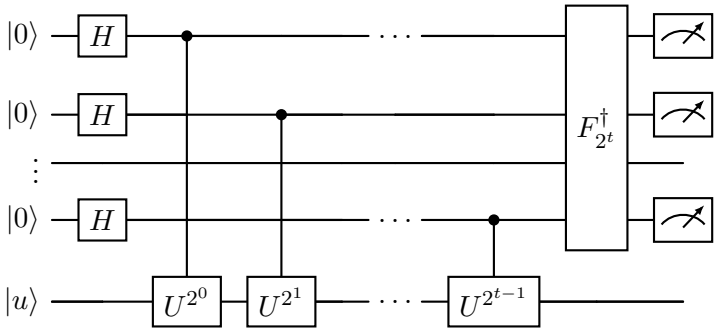
39 What is not here **79**

Part I — Orientation

1 Where the arithmetic sits

1.1 What phase estimation asks for, and what it costs

Phase estimation [2, 3] is a general procedure. Give it a unitary U, an eigenvector $|u\rangle$ with $U|u\rangle = e^{2\pi i\varphi}|u\rangle$, and t blank qubits, and it writes t bits of φ into those qubits. It is entirely indifferent to what U is.



But look at what it charges you. The rungs are $U^1, U^2, U^4, \ldots, U^{2^{t-1}}$, so building them by repetition costs

$$1 + 2 + 4 + \cdots + 2^{t-1} = 2^t - 1$$

applications of U — exponentially many. **Phase estimation is efficient only for unitaries whose powers are cheap.** For a generic U — a black box, a Hamiltonian evolution e^{iHt} — there is no shortcut, and the procedure is useless. This is worth saying plainly because the usual presentation, which introduces QPE abstractly and then "applies" it to U_a, makes the application look routine. It is not routine; it is the whole trick.

Modular multiplication is special twice over:

Gift 1: the powers collapse. $U_a^{2^j} = U_{a^{2^j} \bmod N}$. A high power of U_a is not a long circuit — it is *the same circuit with a different classical constant*, and that constant is computed classically by repeated squaring before the machine is switched on. This turns $2^t - 1$ applications into t.

Gift 2: one rung is cheap. U_a itself has a circuit of size $O(n^2)$. This is Part II.

Both are needed, and neither implies the other. Gift 1 without Gift 2 would be a short ladder of unaffordable rungs. Gift 2 without Gift 1 would be an affordable rung repeated an unaffordable number of times. Shor's algorithm [1] exists in the narrow space where arithmetic hands you both.

1.2 The four demands on U_a

Phase estimation does not accept just any circuit that computes $ay \bmod N$. Four properties are load-bearing, and each one is discharged at a specific level of the tower.

Demand	Why	Discharged at
Unitary	QPE's whole framework is spectral; a non-unitary map has no eigenphase to read.	Level 4 (§7)
Cheap powers	otherwise the ladder is exponentially long (§1.1).	Level 5 (§8)
Controllable	an *uncontrolled* U contributes a global phase, which no measurement can see; the control is what makes the phase relative.	Level 4 (§7)
Garbage-free	retained scratch entangles with the counting register and destroys the interference the inverse QFT depends on.	Levels 1–2 (§4, §5)

Unitarity is a number-theoretic condition. The map $y \mapsto ay \bmod N$ is injective on $\{0, \ldots, N-1\}$ precisely when a is invertible mod N, i.e. when $\gcd(a, N) = 1$. If that fails, the map collapses distinct inputs together, is not a permutation, and is not unitary — so there is no operator, no spectrum, and nothing for QPE to estimate. (In the algorithm this is checked classically first, and if it fails you have already factored N by computing the gcd.) The same condition returns at Level 4 in a second role: $\bar{a} = a^{-1} \bmod N$ is what makes the multiplier realisable *in place*. One hypothesis, two jobs.

1.3 One circuit, two descriptions

Here is the conceptual hinge of the whole subject, and the reason a note about *classical* arithmetic belongs inside a *quantum* algorithm.

The ladder is a single unitary. Written in the computational basis it says

$$|x\rangle|y\rangle \;\longmapsto\; |x\rangle\,|a^x y \bmod N\rangle \qquad \text{— it tabulates the function } f(x) = a^x \bmod N, \tag{1}$$

which is how Shor described it in 1994 [1]: build a periodic function, Fourier-sample its period. Written in the eigenbasis of U_a the very same unitary says

$$|x\rangle|u_s\rangle \;\longmapsto\; e^{2\pi i x s/r}|x\rangle|u_s\rangle \qquad \text{— it kicks an eigenphase onto the control register,} \tag{2}$$

which is Kitaev's [2] and CEMM's [3] reading: phase estimation on U_a. The two are related by the change of basis $|1\rangle = r^{-1/2}\sum_s |u_s\rangle$, and they are descriptions of one circuit, not two algorithms.

Why this matters for the arithmetic

Reading (1), the ladder does nothing but arithmetic — and indeed everything in Part II is a permutation matrix with no complex number anywhere in it. Reading (2), the same gates manufacture the phases that the algorithm ultimately measures.

Nothing was added in between. The phases are not written by any gate; they appear because a classical permutation was applied to a *superposition* rather than to a number, and then interfered. That is the single thing the classical circuit — from which every gate in Part II is borrowed — could not do.

1.4 Why garbage is fatal, not untidy

This is the demand that most shapes the construction, so it deserves the argument rather than the slogan. Suppose some ancilla ends in a state $|g_x\rangle$ that depends on which branch of the superposition it is in. Then instead of the clean state the algorithm wants, you have

$$\frac{1}{\sqrt{2^t}}\sum_x |x\rangle\,|a^x \bmod N\rangle\,|g_x\rangle.$$

The inverse QFT works by making branches x and x' interfere. Their interference term carries a factor $\langle g_{x'}|g_x\rangle$. If the garbage records anything about x — and for a useful computation it always does — those states are orthogonal, the factor is 0, and there is no interference at all. The measurement returns uniform noise. The circuit is still perfectly unitary; it simply computes nothing.

The consequence for every block below

You cannot erase scratch by overwriting it (that is not reversible) and you cannot discard it (it stays entangled). The only way out is to *run a computation backwards* and unwrite it. Hence:

- Level 1's adder ends with a reversed chain of UMA blocks that erase the carries;
- Level 2's modular adder spends three of its seven blocks recomputing and clearing one flag qubit;
- Level 4's multiplier runs a second multiplication *in reverse* purely to restore a scratch register to $|0\rangle$.

Roughly half of every circuit in Part II exists to undo the other half. That is not waste — it is the price of the interference, and §4 shows the undoing can be made to write the answer on its way out.

1.5 The zoom

Every box at one level is a whole circuit at the level below. Reading downwards is the plan of the document; the right-hand column is where the n^3 comes from. The last column names the method of the `qiskit-shor` package [25] that realises the level — Part IV reads those methods in full.

Level	Object	Made of	Multiplier	In code (§24)
5	the ladder	t rungs, one per counting qubit	$\times 2n$	`exponentiate_modulo`
4	$c\text{-}U_{a^{2^j}}$	$2\times$ CMULT and n Fredkins		`c_multiply_modulo`
3	CMULT(k)	doubly-controlled modular adders	$\times n$	`c_add_quantum_modulo`
2	ADD$^{\bmod N}(k)$	constant additions and sign tests	$\times 5$	`c_add_classical_modulo`
1	ADD(k)	MAJ chain, CNOT, UMA chain	$\times 2n$	`add_classical`
0	Toffoli	Clifford+T	$7T$; $4T$ if uncomputed	`MAJGate`, `UMAGate`

Three of those steps each multiply by roughly n — rungs, bits of the multiplicand, bits per addition — and that, and nothing subtler, is why the algorithm is cubic.

2 How to read these circuits

The single most misleading thing about circuit diagrams: **the horizontal lines are not wires and nothing flows along them.** There is no signal travelling rightwards. Each line is *one qubit*,

and moving right means moving *forward in time*. The diagram is closer to sheet music than to a plumbing schematic: one staff line per qubit, read left to right.

So a diagram with eight lines has eight qubits and always will. Lines never split, merge, or get created. Whatever is written at the far left is the input; whatever is at the far right is the output. *Read those two edges first* — the middle is only the recipe for getting from one to the other.

Symbol	Meaning
filled dot •	a *control*: look at this qubit, change nothing
$\oplus$	a *target*: flip this qubit, if every dot on its vertical line is 1
box across several lines	one operation on those qubits jointly (its innards are drawn elsewhere)
vertical line	not a wire — visual glue saying "these symbols are simultaneous"
line labelled $\|x\rangle_{n+1}$	a *bundle*: $n{+}1$ qubits drawn as one line to keep the picture small

Everything in one vertical column happens at the same instant. When a control sits on a bundle, the caption must tell you *which* qubit of the bundle is the control — in the modular adder of §5 it is always the top one, the sign qubit.

Part II — The tower

3 Level 0: the gate set

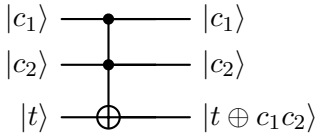
The conceptual point of this level. Every gate in Part II is a *permutation matrix*: one 1 per row and per column, everything else 0. Such a matrix is real and orthogonal, hence unitary, and it maps computational basis states to computational basis states. In other words the entire tower is a classical reversible circuit that happens to act linearly on superpositions. **No complex number appears anywhere below Level 5.** Keep §1.3 in mind: this is exactly why the phases of Shor's algorithm cannot be attributed to any gate.

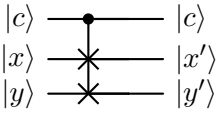
$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad \mathrm{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix},$$

$$\mathrm{TOF} = \begin{pmatrix} 1&0&0&0&0&0&0&0 \\ 0&1&0&0&0&0&0&0 \\ 0&0&1&0&0&0&0&0 \\ 0&0&0&1&0&0&0&0 \\ 0&0&0&0&1&0&0&0 \\ 0&0&0&0&0&1&0&0 \\ 0&0&0&0&0&0&0&1 \\ 0&0&0&0&0&0&1&0 \end{pmatrix}, \qquad \mathrm{FRED} = \begin{pmatrix} 1&0&0&0&0&0&0&0 \\ 0&1&0&0&0&0&0&0 \\ 0&0&1&0&0&0&0&0 \\ 0&0&0&1&0&0&0&0 \\ 0&0&0&0&1&0&0&0 \\ 0&0&0&0&0&0&1&0 \\ 0&0&0&0&0&1&0&0 \\ 0&0&0&0&0&0&0&1 \end{pmatrix}$$

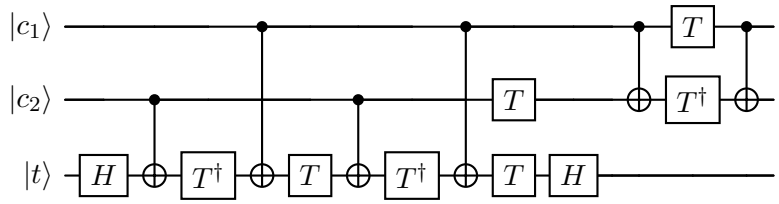
$$X|a\rangle = |a \oplus 1\rangle, \qquad \mathrm{CNOT}|c,t\rangle = |c,\ t \oplus c\rangle, \qquad \mathrm{TOF}|c_1,c_2,t\rangle = |c_1,c_2,\ t \oplus c_1c_2\rangle,$$

$$\mathrm{FRED}|c,x,y\rangle = |c,\ x \oplus c(x \oplus y),\ y \oplus c(x \oplus y)\rangle \quad (\text{swap } x,y \text{ iff } c=1).$$





The Toffoli is the *only* nonlinear gate. CNOT implements $\mathbb{F}_2$-linear maps and is therefore free in the fault-tolerant cost model; the Toffoli must be synthesised, and its standard Clifford+T decomposition costs 7 T gates and 6 CNOTs:



This is why gate counts in this subject are quoted in *Toffolis* rather than in gates: the Toffoli count is a proxy for the T count, and the T count is what magic-state distillation actually bills you for.

Remark (the halving that matters). A Toffoli that is *later uncomputed* — a *temporary logical-AND* [16] — costs only 4 T for the compute/uncompute *pair*, because the uncomputation can be done by an X-basis measurement of the ancilla and a classically conditioned CZ, which is Clifford. By §1.4 nearly every Toffoli below is of exactly this kind, so this is not a corner case — it is the common case.

4 Level 1: addition

The problem this level solves. Adding two n-bit numbers is column-by-column addition: add the two digits and whatever carried in, write a digit, carry out. The carries are the difficulty, because column $i{+}1$ cannot start until column i has finished. Everything expensive about Shor's algorithm traces back to that chain.

4.1 The recurrences

With $c_0 = 0$,

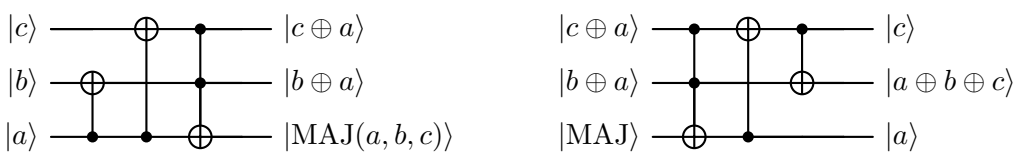
$$s_i = a_i \oplus b_i \oplus c_i, \qquad c_{i+1} = \mathrm{MAJ}(a_i, b_i, c_i) = a_i b_i \oplus a_i c_i \oplus b_i c_i, \qquad a + b = \sum_i s_i 2^i + c_n 2^n. \tag{3}$$

Read this as a cost statement. The sum bits are $\mathbb{F}_2$-*linear* given the carries, so they are CNOTs and therefore free. The carries are *quadratic*, so each one costs a Toffoli. And they form a chain, so the depth is $\Theta(n)$. Linear parts free, quadratic parts expensive, chain gives depth: that one line prices the algorithm.

4.2 The two blocks

On an ordered wire triple (c, b, a) define, following Cuccaro, Draper, Kutin and Moulton [9],

$$\begin{aligned} \mathrm{MAJ}: &\ \mathrm{CNOT}(a \to b),\ \mathrm{CNOT}(a \to c),\ \mathrm{TOF}(c, b \to a); \\ \mathrm{UMA}: &\ \mathrm{TOF}(c, b \to a),\ \mathrm{CNOT}(a \to c),\ \mathrm{CNOT}(c \to b). \end{aligned}$$



Lemma 4.1 (MAJ/UMA are partial inverses)**.**

$$\mathrm{MAJ}: (c, b, a) \mapsto \big(c{\oplus}a,\ b{\oplus}a,\ \mathrm{MAJ}(a, b, c)\big), \qquad \mathrm{UMA}: \big(c{\oplus}a,\ b{\oplus}a,\ \mathrm{MAJ}(a, b, c)\big) \mapsto \big(c,\ a{\oplus}b{\oplus}c,\ a\big).$$

Proof. For MAJ the only nontrivial step is the Toffoli, which sends $a \mapsto a \oplus (c \oplus a)(b \oplus a) = a \oplus \big(cb \oplus ca \oplus ab \oplus a\big) = ab \oplus ac \oplus bc$, using $a^2 = a$ and $a \oplus a = 0$ over $\mathbb{F}_2$. For UMA the same computation read backwards gives $\mathrm{MAJ}(a,b,c) \oplus (c \oplus a)(b \oplus a) = a$; then $c \oplus a \mapsto (c \oplus a) \oplus a = c$, and $b \oplus a \mapsto (b \oplus a) \oplus c$, which is the sum bit $s = a \oplus b \oplus c$. □

The idea, in one sentence

Look at what the second half of Lemma 4.1 says: UMA **erases the carry and writes the sum bit in the same three gates.**
By §1.4 the carries *must* be erased, so a reverse pass is compulsory. The design makes that compulsory pass do the useful work. The paper analogy is exact: when you add by hand you scribble little carry digits above the columns, and you are left with a page full of scribbles. Here you go back over the columns rubbing out each scribble, and the act of rubbing out deposits the answer.

4.3 The matrices

Ordering the basis $|c\,b\,a\rangle$ as $0, \ldots, 7$, the blocks are the permutations $\mathrm{MAJ} = (0)(1\,6\,7)(2)(3\,5)(4)$ and $\mathrm{UMA} = (0)(1\,7\,4\,6\,3\,5)(2)$:

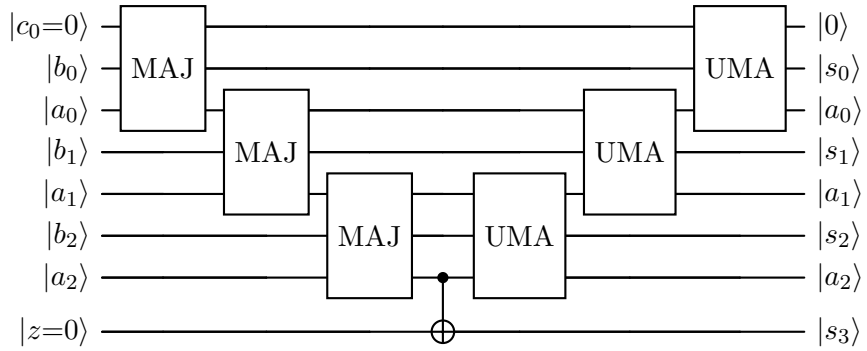
$$\mathrm{MAJ} = \begin{pmatrix} 1&0&0&0&0&0&0&0\\ 0&0&0&0&0&0&0&1\\ 0&0&1&0&0&0&0&0\\ 0&0&0&0&0&1&0&0\\ 0&0&0&0&1&0&0&0\\ 0&0&0&1&0&0&0&0\\ 0&1&0&0&0&0&0&0\\ 0&0&0&0&0&0&1&0 \end{pmatrix}, \qquad \mathrm{UMA} = \begin{pmatrix} 1&0&0&0&0&0&0&0\\ 0&0&0&0&0&1&0&0\\ 0&0&1&0&0&0&0&0\\ 0&0&0&0&0&0&1&0\\ 0&0&0&0&0&0&0&1\\ 0&0&0&1&0&0&0&0\\ 0&0&0&0&1&0&0&0\\ 0&1&0&0&0&0&0&0 \end{pmatrix}$$

(rows indexed by output, columns by input). Both are orthogonal, and $\mathrm{UMA} \cdot \mathrm{MAJ} : (c, b, a) \mapsto (c,\ a \oplus b \oplus c,\ a)$ — the permutation $0, 3, 2, 1, 6, 5, 4, 7$, which is Lemma 4.1.

4.4 The CDKM ripple-carry adder

Why the wires look scrambled

The wire order is $c_0, b_0, a_0, b_1, a_1, \ldots, z$ — alternating, not "all the a's then all the b's". This is the feature that makes the diagram look impenetrable, and it means *nothing mathematically.* It is a seating plan. Each block must touch three particular qubits; interleave them this way and those three are always *adjacent*, so the blocks are compact rather than a tangle of long-range connections. Re-order the wires and the circuit is identical, only uglier.



Theorem 4.2 (CDKM adder [9])**.** *The circuit above implements*

$$|c_0{=}0\rangle\,|b\rangle\,|a\rangle\,|z\rangle \;\longmapsto\; |0\rangle\,|(a+b) \bmod 2^n\rangle\,|a\rangle\,|z \oplus c_n\rangle$$

using $2n$ *Toffoli gates,* $4n{+}1$ *CNOTs and* $2n{+}2$ *qubits, with the operand* a *preserved, addition in place on* b*, and* ***zero garbage****. Fault-tolerantly it costs* $4n$ T *gates, each carry being a temporary AND [16].*

A run, in full: $a = 5$, $b = 3$

after	c_0	b_0	a_0	b_1	a_1	b_2	a_2	z
start	0	**1**	1	**1**	0	**0**	1	0
MAJ(c_0, b_0, a_0)	1	0	**1**	1	0	0	1	0
MAJ(a_0, b_1, a_1)	1	0	1	1	**1**	0	1	0
MAJ(a_1, b_2, a_2)	1	0	1	1	0	1	**1**	0
CNOT $a_2 \to z$	1	0	1	1	0	1	1	**1**
UMA(a_1, b_2, a_2)	1	0	1	1	1	**0**	1	1
UMA(a_0, b_1, a_1)	1	0	1	**0**	0	0	1	1
UMA(c_0, b_0, a_0)	**0**	**0**	1	0	0	0	1	1

Read the b column downwards: it starts as $011 = 3$ and ends as 000, with $z = 1$ — that is $1000_2 = 8$. The a wires end at $1, 0, 1 = 5$, exactly as they started, and c_0 is back to 0. Nothing is left dirty, which is what Theorem 4.2 promises and §1.4 demands.

The bold entries on the way down are the carries being deposited. Notice that a_1 shows 1 after its own block and then drops to 0 after the *next* one — because each MAJ also XORs into the wire below it. **The only wire holding a clean, unmodified carry at any moment is the topmost one, a_2, immediately after the last** MAJ**.** That is exactly where the single CNOT sits. The placement is not arbitrary: it is the one instant at which the final carry is sitting in the clear.

Verified

The circuit was built as a literal CNOT/TOF list and applied to all 2^{2n+2} basis states for $n = 2, 3, 4, 5$: the map is a permutation (reversible), the gate counts are exactly $2n$ Toffoli and $4n{+}1$ CNOT, and $\big(b, a, c_0, z\big) \mapsto \big((a{+}b) \bmod 2^n,\ a,\ 0,\ z \oplus c_n\big)$ holds for every input pair. The trace above is machine-generated.

Remark (where the ancillas went)*.* CDKM was a genuine economy. The first reversible adders — Vedral–Barenco–Ekert's [4], and the CARRY/SUM networks Draper [7] reviews before departing from them — kept every carry in its own ancilla: $3n$ qubits to add two n-bit numbers. The MAJ trick stores each carry *in place of* an operand bit it has already consumed, and $3n$ drops to $2n + 2$.

Draper's own answer to the same problem is more radical still — no carries at all — and is the Fourier route of §25.2.

4.5 Subtraction and comparison

Running Theorem 4.2 *backwards* gives SUB: $|b\rangle|a\rangle \mapsto |b - a \bmod 2^n\rangle|a\rangle$.

For comparison, the trick is to make the answer free rather than to build a comparator. Run the MAJ chain on a and $\bar{b}$ (bitwise complement) with carry-in 1: the register then holds $a + \bar{b} + 1 = a - b + 2^n$, whose carry-out satisfies

$$c_n = [\, a \geq b \,]. \tag{4}$$

Copy it with one CNOT and reverse the chain: n compute/uncompute Toffoli pairs, $4n$ T. Equivalently — and this is the form Level 2 uses — work one bit wider than you need, in two's complement, and the *top bit* of $x - y$ *is* the predicate $[\, x < y \,]$. **Comparison therefore costs nothing beyond the subtraction you were doing anyway**; you simply look at a qubit that is already there. That single sign bit is the workhorse of the whole level above.

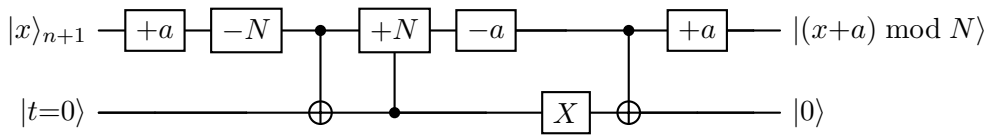
Remark (compile-time constants). Every addition inside Shor's algorithm has one operand *classically known*: the constants $2^i a \bmod N$ and the modulus N. Specialising the circuit to a fixed operand deletes or fixes roughly half the gates before emission — controls on known-0 bits vanish, controls on known-1 bits become unconditional. See the box in §6 for what this does and does not buy.

5 Level 2: modular addition by a classical constant

The problem this level solves, in one sentence: you want $(x + a) \bmod N$, which classically is "add them, and *if* the result is at least N, subtract N" — and a quantum computer cannot do *if*.

It cannot branch, because in general it is holding a superposition in which some branches overshot and some did not; there is no single answer to "did it overshoot?". So the conditional must be turned into arithmetic: compute the condition into a qubit, then use that qubit as a control. Blocks (1)–(4) below do exactly that, and if you could stop there the arithmetic would be finished. **Blocks (5)–(7) exist entirely to destroy the evidence**, for the reason in §1.4.

Theorem 5.1 (Beauregard's modular adder [8])**.** *For $0 \leq a < N$ the seven blocks below implement $|x\rangle \mapsto |(x + a) \bmod N\rangle$ for all $0 \leq x < N$, using five constant additions and one ancilla returned to $|0\rangle$. The value register is $n{+}1$ qubits wide so that intermediates up to $2N$ fit.*



Left to right, the columns are blocks (1)–(7) of the table below (block (6) is the X together with the CNOT that follows it). The two vertical lines into t are CNOTs *from the sign qubit* — the top bit of the $(n{+}1)$-qubit value register — and block (4) is the addition of N controlled on t.

	block	value register	t
(1)	ADD(a)	$x+a$ ($<2N$, fits)	0
(2)	SUB(N)	$x+a-N$, sign bit $=[\,x+a<N\,]$	0
(3)	CNOT sign $\to t$	$x+a-N$	$[\,x+a<N\,]$
(4)	ADD(N) controlled on t	$R:=(x+a) \bmod N$	unchanged
(5)	SUB(a)	$R-a$, sign bit $=[\,R<a\,]$	unchanged
(6)	X on t; CNOT sign $\to t$	$R-a$	**0**
(7)	ADD(a)	R	0

Proof. Blocks (1)–(2) form $x+a-N$, which is negative exactly when $x+a<N$; in two's complement the sign qubit *is* that predicate, so (3) records it and (4) restores $R\in[0,N)$. The remaining question is how to clear the ancilla, and the answer is that the flag is recomputable from the output alone. Since $x<N$,

$$t=1 \iff x+a<N \iff R=x+a\geq a, \qquad t=0 \iff R=x+a-N<a,$$

so $[\,R<a\,]=\neg t$. Block (5) forms $R-a$, whose sign qubit is therefore $\neg t$; the X in (6) maps $t\mapsto\neg t$ and the following CNOT gives $\neg t\oplus\neg t=0$. Block (7) restores R. □

The idea: deduce the flag, don't remember it

After block (4) you hold the right answer but you are stuck with a qubit remembering whether you wrapped around. You may not erase it and you may not discard it.
The way out is that **you do not need to remember, because you can work it out again from the answer**. Since x started below N, wrapping happened exactly when $R<a$. So block (5) recomputes the flag from the output and block (6) XORs it against itself.
It is like this: you cannot remember whether you borrowed during a subtraction, but you can look at your answer and deduce that you must have — and then honestly forget. **Every block in this tower ends by recomputing its own flag from its own output.** That single discipline is what keeps the whole construction garbage-free.

The precondition is load-bearing

The proof uses $x<N$ twice. Fed a value $\geq N$ the circuit silently leaves t dirty and the value register wrong by N — and by §1.4 a dirty t does not produce a wrong answer, it produces *no* answer, since the interference downstream dies.
The invariant holds because *every* adder in the tower reduces mod N (§6). Note that this is the same hypothesis that licenses the 3-of-7 control saving in §7: the uncontrolled blocks compose to the identity only when $b<N$. One precondition, two uses.

Remark (traced, $N=15$). $x=3$, $a=7$: 3+7=10 → 10−15= − 5 (sign 1) → t=1 → +15 → R=10 → 10−7=3 (sign 0) → $t=1\oplus1\oplus0=0$. $x=9$, $a=7$: 9+7=16 → 16−15=1 (sign 0) → t=0 → R=1 → 1−7= − 6 (sign 1) → $t=0\oplus1\oplus1=0$. Both branches: correct output, ancilla clean.

6 Level 3: modular multiplication

The problem this level solves. You have an adder; you need a multiplier. The bridge is the schoolbook one — reversible shift-and-add multiplication goes back to Vedral, Barenco and Ekert [4]

— and each shifted multiple is a *classically known constant*. For $x = \sum_i x_i 2^i$,

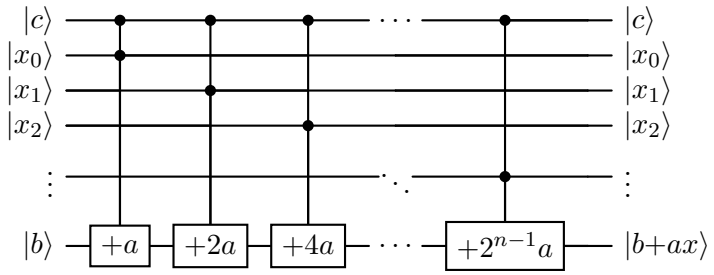
$$a\,x \bmod N = \Big(\sum_{i=0}^{n-1} x_i \left(2^i a \bmod N\right) \Big) \bmod N, \tag{5}$$

i.e. binary long multiplication written reversibly. The bit x_i decides whether its multiple joins the sum, so it becomes a *control*: the data has turned into conditioning.

Theorem 6.1.

$$\mathrm{CMULT}(a):\ |c\rangle|x\rangle|b\rangle \ \mapsto\ |c\rangle|x\rangle|\,b + c\cdot ax \bmod N\,\rangle \qquad \textit{is} \qquad \prod_{i=0}^{n-1} \mathrm{ADD}_{c,\,x_i}^{\bmod N}\big(2^i a \bmod N\big),$$

n doubly-controlled modular additions of compile-time constants.



Each box is one Level-2 modular adder for the constant $2^i a \bmod N$, doubly controlled by the outer control c and the bit x_i. Every partial sum is reduced mod N, which is what maintains the invariant $b < N$ that Theorem 5.1 requires.

Proof. Immediate from (5): the i-th adder fires exactly when $x_i = 1$, and each adder reduces, so all partial sums stay below N. □

The constant is known; the operand is not

It is tempting to think that because a is chosen in advance, the multiplication is "already done". But a multiplication has *two* operands, and precomputation gives you one:

$$\underbrace{a}_{\text{known at compile time}} \times \underbrace{x}_{\text{unknown — it is the register}} \bmod N.$$

The register holds $a^x \bmod N$ for a counting register in superposition over all 2^t values of x. If you knew which x mattered you would compute $a^x \bmod N$ on a laptop and never build the machine. The machine exists precisely because it evaluates at every input at once — so every multiplication in the ladder is fed a *superposition*, and must be a circuit that maps all inputs correctly and simultaneously. For $N = 15$, $a = 7$ the first rung must realise

$$1\to 7, \quad 2\to 14, \quad 4\to 13, \quad 7\to 4, \quad 8\to 11, \quad 11\to 2, \quad 13\to 1, \quad 14\to 8$$

in one shot. Precomputing the whole input/output table instead would need 2^n rows: about 10^{616} for RSA-2048, against 10^{80} atoms in the observable universe. The constant, by contrast, is 2048 bits — 256 bytes. That gap is exactly the gap between what precomputation gives you and what still has to be built.

What precomputation *does* buy is real and is used everywhere: the rung constants (§8), the shifted multiples $2^i a \bmod N$ above, the inverse $\bar{a}$ (§7), and the roughly half of all gates that can be deleted because one adder operand is classical.

Remark ($N = 15$, $a = 7$). The four constants are $2^i \cdot 7 \bmod 15 = 7, 14, 13, 11$. For $x = 3 = 0011_2$ the first two adders fire: $7 + 14 = 21 \equiv 6 \pmod{15}$, and indeed $7 \cdot 3 = 21 \equiv 6$.

7 Level 4: the controlled multiplier, in place

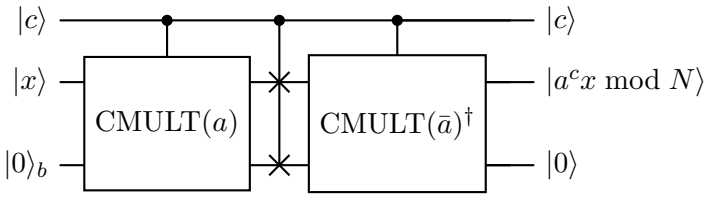
The problem this level solves. CMULT writes its answer into a *fresh* register, but the ladder must apply many multiplications one after another to the *same* register. A circuit that consumed a clean register per rung would need thousands of registers. So the out-of-place multiplier has to be converted into an in-place map $|x\rangle \mapsto |ax \bmod N\rangle$, and the leftover input x has to disappear — which, by §1.4, means it must be *computed away*, not discarded.

The escape is that a is invertible. You can uncompute x from the product. The construction is Beauregard's [8].

Theorem 7.1. *Let* $\gcd(a, N) = 1$ *and* $\bar{a} := a^{-1} \bmod N$ *(classical, extended Euclid). Then*

$$c\text{-}U_a \;=\; \mathrm{CMULT}(\bar{a})^\dagger \cdot c\text{-SWAP} \cdot \mathrm{CMULT}(a)$$

implements $|c\rangle|x\rangle|0\rangle \mapsto |c\rangle|a^c x \bmod N\rangle|0\rangle$ *for all* $0 \le x < N$.



Proof. Take $c = 1$ and follow the register pair:

$$|x\rangle|0\rangle \xrightarrow{\text{CMULT}(a)} |x\rangle|ax\rangle \xrightarrow{\text{SWAP}} |ax\rangle|x\rangle \xrightarrow{\text{CMULT}(\bar{a})^\dagger} |ax\rangle|x - \bar{a}(ax)\rangle = |ax\rangle|0\rangle,$$

since $\bar{a}a \equiv 1 \pmod N$. For $c = 0$ every factor is the identity. Both branches leave the scratch in $|0\rangle$, unentangled, so $c\text{-}U_a$ is realised exactly. □

The controlled SWAP is n Fredkin gates sharing one control. Note that no controlled QFT appears and the constant $\bar{a}$ carries no control — its inverse is classical. For $N = 15, a = 7$: $\bar{a} = 13$, since $7 \cdot 13 = 91 = 6 \cdot 15 + 1$.

Where the outer control goes, and why not everywhere

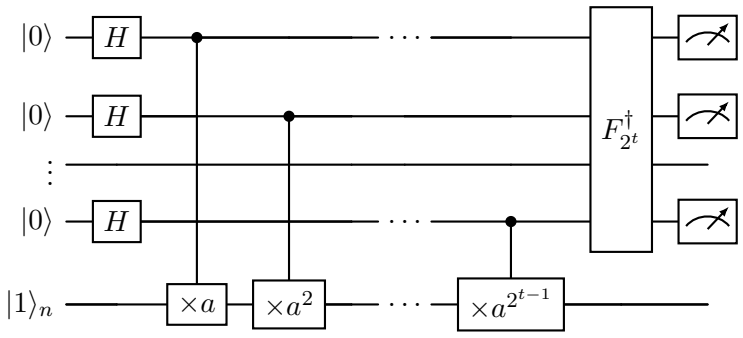
Let $V = G_L \cdots G_1$ and let V_S replace G_j by $c\text{-}G_j$ for $j \in S$. Since $V_S = |0\rangle\langle 0| \otimes R_S + |1\rangle\langle 1| \otimes V$ with $R_S = \prod_{j \notin S} G_j$, we have $V_S = c\text{-}V$ on a subspace W *iff* $R_S = \mathbb{1}$ on W. In words: **a gate needs the control only if leaving it uncontrolled would not cancel out**. Two patterns exhaust the uses:

- *Conjugation.* If $V = ABA^\dagger$ then controlling B alone suffices, since $AA^\dagger = \mathbb{1}$. A conjugating layer never needs the control.
- *Compensating residue.* In Theorem 5.1 only blocks (1), (5), (7) take the outer control: with the control off, blocks (2), (3), (4), (6) conspire to do nothing — $b - N$ goes negative, so t is set, block (4) adds N back, and block (6) clears t. This works only because $b < N$, the same precondition as before.

Measured on $N = 15$, $a = 7$, transpiled to $\{\text{CNOT}, U\}$: the modular adder costs 2 634 CNOTs at depth 4 836 if every gate is controlled, against 206 / 354 for the 3-of-7 construction (12.8×); the full $c\text{-}U_a$, 20 892 / 37 727 against 1 748 / 2 957 (12.0×). An order of magnitude, from one lemma. Both patterns reappear, letter for letter, in the code of Part IV (§25, §26).

8 Level 5: the ladder

What this level supplies. Gift 1 of §1.1. The rungs use classically precomputed constants $c_j = a^{2^j} \bmod N$, obtained by repeated squaring $c_0 = a$, $c_{j+1} = c_j^2 \bmod N$ — this is the identity $U_a^{2^j} = U_{a^{2^j} \bmod N}$, which is why the ladder has t rungs and not 2^t.



Each $\times a^{2^j}$ box is one $c\text{-}U_a$ of Theorem 7.1, hence $2n$ modular adders (Theorem 6.1) and n Fredkins; each modular adder is five constant additions (Theorem 5.1); each addition is the CDKM circuit (Theorem 4.2) at $2n$ Toffoli.

The ladder is not one circuit

Every rung uses a *different* constant, so the rungs are t structurally different circuits that happen to sit side by side. For $N = 15, a = 7$ the constants are $7, 4, 1, 1, 1, \ldots$ — so most of that ladder is literally the identity, which is why 15 is everyone's demo integer.

A consequence worth stating, because it surprises people: **the compiled circuit is specific to** a, and Shor's algorithm may need to retry with a fresh base. That is genuine, and it is cheap. Recompiling means repeated squaring plus one extended Euclid — $O(n^3)$ classical bit operations, measured at 50 ms for a 2048-bit modulus, against days of quantum runtime. And it is rarely needed: a bad measurement or a $\gcd(s, r) > 1$ failure is re-run on the *same* circuit, and only a bad base forces recompilation, which happens with probability at most $1/2$ for $N = pq$. Expected number of distinct circuits: about two.

Part III — Closing the loop

9 From permutations to phases

Now follow the state through, and watch classical arithmetic turn into an eigenphase. After the Hadamard layer,

$$\frac{1}{\sqrt{2^t}} \sum_{x=0}^{2^t-1} |x\rangle|1\rangle.$$

The ladder is the only place any arithmetic happens, and by §8 it computes

$$|x\rangle|y\rangle \longmapsto |x\rangle \,\big|\, a^{\sum_j x_j 2^j} y \bmod N \,\big\rangle = |x\rangle|a^x y \bmod N\rangle, \qquad \text{so} \qquad \frac{1}{\sqrt{2^t}} \sum_x |x\rangle|a^x \bmod N\rangle.$$

Every gate that produced this was a permutation matrix with entries 0 and 1. Now rewrite the *same state* in the eigenbasis of U_a, using $|1\rangle = r^{-1/2} \sum_s |u_s\rangle$:

$$\frac{1}{\sqrt{2^t}} \sum_x |x\rangle|a^x \bmod N\rangle \;=\; \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} \underbrace{\Big[\frac{1}{\sqrt{2^t}} \sum_x e^{2\pi i x s/r} |x\rangle\Big]}_{\text{a phase ramp of frequency } s/r} \otimes |u_s\rangle.$$

The phases were not written by any gate. They are what the permutation *looks like* in the other basis — and they are visible only because the register was in a superposition, so that a permutation of basis labels became a relative phase between branches. The inverse QFT then reads the frequency of the ramp, and the measurement returns an approximation to s/r for a uniformly random s; continued fractions recover r [1].

The whole story in four lines

1. Phase estimation is cheap only when U has cheap powers; arithmetic makes U_a's powers cheap (Gift 1) *and* makes one power cheap to build (Gift 2).
2. Building it means reversible classical arithmetic — adders, a modular reduction, a shift-and-add multiplier, an in-place trick that needs a^{-1}.
3. Every scratch bit must be uncomputed, or the interference that reads the phase does not happen. Half of every circuit in Part II is that uncomputation.
4. Nothing in the arithmetic is complex-valued. The phases appear on the change of basis, and that change of basis is the algorithm.

10 The bill

$$\underbrace{2n}_{\text{rungs}} \times \underbrace{2n}_{\substack{\text{modular adds}\\ \text{per } c\text{-}U_a}} \times \underbrace{5}_{\substack{\text{additions per}\\ \text{modular add}}} \times \underbrace{2n}_{\substack{\text{Toffoli per}\\ \text{addition}}} \approx 40\,n^3 \text{ Toffoli},$$

which windowed arithmetic [21] plus the 3-of-7 control trick reduces to $\approx 0.3\,n^3$. Windowing is a *different* construction from the one built here — it replaces the controlled blocks by table lookups addressed by the exponent bits — and it is built and measured in Part V (§19).

At $n = 2048$ that is $\approx 2.6 \times 10^9$ Toffoli for the modular exponentiation against $\approx 10^5$ gates for the approximate QFT: **the Fourier layer, which carries all the conceptual content, is** 0.003% **of the circuit.** Everything that determines whether RSA falls is the cost of multiplying two 2048-bit numbers, reversibly, in superposition.

What was machine-checked

MAJ/UMA (Lemma 4.1) on all 8 inputs and as 8×8 orthogonal matrices; the Clifford+T Toffoli against the 8×8 permutation (7 T gates); the CDKM circuit (Theorem 4.2) as a literal gate list on all 2^{2n+2} basis states for $n = 2, 3, 4, 5$ — reversibility, gate counts, and arithmetic — and the $a = 5, b = 3$ trace; the comparator identity (4) for all pairs at $n = 4, 6$; the seven-block modular adder (Theorem 5.1) for all $x, a < N$ at $N = 15, 21, 33, 55$, checking both the output and that t returns to 0; the shift-and-add identity (5) and the conjugation of Theorem 7.1 for all $x < N$ and all a coprime to N at $N = 15, 21, 55$; and the ladder identity $|x\rangle|1\rangle \mapsto |x\rangle|a^x \bmod N\rangle$.

Part IV — Building it: a verified implementation

Parts I–III said what the arithmetic must do and why. This part does it: a complete order-finding implementation in about 150 lines of Qiskit, bottom up, with the test at each level that makes it believable. It is self-contained — no circuit-library arithmetic, and the QFT built from scratch — and it takes the Fourier route, because that is the shortest correct path from nothing to a factored integer. Part V rebuilds the slow parts three different ways.

11 Level 0: the QFT

Everything in Parts IV and V needs only these imports — no Qiskit circuit-library primitives, no prebuilt arithmetic:

```
import math
from fractions import Fraction
import numpy as np
from qiskit.circuit import ClassicalRegister, QuantumCircuit, QuantumRegister
```

The one primitive everything rests on is the QFT, so build it rather than import it. It is

$$|j\rangle \longmapsto \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{2\pi i\, jk/2^n} |k\rangle,$$

and the standard circuit follows from the product form of that sum: for each qubit, one Hadamard, then a controlled phase from every less significant qubit, at halving angles. A final layer of swaps reverses the qubit order, which the product form leaves reversed.

```
def qft(n, approx=0):
    """QFT on n qubits, Qiskit convention (qubit 0 = LSB):
        |j> -> 2^(-n/2) sum_k exp(2*pi*i*j*k/2^n) |k>.
    approx > 0 drops controlled rotations by less than pi/2^approx."""
    qc = QuantumCircuit(n, name="qft")
    for j in reversed(range(n)):
        qc.h(j)
        for k in range(j):
            if approx and (j - k) > approx:
                continue                        # angle too small to matter
            qc.cp(np.pi / 2 ** (j - k), k, j)
    for i in range(n // 2):                     # reverse the qubit order
        qc.swap(i, n - 1 - i)
    return qc.to_gate()
```

That is $n(n+1)/2$ gates plus $\lfloor n/2 \rfloor$ swaps: $O(n^2)$. The `approx` parameter is the approximate QFT of §23 — rotations by less than $\pi/2^d$ are below any hardware's resolution, and dropping them costs negligible error while cutting the count to $O(n \log n)$. It is one `continue`, so there is no reason not to have it.

Verify this before anything else

Do not test the QFT by eye. Build its unitary and compare it with the definition:

```
ref = np.array([[np.exp(2j*np.pi*j*k/2**n) for j in range(2**n)]
                for k in range(2**n)]) / np.sqrt(2**n)
assert np.allclose(Operator(qft(n)).data, ref)        # qiskit.quantum_info.Operator
```

Checked for $n = 1..6$ against both the definition and Qiskit's own `QFTGate`: they agree exactly, and `qft(n).inverse()` undoes it. If you drop the swap layer, or reverse the loop order, you get a QFT of the bit-reversed integer — which is still unitary, still looks fine, and will make every addition below wrong. This is the single most common place to lose an afternoon.

12 Level 1: addition by a classical constant

Classically, addition means carries, and carries mean a chain of Toffolis and ancilla qubits. Draper's insight [7] is to avoid them entirely by working in the Fourier basis. Write the QFT of an n-qubit register as a product state:

$$F|y\rangle = \bigotimes_{i=0}^{n-1} \frac{|0\rangle + e^{2\pi i\, y\, 2^{\,i-n}}|1\rangle}{\sqrt{2}}.$$

Every qubit carries the whole value y — in the *phase*, at a different scale. Adding X means advancing each phase by the corresponding amount, so

$$\text{qubit } i \text{ gets } P\big(2\pi X\, 2^{\,i-n}\big), \qquad e^{2\pi i y 2^{i-n}} \longmapsto e^{2\pi i (y+X) 2^{i-n}},$$

and an inverse QFT lands back in the computational basis holding $|(y+X) \bmod 2^n\rangle$. No carries, no ancillas, and since X is classical every qubit needs exactly *one* phase gate, so the addition layer has depth 1.

```
def add_const(qc, X, y):
    """|y> -> |y + X mod 2^n> on the qubit list y (LSB first)."""
    n = len(y)
    qc.append(qft(n), y)
    for i in range(n):
        qc.p(2 * np.pi * X * 2.0 ** (i - n), y[i])
    qc.append(qft(n).inverse(), y)

def c_add_const(qc, ctrls, X, y):
    """add_const controlled on every qubit in ctrls."""
    n = len(y)
    qc.append(qft(n), y)
    for i in range(n):
        qc.mcp(2 * np.pi * X * 2.0 ** (i - n), ctrls, y[i])
    qc.append(qft(n).inverse(), y)
```

Three things to notice

Negative constants just work. $X = -N$ needs no separate subtractor: the phases are periodic, so $-N$ and $2^n - N$ give the same gates. Subtraction is addition of a negative Python integer, and §13 relies on this.

Only the phases take the control. In `c_add_const` the two QFT layers are *uncontrolled*. They form a conjugation $F^\dagger \cdots F$, so with the control off they cancel: controlling them would be pure waste. Keep this in mind — it is the same argument that saves an order of magnitude in §13.

The composite is a permutation. A single $P(\theta)$ is not, and intermediate states carry irrational phases, but $F^\dagger \cdot (\text{phases}) \cdot F$ is exactly the 0/1 permutation matrix $|y\rangle \mapsto |y+X\rangle$. Nothing above Level 1 can tell this apart from a ripple-carry adder.

13 Level 2: modular addition — the hard one

You want $(y + X) \bmod N$, which classically is "add, and *if* the result is $\geq N$, subtract N". A quantum computer cannot branch: it holds a superposition in which some branches overshot and some did not, so there is no single answer to "did it overshoot?".

So compute the condition into a qubit and use that qubit as a control. Give the value register one extra qubit (the *sign* qubit) so intermediates up to $2N$ fit and two's-complement negatives are readable: after subtracting, the top bit *is* the predicate. Then:

	block	value register	flag t
(1)	$+X$ (controlled)	$y + X$ $(< 2N$, fits)	0
(2)	$-N$	$y + X - N$, sign bit $= [\, y + X < N \,]$	0
(3)	CNOT sign $\to t$	unchanged	$[\, y + X < N \,]$
(4)	$+N$ if t	$R := (y + X) \bmod N$	unchanged
(5)	$-X$	$R - X$, sign bit $= \neg t$	unchanged
(6)	clear t from that sign bit	$R - X$	**0**
(7)	$+X$	R	0

Blocks (1)–(4) do the arithmetic. **Blocks (5)–(7) exist entirely to destroy the evidence**, and they are the interesting half. After (4) you hold the right answer but one qubit remembers whether you wrapped — which you may neither erase nor discard.

The escape: *you do not need to remember, because you can deduce it from the answer.* Since $y < N$, wrapping happened exactly when $R \geq X$. So block (5) recomputes the flag from the output and block (6) XORs it against itself. Concretely, $t = 1 \iff y + X < N \iff R = y + X \geq X$, so $[\, R < X \,] = \neg t$, which is the sign bit after (5); XOR that into t and apply X to finish.

```
def c_add_mod(qc, ctrls, X, y, sign, flag, N):
    """|y> -> |(y + X) mod N> when all ctrls are 1; identity otherwise.
    Needs 0 <= X < N, 0 <= y < N, sign and flag in |0>."""
    yw = list(y) + [sign]                       # (n+1)-wide value register
    c_add_const(qc, ctrls, X, yw)               # (1) + X          (controlled)
    add_const(qc, -N, yw)                       # (2) - N
    qc.cx(sign, flag)                           # (3) sign -> flag
    c_add_const(qc, [flag], N, yw)              # (4) + N  if flag
    add_const(qc, -X, yw)                       # (5) - X
    qc.mcx(list(ctrls) + [sign], flag)          # (6) clear flag from the answer
    qc.x(flag)
    add_const(qc, X, yw)                        # (7) + X
```

Where the control goes — the biggest constant factor in the circuit

Notice what is *not* controlled. Blocks (2)–(5) and (7) are plain, and inside every `add_const` the QFT layers are plain too. Only block (1), block (4)'s $+N$, and block (6)'s X carry a control. The governing lemma: replacing G_j by c-G_j only for $j \in S$ gives a correct c-V *iff* the leftover product $\prod_{j \notin S} G_j$ is the identity on the states you care about. Two patterns satisfy it, and both appear here.

Conjugation. If $V = ABA^\dagger$, controlling B alone suffices, since $AA^\dagger$ is the identity. This is the QFT sandwich in §12: never control a conjugating layer.

Compensating residue. Ask what the uncontrolled blocks do when the control is **off**: $y - N$ goes negative ⇒ sign set ⇒ (3) sets t ⇒ (4) adds N back. Then (5) subtracts X, the Toffoli in (6) does nothing (its outer control is off), the `x` flips t back to 0, and (7) restores X. Everything cancels. **This works only because** $y < N$ — the same precondition the correctness proof needs. With the control **on**: after (5) the sign qubit holds $\neg t$, so the Toffoli sets $t \mapsto t \oplus \neg t = 1$ and the `x` clears it.

Measured ($N = 15$, transpiled to $\{\text{CNOT}, U\}$), against the naive circuit built by wrapping an uncontrolled modular adder in `.control(k)` — i.e. controlling every gate:

	every gate controlled		as written here	
	CNOT	depth	CNOT	depth
1 control	2 353	4 351	287	372
2 controls (the real usage, §14)	5 916	10 689	315	418

8× and 19×. Almost all of it is the conjugation pattern — leaving the QFT layers alone. Isolating the residue pattern (blocks controlled or not, QFTs plain either way) is worth a further 1.6× at two controls. Do both; the first is nearly free to get right.

If you prefer, Beauregard's original choice is to control blocks (1), (5), (7) and leave (6) a plain CNOT. Both are correct, and only the residue bookkeeping changes.

The precondition is load-bearing

The proof uses $y < N$ twice. Fed $y \geq N$, the circuit leaves t dirty and the value wrong — and a dirty t does not give a wrong answer, it gives *no* answer. The invariant holds because every adder above reduces mod N; if you ever add without reducing, you have broken it.

14 Levels 3–4: multiplication, in place

Level 3, accumulate. Multiplication is shifted addition, and each shifted multiple $2^i A \bmod N$ is classically known. Bit x_i of the register decides whether its multiple joins the sum — so it becomes a *control*, and the data turns into conditioning:

$$A\,x \bmod N = \Big(\sum_{i=0}^{n-1} x_i \left(2^i A \bmod N\right) \Big) \bmod N.$$

Every partial sum is reduced, which is what maintains the $y < N$ invariant Level 2 demands.

```
def c_mult_acc(qc, c, A, x, y, sign, flag, N):
    """|c>|x>|y> -> |c>|x>|(y + A*x) mod N> : shift-and-add accumulate."""
    for i in range(len(x)):
```

```
        c_add_mod(qc, [c, x[i]], (A << i) % N, y, sign, flag, N)
```

Level 4, in place. `c_mult_acc` writes into a *fresh* register, but the ladder must apply rung after rung to the *same* register. Swapping is easy; the problem is that the old input x is still sitting in the scratch, and by the no-garbage rule it must be computed away, not discarded.

The escape is that A is invertible. Multiply, swap, then subtract A^{-1} times the product:

$$|x\rangle|0\rangle \xrightarrow{\times A} |x\rangle|Ax\rangle \xrightarrow{\text{SWAP}} |Ax\rangle|x\rangle \xrightarrow{-A^{-1}\cdot} |Ax\rangle|x - A^{-1}Ax\rangle = |Ax\rangle|0\rangle.$$

```
def c_ua(qc, c, A, x, y, sign, flag, N):
    """|c>|x>|0> -> |c>|A^c x mod N>|0> : multiply, swap, uncompute."""
    c_mult_acc(qc, c, A % N, x, y, sign, flag, N)
    for i in range(len(x)):
        qc.cswap(c, x[i], y[i])
    B = pow(A, -1, N)                          # classical inverse, gcd(A,N)=1
    c_mult_acc(qc, c, (-B) % N, x, y, sign, flag, N)
```

Note what is *not* here: no circuit is reversed. The adjoint of "add $A^{-1}x$ mod N" is "add $-A^{-1}x$ mod N", so the dagger becomes a minus sign on a classical constant. Note also that $\gcd(a, N) = 1$ is doing two jobs at once: it makes $y \mapsto ay$ a permutation (hence unitary, hence something phase estimation can act on *at all*), and it makes A^{-1} exist so the in-place trick works. One hypothesis, two roles.

15 Level 5: the ladder and the full circuit

```
def order_circuit(A, N, t=None):
    """Counting register of t qubits (default 2n), target, n+2 scratch."""
    n = math.ceil(math.log2(N))
    t = t if t is not None else 2 * n
    ctr, tgt = QuantumRegister(t, "ctr"), QuantumRegister(n, "tgt")
    anc = QuantumRegister(n, "anc")
    sf = QuantumRegister(2, "sf")              # sign qubit, flag qubit
    out = ClassicalRegister(t, "out")
    qc = QuantumCircuit(ctr, tgt, anc, sf, out)
    qc.h(ctr)                                  # Hadamard layer
    qc.x(tgt[0])                               # target = |1>
    for i in range(t):                         # the ladder: t rungs
        c_ua(qc, ctr[i], pow(A, 2**i, N), tgt, anc, sf[0], sf[1], N)
    qc.append(qft(t).inverse(), ctr)      # inverse QFT
    qc.measure(ctr, out)
    return qc
```

Qubit budget: t counting + n target + n scratch product + 1 sign + 1 flag $= 4n + 2$ with $t = 2n$. For $N = 15$: $8 + 4 + 4 + 2 = 18$.

Why it works. After the Hadamards and the ladder the state is $\frac{1}{\sqrt{2^t}} \sum_x |x\rangle|a^x \bmod N\rangle$ — and every gate that produced it was a permutation matrix of 0s and 1s. Rewriting the *same state* in the eigenbasis of U_a, using $|1\rangle = r^{-1/2} \sum_s |u_s\rangle$:

$$\frac{1}{\sqrt{2^t}} \sum_x |x\rangle|a^x \bmod N\rangle = \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} \underbrace{\left[\frac{1}{\sqrt{2^t}} \sum_x e^{2\pi i x s/r} |x\rangle\right]}_{\text{phase ramp of frequency } s/r} \otimes |u_s\rangle.$$

No gate wrote those phases. They are what a permutation of basis labels *looks like* in another basis, visible only because the register was in superposition. The inverse QFT reads the frequency, and the measurement returns $\approx 2^t s/r$ for a uniformly random s.

Why $t = 2n$ — and what happens if you skimp

The measured integer y satisfies $y/2^t \approx s/r$. To pin $r < N$ down uniquely from a rational approximation, continued fractions need the error below $1/2r^2$, and since r can be nearly N that means $2^t > N^2$, i.e. $t \geq 2n$.

Beware the $N = 15$ trap: there $r = 4$ divides 2^t exactly, so $y/2^t$ is exact for *any* $t \geq 2$ and a too-short register still works. $N = 21$ with $a = 2$ ($r = 6$, $n = 5$) is the honest test, and running it at three precisions shows the whole story:

t	qubits	recovered r	what the peaks look like
$5 = n$	17	**18** — wrong	$y/2^t$ lands on 7/20, 3/19: junk fractions
7	19	6 — correct	$21/128 \to 1/6$, $107/128 \to 5/6$
$10 = 2n$	22	6 — correct	clean peaks at $0, 171, 341, 512, 683, 853$

The $t = n$ failure is the instructive one. It did not return nonsense — it returned 18, which *passes* `pow(A, r, N) == 1` because 18 is a multiple of 6. A too-short register gives you a plausible, self-consistent, wrong answer. ($t = 7$ sufficing here is luck: the bound $t \geq 2n$ covers the worst case $r \approx N$, and you do not know r in advance.)

16 The classical half

```
def order_from_counts(counts, A, N, t):
    for bits in sorted(counts, key=counts.get, reverse=True):
        y = int(bits, 2)
        if y == 0:
            continue
        r = Fraction(y, 2**t).limit_denominator(N - 1).denominator
        if pow(A, r, N) == 1:
            return r
    return 0

def find_factor(N, run):                    # run: circuit -> counts dict
    if N % 2 == 0:
        return 2
    for k in range(2, N.bit_length() + 1):  # prime-power check
        d = round(N ** (1 / k))
        if d > 1 and d**k == N:
            return d
    import random
    while True:
        a = random.randrange(2, N)
        if (d := math.gcd(a, N)) > 1:
            return d                        # lucky gcd
        t = 2 * math.ceil(math.log2(N))
        r = order_from_counts(run(order_circuit(a, N)), a, N, t)
        if r and r % 2 == 0:
            d = math.gcd(pow(a, r // 2, N) - 1, N)
```

```
            if 1 < d < N:
                return d
```

`limit_denominator` *is* the continued-fraction expansion — Python ships the number theory. Candidates are checked with `pow(A, r, N) == 1` before being believed; $y = 0$ carries no information and is skipped.

Most of the peaks are useless — by design

The loop looks like defensive coding. It is not: **a majority of the peaks cannot give you** r, and you must expect to discard them. The measurement returns $y \approx 2^t s/r$ for uniformly random s, and continued fractions recover s/r *in lowest terms* — so what you get back is $r/\gcd(s, r)$. Only s coprime to r yields r itself.
At $N = 21$, $a = 2$ ($r = 6$, $t = 10$), the six peaks land as:

s	0	1	2	3	4	5
peak $y \approx$	0	171	341	512	683	853
$y/2^t \to$	0	1/6	1/3	1/2	2/3	5/6
recovered r	1	**6**	3	2	3	**6**
`pow` check	fails	**passes**	fails	fails	fails	**passes**

Two of six — that is $\varphi(6)/6$ — and the other four return proper divisors of r that the `pow` check rejects. This is not hypothetical: in the measured run the most frequent *nonzero* outcome was $y = 512$, which gives $r = 2$, and $\gcd(2^1-1, 21) = 1$ factors nothing. Trusting the top outcome would have failed; scanning all of them by descending frequency, as the loop does, finds $y = 171$ and $r = 6$. The $N = 33$ run repeats the lesson at larger scale: with $r = 10$ only $\varphi(10)/10 = 40\%$ of peaks work, the recovered fractions splitting cleanly into $\{\frac{1}{10}, \frac{3}{10}, \frac{7}{10}, \frac{9}{10}\}$ (accepted) and $\{\frac{1}{5}, \frac{2}{5}, \frac{3}{5}, \frac{4}{5}\}$ (rejected), and again the top nonzero outcome was one of the rejects. One caveat the check cannot catch: `pow(A, r, N) == 1` accepts any *multiple* of the true order, so a stray y can return 12 instead of 6. Harmless for factoring when the result is even, but do not treat the returned value as certainly minimal.

Retries are cheap and expected: a bad measurement re-runs the *same* circuit; only a bad base a forces rebuilding, and that happens with probability $\leq 1/2$. That bound is close to tight, and easy to see for yourself — at $N = 33$, of the 19 nontrivial bases coprime to 33, nine fail: four have odd order ($r = 5$, for $a = 4, 16, 25, 31$) and five hit $a^{r/2} \equiv -1$, where $\gcd(a^{r/2}-1, N)$ collapses to 1 ($a = 2, 8, 17, 29, 32$). Ten succeed. So 47% of bases send you back to the top of the loop with a *correct* order in hand — order finding worked, and the reduction still failed. Budget for it: this is the common case, not an error.

17 Verifying it — level by level

This is the part most write-ups omit, and it is where the time actually goes. Each level is testable in isolation on a simulator, and **a bug at level** k **is invisible at level** $k+1$ **except as noise**, so test in order. Prepare a basis state, run one primitive, measure, compare with Python.

```
def set_value(qc, val, qubits):              # prepare a computational basis state
    for i, q in enumerate(qubits):
        if (val >> i) & 1:
            qc.x(q)
```

The suite below was run against the listings in this note; the results are its actual output.

Level	Test	Result
0	`qft`: unitary vs. the definition and vs. Qiskit's `QFTGate`, $n = 1..6$	exact agreement; `.inverse()` undoes it
0	`_maj`/`_uma` truth tables, all 8 inputs (§18)	MAJ = ($c\oplus a$, $b\oplus a$, MAJ) and UMA·MAJ = (c, $a\oplus b\oplus c$, a)
1	`rc_add`: all (x, y) pairs, $n = 2..5$, control on and off	sum correct, x preserved, carry ancilla clean; exactly $2n$ Toffoli
1	`rc_add_const`: all y, negative constants, $n = 3, 4$	correct mod 2^n; all $n{+}1$ ancillas clean
2	`rc_c_add_mod`: all pairs $y, X < N$ for $N = 9, 15$, control on/off	matches the Fourier version exactly; sign, flag and scratch all clean
4	`rc_c_ua`: $N = 15$, all coprime A, all x, control on/off	$x \mapsto Ax$ mod 15; all scratch clean — the tower is adder-agnostic
0	`lookup`: every address, $m = 1, 2, 3$ (§19)	T[addr] correct; address preserved, unary ancillas clean
2	`add_quantum_mod`: all (y, t) pairs, $N = 9, 15$	$(y{+}t)$ mod N; addend preserved, sign and flag clean
3	`windowed_c_mult_acc`: $N = 9$, $w = 1, 2$, all x, control on/off	agrees with `c_mult_acc` on every input; all scratch clean
3	`run_windowed_acc_mbu` (§19.6): $N = 15$, all k, all x, control on/off	every lookup uncomputed by measurement; x preserved, $y = kx$ mod 15, scratch clean, 180 mid-circuit measurements
0	lookup uncomputed by measurement on a *random non-uniform* address superposition	address restored exactly; leaked norm 0
0	`lookup_ui` (unary iteration, §19.5): every address, both control values, $m = 1..4$	correct; exactly $2(L{-}1)$ Toffoli; m ancillas not 2^m
0	`phase_fixup` (§19.6): every address, every split $l = 0..m$, $m = 2, 3, 4$	acts as exactly diag$((-1)^{F[a]})$; ancillas clean
0	X-basis measurement identity, $L = 4, 8$	every outcome leaves exactly $(-1)^{\mathrm{parity}(m \wedge T[a])}$; output disentangled
5	`windowed_order_circuit(7,15,w=2,t=8)`, 24 qubits	peaks at $\{0, 64, 128, 192\}$, all 2048 shots; $r = 4$; $15 = 3 \times 5$ (58 s)
5	`windowed_order_circuit`, N=21, w=2, t=2n=10, 29 qubits	six peaks at multiples of 170.67 (smeared, $6 \nmid 2^t$); $r = 6$; $21 = 7 \times 3$ (4 062 s)
5	`windowed_exponentiate` (nested, §19.7): every $e < 16$, $N = 15$, $k = 7$	$\lvert e\rangle\lvert 1\rangle\lvert 0\rangle \mapsto \lvert e\rangle\lvert 7^e \bmod 15\rangle\lvert 0\rangle$; scratch clean
5	`nested_order_circuit`, N=15, w_e=w_m=2, 23 qubits	2 multiplications not 4; $r = 4$; $15 = 3 \times 5$ (28 s)
1	`add_const`: all y, several X incl. negative, $n = 3, 4$	single outcome = $(y + X)$ mod 2^n every time
2	`c_add_mod`: all $y, X < N$ for $N = 9, 15$, control on *and* off; measure sign and flag too	$(y + X)$ mod N with control on, identity with it off; **sign and flag always** $\lvert 00\rangle$
4	`c_ua`: all $x < N$, all $A \in \{2, 4, 7, 8, 11, 13\}$, $N = 15$, both control values	$x \mapsto Ax$ mod N; **all $n + 2$ scratch qubits back to** $\lvert 0\rangle$
5	`order_circuit(7,15)`, $t = 8$, 4096 shots	peaks at exactly $\{0, 64, 128, 192\}$, all 4096 shots on peaks
5	post-processing	$r = 4$ for both $a = 7$ and $a = 2$; `find_factor(15)` returns 5

$N = 15$ is too friendly to prove much, so the same suite was run at $N = 21$ ($n = 5$, $4n + 2 = 22$

qubits, orders 2, 3 and 6 — and $6 \nmid 2^t$, so continued fractions do real work):

Level	Test	Result
2	`c_add_mod`: all 441 pairs $y, X < 21$, control on and off	correct, ancillas clean, every case (7 s)
4	`c_ua`: all 11 bases coprime to 21, all $x < 21$, both control values	$x \mapsto Ax \bmod 21$; scratch clean throughout (88 s)
5	`order_circuit(2,21)`, $t = 10$, 22 qubits, depth ∼23 000	six peaks at $0, 171, 341, 512, 683, 853$ as predicted; $r = 6$; $\gcd(2^3-1, 21) = 7 \Rightarrow 21 = 7 \times 3$ (63 s)
5	`order_circuit(8,21)` ($r = 2$, exact)	two peaks, 1049 and 999 of 2048 shots; $r = 2$; again $21 = 7 \times 3$
and once more at $N = 33$ ($n = 6$, 26 qubits, depth ∼43 700)		
2	`c_add_mod`: all 1089 pairs $y, X < 33$, control on/off	correct, ancillas clean (24 s)
4	`c_ua`: all 19 bases coprime to 33, all $x < 33$	correct, scratch clean (575 s)
5	`order_circuit(5,33)`, $t = 12$, 2048 shots	ten peaks at every multiple of 409.6; $r = 10$; $\gcd(5^5-1, 33) = 11 \Rightarrow 33 = 11 \times 3$ (1373 s)
5	`order_circuit(10,33)` ($r = 2$)	$r = 2$; $\gcd(10-1, 33) = 3 \Rightarrow 33 = 3 \times 11$ (1511 s)

The $a = 2$ run is the one worth staring at. The peaks sit at $2^{10}s/6 = 170.67s$, which is *not* an integer, so the distribution smears onto neighbours (682 and 854 pick up 68 and 65 shots beside the main 683 and 853) — exactly the behaviour $N = 15$ can never show you, and exactly what the continued-fraction step exists to absorb.

The two assertions that matter most

Assert the ancillas are clean. At Levels 2 and 4, measure the scratch qubits along with the answer and assert they read $|0 \cdots 0\rangle$. A circuit can compute the right value and still be broken; this is the only cheap way to catch it before it becomes noise.

Assert the peaks, not the factors. At $t = 8$, $r = 4$, the measurement must concentrate on $y = 2^t s/r \in \{0, 64, 128, 192\}$. Do not assert "`find_factor` returned 3 or 5": the classical tail verifies its own candidates, so it can succeed by chance on a *uniform* distribution — exactly what a garbage bug produces. A published from-scratch notebook has this failure: flat histogram over all sixteen outcomes, retry loop still prints 3×5. The algorithm's robustness masks the bug. Test the distribution.

Pitfalls, in the order you will hit them

- **Endianness.** Qiskit is little-endian: `qr[0]` is the LSB, and printed bitstrings read MSB-first. Half of all bugs here are a reversed register.
- **Phase sign and scale.** `2*np.pi*X*2**(i-n)` — get the exponent wrong and you add the wrong constant, silently. Test `add_const` exhaustively; it takes seconds.
- **Negative exponents.** Plain Python is safe (`2 ** -3` is `0.125`), but if n or i arrives as a *numpy* integer — easy, if a bit width came from an array — `2 ** (i-n)` raises `ValueError: Integers to negative integer powers are not allowed`, and `1 << (i-n)` raises on any type. Write `2.0 ** (i - n)` and stop thinking about it.
- **The missing $+1$.** The value register must be $n+1$ qubits (sign qubit included) everywhere in §13. Drop it and the comparison silently reads a data bit.
- **Reducing the constant.** Pass X with $0 \le X < N$ into `c_add_mod` — hence `(A << i) % N` and `(-B) % N` at Level 3/4.
- **Simulation size.** $4n + 2$ qubits at $N = 15$ is 18 — fine. $N = 21$ is 22 qubits at depth $\sim$23 000, about a minute per run on a laptop; $N = 33$ is 26 and roughly 16$\times$ worse. This is a wall, not a slope. Test the arithmetic levels with `Statevector.from_instruction` instead of sampling — they are permutations on basis states, so one exact evolution replaces thousands of shots, and the $N = 21$ sweeps above finish in seconds.
- **Transpile once.** Run the pass manager on the circuit once and reuse it across shots.

Running it

```
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager
from qiskit_aer import AerSimulator
from qiskit_aer.primitives import SamplerV2 as Sampler

SIM, SAMPLER = AerSimulator(), Sampler(seed=7)
PM = generate_preset_pass_manager(backend=SIM, optimization_level=1)

def run(qc, shots=4096):
    return SAMPLER.run([PM.run(qc)], shots=shots).result()[0].data.out.get_counts()

counts = run(order_circuit(7, 15))           # -> peaks at 0, 64, 128, 192
print(order_from_counts(counts, 7, 15, t=8)) # -> 4
print(find_factor(15, run))                  # -> 3 or 5
```

Part V — Making it fast

Part IV's implementation is correct and about as simple as the algorithm gets. It is also slower than it needs to be in three separate ways, each with its own literature and its own trade. This part builds all three and measures them: ripple-carry arithmetic (a different gate set — the one error correction actually wants), windowing (fewer additions, paid for in qubits), and the two refinements, unary-iteration lookups and measurement-based uncomputation, that make windowing worth having.

18 The other Level 1: ripple-carry arithmetic

Everything so far rests on phase gates, and that is a liability. The smallest rotation in an n-qubit Fourier addition is $2\pi/2^{n+1}$: 0.20 rad at $n = 4$, 10^{-9} at $n = 32$, and 10^{-616} at $n = 2048$ — a number no apparatus will ever apply. Worse, arbitrary-angle rotations are not fault-tolerant gates; each must be synthesised from Clifford+T at a cost that grows with the precision demanded.

The alternative is to do addition the way hardware wants it: carries, in the computational basis, using only X, CNOT and Toffoli. Then the *Toffoli count* is the cost, because it is a proxy for the T count, and T gates are what magic-state distillation actually bills you for. This section builds Level 1 a second time. **Nothing above Level 1 changes.**

18.1 Two blocks that do everything

Schoolbook addition is $s_i = a_i \oplus b_i \oplus c_i$ with carry $c_{i+1} = \mathrm{MAJ}(a_i, b_i, c_i) = a_i b_i \oplus a_i c_i \oplus b_i c_i$. Read that as a cost statement: the sum bits are $\mathbb{F}_2$-linear, so they are CNOTs and effectively free; the carries are quadratic, so each one costs a Toffoli; and they form a chain, so depth is $\Theta(n)$.

Cuccaro, Draper, Kutin and Moulton [9] express the whole adder with two three-qubit blocks acting on a triple (c, b, a) — carry-in, addend, augend:

$$\begin{aligned} \mathrm{MAJ}: &\ \mathrm{CNOT}(a \to b),\ \mathrm{CNOT}(a \to c),\ \mathrm{TOF}(c, b \to a); \\ \mathrm{UMA}: &\ \mathrm{TOF}(c, b \to a),\ \mathrm{CNOT}(a \to c),\ \mathrm{CNOT}(c \to b). \end{aligned}$$

MAJ sends $(c, b, a) \mapsto (c \oplus a,\ b \oplus a,\ \mathrm{MAJ}(a, b, c))$: it computes the carry *into the wire that held* a, which is why no ancilla per carry is needed. Running the chain forward computes all n carries; running UMA back down undoes them. And here is the point of the design:

$$\mathrm{UMA} \cdot \mathrm{MAJ} : (c, b, a) \mapsto (c,\ a \oplus b \oplus c,\ a)$$

— UMA **erases the carry and writes the sum bit in the same three gates**. The uncomputation that §12's no-garbage rule makes compulsory is exactly the pass that delivers the answer. You are not paying twice; you are paying once and collecting on the way out.

```
def _maj(qc, c, b, a, ctrls=()):
    """MAJ: |c,b,a> -> |c^a, b^a, MAJ(a,b,c)>.  Only the b-target takes ctrls."""
    qc.mcx([*ctrls, a], b)              # plain CNOT when ctrls is empty
    qc.cx(a, c)
    qc.ccx(c, b, a)

def _uma(qc, c, b, a, ctrls=()):
    """UMA: undoes MAJ's carry and writes the sum bit into b."""
    qc.ccx(c, b, a)
    qc.cx(a, c)
    qc.mcx([*ctrls, c], b)              # plain CNOT when ctrls is empty

def rc_add(qc, x, y, carry, ctrls=()):
    """|x>|y> -> |x>|(y + x) mod 2^n>.  carry: one clean ancilla, returned |0>."""
    n = len(y)
    _maj(qc, carry, y[0], x[0], ctrls)
    for i in range(n - 1):
        _maj(qc, x[i], y[i + 1], x[i + 1], ctrls)
    for i in range(n - 1, 0, -1):
        _uma(qc, x[i - 1], y[i], x[i], ctrls)
    _uma(qc, carry, y[0], x[0], ctrls)
```

That is $2n$ Toffolis, $4n$ CNOTs and one ancilla, with x preserved and y overwritten by the sum — verified exhaustively below. The interleaved wire order you see in the literature $(c_0, b_0, a_0, b_1, a_1, \dots)$ is only a seating plan that keeps each block's three qubits adjacent; it means nothing mathematically.

18.2 Adding a classical constant

CDKM adds two *quantum* registers, but Level 2 wants "add the known constant X". The cheapest honest fix: write X into n clean ancillas with X gates, run the adder, and unwrite it. That is where the ripple-carry route's extra qubits go.

```
def rc_add_const(qc, X, y, anc, ctrls=()):
    """|y> -> |y + X mod 2^n>.  anc: n+1 clean ancillas, returned clean."""
    n = len(y)
    carry, xreg = anc[0], anc[1:n + 1]
    Xm = X % (2**n)                      # negative X just works, as in Fourier
    for i in range(n):                   # encode the constant -- uncontrolled
        if (Xm >> i) & 1:
            qc.x(xreg[i])
    rc_add(qc, xreg, y, carry, ctrls)
    for i in range(n):                   # unencode
        if (Xm >> i) & 1:
            qc.x(xreg[i])
```

Negative constants work for the same reason as in §12 — `X % 2**n` is the two's complement — so subtraction again needs no separate circuit. With this in place the modular adder of §13 is reproduced *line for line*, only the adder calls changing:

```
def rc_c_add_mod(qc, ctrls, X, y, sign, flag, N, anc):
    yw = list(y) + [sign]
    rc_add_const(qc, X, yw, anc, ctrls)  # (1) + X          (controlled)
    rc_add_const(qc, -N, yw, anc)        # (2) - N
    qc.cx(sign, flag)                     # (3) sign -> flag
    rc_add_const(qc, N, yw, anc, [flag])  # (4) + N  if flag
    rc_add_const(qc, -X, yw, anc)        # (5) - X
    qc.mcx(list(ctrls) + [sign], flag)    # (6) clear flag from the answer
    qc.x(flag)
    rc_add_const(qc, X, yw, anc)         # (7) + X
```

And Levels 3–5 are then genuinely unchanged — same loops, same constants, same modular inverse, with only the extra ancilla register threaded through:

```
def rc_c_mult_acc(qc, c, A, x, y, sign, flag, N, anc):
    for i in range(len(x)):
        rc_c_add_mod(qc, [c, x[i]], (A << i) % N, y, sign, flag, N, anc)

def rc_c_ua(qc, c, A, x, y, sign, flag, N, anc):
    rc_c_mult_acc(qc, c, A % N, x, y, sign, flag, N, anc)
    for i in range(len(x)):
        qc.cswap(c, x[i], y[i])
    B = pow(A, -1, N)                        # classical inverse, unchanged
    rc_c_mult_acc(qc, c, (-B) % N, x, y, sign, flag, N, anc)
```

Checked exhaustively at $N = 15$ (17 qubits): every base coprime to 15, every $x < 15$, control on and off, $x \mapsto Ax \bmod 15$ correct and all $n{+}4$ scratch qubits returned to $|0\rangle$. The abstraction really does hold — swap the adder, keep the tower.

The control trick, again — and why it telescopes

Look at `_maj` and `_uma`: of their six gates, exactly *two* accept the outer control, and both are the CNOT targeting the b wire. Everything else is unconditional. The justification is the residue argument of §13, and here it is unusually clean.

With the control off, what survives of MAJ is $[\mathrm{CNOT}(a\to c),\ \mathrm{TOF}(c,b\to a)]$ and what survives of UMA is $[\mathrm{TOF}(c,b\to a),\ \mathrm{CNOT}(a\to c)]$ — the same two gates in the opposite order, i.e. each other's inverse. Since the UMA chain retraces the MAJ chain in reverse, the residues cancel pairwise from the inside out and the whole adder telescopes to the identity.

So a controlled n-bit addition costs $2n$ controlled gates rather than $6n$ — and the constant encoding stays uncontrolled too, since encode/unencode brackets an identity. Verified exhaustively for $n = 2..5$ with the control both on and off.

18.3 What it costs, measured

Both backends, the same doubly-usable modular adder, same modulus, counted before and after transpiling to $\{\mathrm{CNOT}, U\}$:

N	backend	qubits	rotations	Toffoli	CNOT	depth
15 (n=4)	Fourier	7	125	0	287	492
	ripple-carry	13	**0**	71	437	578
63 (n=6)	Fourier	9	245	0	545	948
	ripple-carry	17	**0**	99	637	**844**

Read the trend, not the absolute numbers. Fourier rotations grow as $O(n^2)$ per addition ($125 \to 245$) because of the QFT sandwich; ripple-carry Toffolis grow as $O(n)$ ($71 \to 99$). By $n = 6$ the ripple-carry circuit is already *shallower*, and the gap widens from there. What you buy is a factor of n in gate count and a gate set that survives error correction; what you pay is $n + 2$ extra qubits, taking the full order-finding circuit from $4n + 2$ to $5n + 4$.

Which to use

For *simulating* small N on a laptop, Fourier wins: fewer qubits, and simulation cost is exponential in qubit count, so $4n + 2$ versus $5n + 4$ is the whole ballgame.

For anything aimed at real fault-tolerant hardware, ripple-carry wins and it is not close. Zero arbitrary-angle rotations, a gate count lower by $O(n)$, and a cost model (T count) that actually predicts the resource bill. The 10^{-616} rad rotation is not a rounding problem you can engineer around — it is a statement that Fourier arithmetic at cryptographic sizes must be approximated before it can be attempted at all.

19 Windowing: trading lookups for additions

Level 3 spends one modular addition per bit of x — n of them, each controlled twice. Gidney's windowing [21] removes most of them by the classical trick of merging iterations into a table lookup. Unlike the control placement of §13, which is a fixed constant factor, this one *improves with* n — it is the optimisation that changes the exponent's logarithm, and the reason published Toffoli counts are what they are.

19.1 The idea in three steps

Step 1: a controlled addition is a lookup. Adding the constant k into y controlled on one qubit q is the same as adding $T[q]$ where $T = [0, k]$ — a two-entry table addressed by q. Nothing has changed yet.

Step 2: address more than one qubit. If a two-entry table works, so does a 2^w-entry one. Take w bits of x at a time; the value they contribute to kx is $k \cdot (\text{window value}) \cdot 2^i \bmod N$, which is classical — so tabulate all 2^w possibilities, look up the one the register actually holds, add it *once*, then uncompute the lookup. One addition per *window* instead of one per bit: $\lceil n/w \rceil$ instead of n.

Step 3: fold the control into the address. The outer control c becomes address bit 0, with every $c = 0$ entry set to 0 (Gidney Fig. 5). Adding zero modulo N is the identity, so the modular addition needs *no control at all*. This is why windowing composes so well with the ladder: the expensive part stops being controlled.

The cost of a window is one lookup instead of $w - 1$ extra additions, so the Toffoli count goes as $O\big(\frac{n}{w}(n + 2^w)\big)$. Setting $w = \lg n$ balances the two terms and gives $O(n^2/\lg n)$ — a logarithmic factor better than schoolbook.

19.2 What has to be built

Two new primitives. First a **QROM**: read a classical table at a quantum address. Encode the address one-hot, and the data load becomes pure CNOTs — one per set bit of the table.

```
def _unary_gate(m):
    """binary(m qubits) -> one-hot(2^m qubits), using 2^m - 1 Fredkins."""
    addr, unary = QuantumRegister(m, "a"), QuantumRegister(1 << m, "u")
    qc = QuantumCircuit(addr, unary)
    qc.x(unary[0])
    for i in reversed(range(m)):                    # MSB down to LSB
        step = 1 << i
        for j in range(0, 1 << m, 2 * step):
            qc.cswap(addr[i], unary[j], unary[j + step])
    return qc.to_gate(label="unary")

def lookup(qc, addr, out, table, unary):
    """|addr>|out> -> |addr>|out ^ table[addr]>; unary ancillas returned clean."""
    g = _unary_gate(len(addr))
    qc.append(g, list(addr) + list(unary))
    for j in range(len(unary)):
        Tj = table[j] if j < len(table) else 0
        for b in range(len(out)):
            if (Tj >> b) & 1:
                qc.cx(unary[j], out[b])
    qc.append(g.inverse(), list(addr) + list(unary))
```

Second, the lookup produces a *quantum* value, so Level 2 now needs to add a register rather than a constant. That is a smaller change than it sounds: the phase ramp of §12 just takes its rotations from the bits of t instead of the bits of a Python integer, and Beauregard's seven blocks are otherwise untouched.

```
def add_quantum(qc, a, y, sign=+1):
    """y += sign*a (mod 2^len(y)); a and y both quantum.  The quantum twin of
    add_const: same phase ramp, each rotation controlled by a bit of a."""
    n = len(y)
```

```
    qc.append(qft(n), y)
    for i in range(len(a)):
        for j in range(n):
            if i + j < n:                          # higher terms are trivial
                qc.cp(sign * 2 * np.pi * 2.0 ** (i + j - n), a[i], y[j])
    qc.append(qft(n).inverse(), y)

def add_quantum_mod(qc, t, y, sign, flag, N):
    """|y>|t> -> |(y+t) mod N>|t>.  Blocks (1),(5),(7) go quantum; rest as-is."""
    yw = list(y) + [sign]
    add_quantum(qc, t, yw, +1)            # (1) + t
    add_const(qc, -N, yw)                 # (2) - N
    qc.cx(sign, flag)                     # (3) sign -> flag
    c_add_const(qc, [flag], N, yw)        # (4) + N  if flag
    add_quantum(qc, t, yw, -1)            # (5) - t
    qc.cx(sign, flag)                     # (6) [R < t] == not flag, so this
    qc.x(flag)                            #     pair returns flag to |0>
    add_quantum(qc, t, yw, +1)            # (7) + t
```

With those, Level 3 becomes a loop over windows rather than over bits. It calls `lookup_ui` rather than the `lookup` above: same operation, but built by unary iteration so it needs w ancillas instead of 2^w — see §19.5, which is worth reading before you build anything. The one-hot version above is the easier way to see what a QROM *is*; it is not the one to ship.

```
def windowed_c_mult_acc(qc, c, k, x, y, sign, flag, N, t, anc, w):
    """Drop-in for c_mult_acc, but ceil(len(x)/w) modular additions instead of
    len(x).  c is the top-level control of each lookup, so when c=0 no data
    loads at all and the modular addition -- which is uncontrolled -- adds 0.

    t:   len(y) clean qubits holding one looked-up value (returned clean)
    anc: w clean qubits for the unary-iteration QROM     (returned clean)
    w:   window size -- bits of x consumed per lookup
    """
    n = len(y)
    for i in range(0, len(x), w):
        win = list(x[i:i + w])
        # window value j contributes (k * j * 2^i) mod N
        table = [((k * j) << i) % N for j in range(2**len(win))]
        lookup_ui(qc, c, win, t[:n], table, anc[:len(win)])
        add_quantum_mod(qc, t[:n], y, sign, flag, N)
        lookup_ui(qc, c, win, t[:n], table, anc[:len(win)])   # uncompute
```

Level 4 and the ladder then need no new ideas at all — same multiply/swap/uncompute, same classical inverse, with the lookup scratch threaded through:

```
def windowed_c_ua(qc, c, A, x, y, sign, flag, N, t, anc, w):
    """|c>|x>|0> -> |c>|A^c x mod N>|0>.  Same multiply/swap/uncompute as
    c_ua; only the accumulate is windowed."""
    windowed_c_mult_acc(qc, c, A % N, x, y, sign, flag, N, t, anc, w)
    for i in range(len(x)):
        qc.cswap(c, x[i], y[i])
    B = pow(A, -1, N)                          # classical inverse, unchanged
    windowed_c_mult_acc(qc, c, (-B) % N, x, y, sign, flag, N, t, anc, w)
```

Why the $c = 0$ entries being zero is the whole trick

Set $c = 0$ and every table entry is 0, so the lookup writes $|0\rangle$ into t and block (1) adds nothing. Trace the seven blocks with $t = 0$: (2) drives the register negative, so the sign bit sets and (3) sets the flag; (4) adds N back; (5) subtracts nothing, leaving $R = y \geq 0$ so the sign bit is now 0; and (6) computes flag $\oplus$ sign $= 1$, which the X clears. Identity, ancilla clean — *without a single controlled gate in the adder*. The control survives only as a classical fact about the table.

19.3 What it costs, measured

One accumulate $y \mathrel{+}= c{\cdot}kx \bmod N$, transpiled to $\{\mathrm{CNOT}, U\}$:

method	mod. adds	qubits	CNOT	depth	vs. depth
$N = 15$, $n = 4$					
shift-and-add	4	11	1 254	2 119	—
window $w = 2$	2	23	1 184	1 128	1.9×
$N = 63$, $n = 6$					
shift-and-add	6	15	3 474	6 001	—
window $w = 3$	2	37	2 412	2 082	2.9×
$N = 255$, $n = 8$					
shift-and-add	8	19	7 390	13 020	—
window $w = 3$	3	43	4 646	4 873	2.7×
window $w = 4$	2	59	4 432	3 428	3.8×

The addition count falls exactly as $\lceil n/w \rceil$ predicts, and — unlike many asymptotic optimisations — the win is already real at $n = 4$ and grows: 1.06×, 1.44×, 1.67× in CNOTs at $n = 4, 6, 8$, and far more in depth, because the lookups for one window are shallow and the additions they replace were not. What you pay is qubits.

The same holds for the *whole* order-finding circuit, not just one accumulate. At $N = 15$, with the unary-iteration QROM of §19.5, windowing halves the depth for six extra qubits:

full circuit, $N = 15$	qubits	CNOT	depth
plain, $t = 4$	14	10 212	17 239
windowed $w = 2$, $t = 4$	20	7 122	**8 488**
plain, $t = 2n = 8$	18	20 462	34 537
windowed $w = 2$, $t = 2n = 8$	24	14 276	**17 028**

19.4 End to end

Windowed arithmetic factors 15 with nothing else changed. With the unary-iteration QROM of §19.5 the full-precision circuit — $t = 2n = 8$, $w = 2$ — fits in 24 qubits and gives a textbook distribution:

y	shots	$y/2^t$	continued fraction
0	530	0	—
64	490	1/4	$r = 4$ ✓
128	508	1/2	$r = 2$, rejected
192	520	3/4	$r = 4$ ✓

All 2048 shots land on the four ideal peaks, $r = 4$ is recovered, and $\gcd(7^2-1, 15) = 3$ gives $15 = 3\times5$; base $a = 2$ likewise. Note that even in this clean run the $y = 128$ peak returns $r = 2$ and is discarded — §16's scan is doing work here too.

It is worth seeing what §19.5 bought at this level. Built on the one-hot QROM the same circuit needed 30 qubits at $t = 2n$ — 16 GB of statevector, out of reach — so it had to be run at $t = 4$ instead, which is sound at $N = 15$ only because $r = 4$ divides 2^4. Unary iteration turned a compromise into the real thing, and made it faster: 58 s against 100 s.

$N = 15$ is too forgiving to prove much, so the windowed circuit was also run on $N = 21$, where the order is 6 and $6 \nmid 2^t$, so the continued-fraction step has to do real work. At the full $t = 2n = 10$ (29 qubits, depth 14 510, 8 GB of statevector, 68 minutes) it lands on all six ideal peaks $2^{10}s/6 = 170.67\,s$:

y	0	171	341	512	683	853	682	854
shots	326	223	247	337	221	234	74	72
→	—	1/6	1/3	1/2	2/3	5/6	2/3	5/6

— the last two columns being the smearing onto neighbours that a non-integer $2^t/r$ forces, and which $N = 15$ can never show. It recovers $r = 6$, and $\gcd(2^3-1, 21) = 7$ gives $21 = 7 \times 3$. Only 1/6 and 5/6 pass the `pow` check (§16); the other four peaks return divisors of 6 and are discarded. Windowed arithmetic is not doing anything special at $N = 15$; it is just doing arithmetic.

Nested windowing (§19.7) factors 15 too, on 23 qubits at $t = 4$, using *two* modular multiplications where the plain ladder uses four: peaks at $\{0, 4, 8, 12\}$, $r = 4$, $15 = 3 \times 5$, for both $a = 7$ and $a = 2$.

19.5 Making the lookup cheap: unary iteration

The one-hot QROM above is correct but wasteful: it materialises all 2^m address states at once, so it needs 2^m ancillas. Gidney's Fig. 2 walks the address *tree* instead, holding only the AND of the current prefix [17]. Recursively: split on the top address bit, compute node = ctrl $\wedge\, a$ to select the upper half, flip it with one CNOT to get ctrl $\wedge\, \neg a$ for the lower half, then clear it.

```
def _walk(qc, ctrl, addr, out, table, offset, anc):
    """out ^= table[offset + value(addr)], all conditioned on ctrl."""
    if not addr:
        T = table[offset] if offset < len(table) else 0
        for b in range(len(out)):
            if (T >> b) & 1:
                qc.cx(ctrl, out[b])           # leaf: data load is CNOTs
        return
    a, rest = addr[-1], addr[:-1]             # split on the most significant bit
    half, node = 1 << len(rest), anc[0]
    qc.ccx(ctrl, a, node)                     # node = ctrl AND a      (upper half)
    _walk(qc, node, rest, out, table, offset + half, anc[1:])
    qc.cx(ctrl, node)                         # node = ctrl AND NOT a  (lower half)
    _walk(qc, node, rest, out, table, offset, anc[1:])
    qc.cx(ctrl, node)                         # put node back to ctrl AND a ...
    qc.ccx(ctrl, a, node)                     # ... and clear it

def lookup_ui(qc, ctrl, addr, out, table, anc):
    """|addr>|out> -> |addr>|out ^ table[addr]> when ctrl is 1, else identity.
    anc: len(addr) clean ancillas, returned clean.  2(L-1) Toffolis."""
    _walk(qc, ctrl, list(addr), list(out), list(table), 0, list(anc))
```

The recursion depth is m, so it needs m ancillas, not 2^m — and measured Toffoli counts land exactly on $2(L-1)$: $2, 6, 14, 30$ for $m = 1, 2, 3, 4$, against $2, 4, 8, 16$ ancillas saved down to $1, 2, 3, 4$. (With the temporary-AND trick [16] — built in §20 — the uncompute Toffolis become Clifford, giving quoted $L - 1$.) Note this also replaces the control-folding of §19: c is now simply the top-level `ctrl`, so the tree halves as well.

Swapping it in shrinks the windowed circuit substantially — and, unlike the one-hot version, the qubit cost is now barely sensitive to w:

one accumulate, $N = 255$ ($n = 8$)	mod. adds	qubits	CNOT	depth
shift-and-add	8	19	7 390	13 020
window $w = 2$	4	29	4 840	6 497
window $w = 3$	3	30	3 902	4 873
window $w = 4$	2	31	**3 264**	**3 866**

With one-hot at $w = 4$ the same circuit needed 59 qubits; unary iteration brings it to 31, and the CNOT advantage over shift-and-add improves from 1.67× to 2.26× (depth 3.4×). The earlier tables in §19 are the one-hot numbers; these are the ones to build on.

19.6 Uncomputing a lookup for free

We still uncompute each lookup by running it again, at equal cost — and that is avoidable, because the output is about to be discarded. Measure every output qubit in the X basis. On $\sum_a \alpha_a|a\rangle|T[a]\rangle$ this yields a uniformly random outcome m and leaves

$$\sum_a (-1)^{\,\mathrm{parity}(m \wedge T[a])}\, \alpha_a|a\rangle \;\otimes\; |m\rangle,$$

so the output register is disentangled *for free*; all that survives is a phase. Removing it is a phase-only lookup, and *that* can be done in $O(\sqrt{L})$: split the address into l low bits and h high bits, one-hot the low half (2^l Fredkins), then unary-iterate the high half applying CZ into the one-hot register ($2(2^h-1)$ Toffolis). With $l = h = m/2$ the total is $O(\sqrt{L})$ instead of $O(L)$.

```
def phase_fixup(qc, addr, F, one, unary_lo, anc_hi, l):
    """Apply the phase (-1)^F[a] to every basis state |a> of addr, in O(sqrt(L))."""
    lo, hi = list(addr[:l]), list(addr[l:])
    g = _unary_gate(l)
    qc.append(g, lo + list(unary_lo[:1 << l]))
    qc.x(one)                                          # always-true control
    _walk_cz(qc, one, hi, list(unary_lo[:1 << l]), F, 0, list(anc_hi))
    qc.x(one)
    qc.append(g.inverse(), lo + list(unary_lo[:1 << l]))
```

Verified two ways: `phase_fixup` acts as exactly $\mathrm{diag}((-1)^{F[a]})$ with all ancillas clean, for every split $l = 0 \ldots m$ at $m = 2, 3, 4$; and the measurement identity above holds for *every* outcome m (checked amplitude by amplitude at $L = 4, 8$). At $m = 4$ the $\sqrt{L}$ split costs 8 where a direct phase lookup costs 16.

One caveat on the measured costs elsewhere in this section: the CNOT and depth figures in §19–19.7 were taken from the *static* circuits, which uncompute by re-running the lookup. They are therefore a **lower bound** on what windowing delivers — the hybrid version below is cheaper still, by the factors in the table.

19.7 Nesting: windowing the exponent too

Everything so far windows the *multiplicand*. Gidney's headline construction (§3.5 of [21]) windows the *exponent* as well, and that is where the second logarithmic factor comes from. Take w_e counting qubits at a time and let the lookup select the factor $k^{v\cdot 2^i}$ for the window value v; the address becomes the exponent-window bits followed by the multiplicand-window bits, so one table serves both.

Three things fall out at once. There are $\lceil t/w_e \rceil$ multiplications instead of t. The multiplications need *no control at all* — when the exponent window reads 0 the factor is $k^0 = 1$, and multiplying by 1 is the identity. And the swap between accumulators becomes a *relabelling* in the classical loop, so the n Fredkin gates of §14 vanish.

```
for i in range(0, len(e), we):
    ei = list(e[i:i + we])
    ne = len(ei)
    kes = [pow(k, (2**i) * v, N) for v in range(2**ne)]
    kes_inv = [pow(v, -1, N) for v in kes]

    # B += A * k_e   (mod N):  maps (x, 0) -> (x, x*k_e)
    for j in range(0, n, wm):
        mi = A[j:j + wm]
        addr = ei + list(mi)
        table = [((kes[ad & ((1 << ne) - 1)] * (ad >> ne)) << j) % N
                 for ad in range(1 << len(addr))]
        lookup_ui(qc, one, addr, tmp[:n], table, anc[:len(addr)])
        qc.append(madd, tmp[:n] + B + [sign, flag])
        lookup_ui(qc, one, addr, tmp[:n], table, anc[:len(addr)])

    # A -= B * k_e^{-1} (mod N): maps (x, x*k_e) -> (0, x*k_e)
    for j in range(0, n, wm):
        mi = B[j:j + wm]
        addr = ei + list(mi)
        table = [((kes_inv[ad & ((1 << ne) - 1)] * (ad >> ne)) << j) % N
                 for ad in range(1 << len(addr))]
        lookup_ui(qc, one, addr, tmp[:n], table, anc[:len(addr)])
        qc.append(msub, tmp[:n] + A + [sign, flag])
        lookup_ui(qc, one, addr, tmp[:n], table, anc[:len(addr)])

    A, B = B, A                                  # relabelling swap -- free
```

Verified on the ladder identity directly: $|e\rangle|1\rangle|0\rangle \mapsto |e\rangle|k^e \bmod N\rangle|0\rangle$ for *every* exponent $e < 2^4$ at $N = 15$, $k = 7$, with all scratch returned to $|0\rangle$. Measured on the full circuit at $t = 2n = 8$:

$N = 15$, $t = 2n = 8$	mults	qubits	CNOT	depth
plain shift-and-add	8	18	20 462	34 537
windowed $w_m = 2$	8	24	14 276	17 028
nested $w_e = 2$, $w_m = 2$	4	27	**11 908**	**14 229**
nested $w_e = 4$, $w_m = 2$	2	29	16 292	20 095

1.72× the CNOTs and 2.43× the depth of plain shift-and-add, and a further 1.2× on top of single-level windowing. The last row is the useful warning: at $n = 4$, $w_e = 4$ makes the table 2^6 entries and the lookup costs more than the two multiplications it saved. Windows have an optimum ($w \approx \lg n$), and overshooting is worse than not windowing that level at all.

19.8 What of the paper is actually here

Windowing is a family of constructions, and it is worth being exact about which of them this note builds, since partial implementations of it are easy to mistake for the whole thing.

[21]	status	where / why not
§2, Fig. 2 table lookup by unary iteration	**built**	§19.5, `lookup_ui`
§3.3 windowed modular product addition	**built**	§19, `windowed_c_mult_acc`
§3.4 windowed modular multiplication	**built**	§19, `windowed_c_ua`
§3.5 nested exponent + multiplication windows	**built**	§19.7, `windowed_exponentiate`
§2, Fig. 3 measurement-based uncomputation	**built**	§19.6, `run_windowed_acc_mbu` — hybrid, so it runs as circuit/measure/classical/circuit
§3.2 windowed in-place multiplication	**built**	`windowed_inplace.times_equal`, $x \mathrel{*}= k \bmod 2^L$, exact on every input; the modular ladder still multiplies out-of-place and swaps (Fig. 6)

Wiring it in: the classical controller sits in the middle

The fixup table $F[a] = \text{parity}(m \wedge T[a])$ depends on the measurement outcome, so this cannot be one static circuit — it is genuinely *circuit* → *measure* → *classical* → *circuit*, which is exactly how it runs on hardware. `run_windowed_acc_mbu` does that: it emits a stage, measures, builds F in Python, emits the fixup, and continues.

Verified by wiring it into the real windowed accumulate: every k, every $x < 15$, control on and off, across 180 mid-circuit measurements — x preserved, $y = kx \bmod 15$, all scratch back to $|0\rangle$. Also checked on a lookup addressed by a *non-uniform random* superposition, where any residual phase would show: the address register is restored exactly, with leaked norm 0.

w	$L = 2^w$	recompute	measured	saving
2	4	6	3	2.0×
4	16	30	9	3.3×
8	256	510	45	11.3×
11	2048	4 094	157	26.1×

$2(L-1)$ against $2^l - 1 + 2(2^h - 1)$ with $l = h = w/2$. Uncomputation stops being half the lookup cost and becomes a rounding error — at the $w = \lg n \approx 11$ of a cryptographic run, 26× cheaper, which turns the compute/uncompute pair from 2× a lookup into 1.04×.

20 Emitting temporary ANDs

Every Toffoli in the unary-iteration walk of §19.5 is one half of a compute/uncompute pair — the tree computes ctrl ∧ a, uses it, and clears it. Gidney's observation [16] is that such a pair need not cost two Toffolis. Computing an AND *into a qubit known to be* $|0\rangle$ costs 4 T rather than 7, and *uncomputing* it costs none at all: measure the ancilla in the X basis and repair the phase with a Clifford CZ.

```
def and_compute(qc, a, b, t):
    # t must start in |0>; leaves t = a AND b using 4 T gates, no phase
    qc.h(t); qc.t(t)
```

```
    qc.cx(b, t); qc.tdg(t)
    qc.cx(a, t); qc.t(t)
    qc.cx(b, t); qc.tdg(t)
    qc.h(t); qc.sdg(t)

def and_uncompute(qc, a, b, t, cbit):
    # erase t = a AND b for free: X-basis measurement + Clifford repair
    qc.h(t)
    qc.measure(t, cbit)
    with qc.if_test((cbit, 1)):
        qc.cz(a, b)
        qc.x(t)                                    # return the ancilla to |0>
```

The final gate is $S^\dagger$, not S; three plausible variants of this circuit differ only in that phase and are all wrong, which a truth table alone will not catch — the check has to include the phase. One classical bit suffices for the whole lookup, since each outcome is consumed by the CZ immediately after it is produced and may then be overwritten.

Substituting the pair into the walk gives `lookup_ui_ta`, verified to compute the same map on every address for $m = 1, 2, 3$ with both control values and all ancillas clean. Transpiled to Clifford+T:

m	L	T (plain)	T (temporary AND)	saving
2	4	42	12	3.50×
3	8	98	28	3.50×
4	16	210	60	3.50×
5	32	434	124	3.50×

Exactly 7/2, and for the reason you would predict: half the Toffolis were uncomputes and are now free, and the other half fell from 7 T to 4. Since the T count is what magic-state distillation bills, this is the cheapest real saving in the note — a few lines of code, no change to any interface, and it applies to every lookup in Part V.

21 One counting qubit: $4n + 2 \to 2n + 3$

The counting register is the largest in the circuit and it is barely used: each of its qubits is Hadamarded, controls one rung, and is measured. Worse, in the inverse QFT every controlled rotation is controlled by a qubit that is *about to be measured anyway*. So measure it first and make the control classical — the semiclassical Fourier transform of Griffiths and Niu [13], applied to factoring by Mosca–Ekert [14], Parker–Plenio [15] and Beauregard [8]. The counting register then never exists all at once: one qubit is Hadamarded, drives its rung, receives phase corrections conditioned on every previously measured bit, is measured, and is reset. The loop knows nothing about modular arithmetic — it takes the rungs as callbacks — so it lives in `semiclassical.py` and the elliptic-curve circuit of Part VII reuses it unchanged:

```
def semiclassical_iqft(qc, ctrl, out, rungs):
    t = len(rungs)
    assert len(out) >= t, "one classical bit per rung"
    for i in range(t):
        qc.h(ctrl)
        rungs[t - 1 - i](qc, ctrl)                 # heaviest rung first
        for j in range(i):                         # the inverse QFT, classically
            with qc.if_test((out[j], 1)):
```

```
                qc.p(-math.pi / 2 ** (i - j), ctrl)
        qc.h(ctrl)
        qc.measure(ctrl, out[i])
        with qc.if_test((out[i], 1)):                # reset for the next rung
            qc.x(ctrl)


def order_circuit_1c(A, N, t=None):
    n = math.ceil(math.log2(N))
    t = t if t is not None else 2 * n
    ctr = QuantumRegister(1, "ctr")                  # the whole counting register
    tgt = QuantumRegister(n, "tgt")
    anc = QuantumRegister(n, "anc")
    sf = QuantumRegister(2, "sf")                    # sign, flag
    out = ClassicalRegister(t, "out")
    qc = QuantumCircuit(ctr, tgt, anc, sf, out)
    qc.x(tgt[0])                                     # target = |1>

    def rung(k):                                     # controlled U^(2^k)
        return lambda qc, c: c_ua(qc, c, pow(A, 2 ** k, N),
                                  list(tgt), list(anc), sf[0], sf[1], N)
    semiclassical_iqft(qc, ctr[0], out, [rung(k) for k in range(t)])
    return qc
```

$1 + n + n + 2 = 2n + 3$, the count in the title of Beauregard's paper. Measured:

N	qubits	plain	time	result
15 ($a = 7$, $a = 2$)	**11**	18	6–7 s	$r = 4$, $15 = 3 \times 5$
21 ($a = 2$)	**13**	22	48 s	$r = 6$, $21 = 7 \times 3$

Three things that bite

The bit order is not what you expect. With the rungs run most-significant-exponent first, the *first* measured bit is the *least* significant bit of the phase — which happens to be Qiskit's own bit order, so `order_from_counts` reads this circuit unchanged. Assuming the reverse (as the author first did) silently yields $r = 0$ at $N = 15$; only testing both orderings against a known r settles it.

Fewer qubits does not mean faster simulation. Mid-circuit measurement forces Aer to simulate shot by shot rather than evolve one statevector, so every shot is paid for individually. On hardware this is the cheapest circuit in this note; in simulation part of the qubit saving is given back. It is still a large net win — $N = 21$ in 48 s against roughly 1 000 s for the 22-qubit version.

It is incompatible with exponent windowing. Windowing w_e exponent bits (§19.7) needs those w_e qubits to coexist as lookup address bits; this circuit keeps exactly one alive. The two are ends of one spectrum — counting qubits $= w_e$ — and you cannot have both. Windowing *costs* qubits (24 at $N = 15$) where this *saves* them (11).

22 Coset representation: deleting the modular adder

Everything so far pays five constant additions and a flag qubit for each modular addition (§13), and measurement says that machinery costs 5.2–5.5× the addition it wraps — flat in n, from $N = 15$ to

$N = 1023$. It exists for one reason: to turn addition mod 2^m, which the register does natively, into addition mod N.

Zalka's coset representation [18], as used by Gidney and Ekerå [19], removes the need. Instead of storing k as $|k\rangle$, store the periodic superposition

$$|k\rangle_{\text{coset}} \;=\; 2^{-c_{\text{pad}}/2} \sum_{j=0}^{2^{c_{\text{pad}}}-1} |jN + k\rangle,$$

in a register widened by c_{pad} padding qubits above the high bits. Every branch is congruent to k modulo N, so a *plain, non-modular* addition of a already produces a state whose every branch is congruent to $k + a$. When $k + a \geq N$ the sum simply re-indexes,

$$\sum_j |jN + (k{+}a)\rangle \;=\; \sum_{j'=1}^{2^{c_{\text{pad}}}} |j'N + (k{+}a{-}N)\rangle,$$

which differs from the ideal coset state of $(k{+}a)$ mod N in only the two edge terms out of $2^{c_{\text{pad}}}$. **That is the entire approximation.** Level 2 collapses into Level 1, and the sign and flag qubits go with it:

```
def add(qc, X, reg, N=None):
    """Modular addition -- as a PLAIN addition.  This is the entire point:
    no compare, no subtract, no restore, no flag qubit, no sign qubit."""
    if N is not None:
        X %= N
    add_const(qc, X, list(reg))
```

22.1 The deviation law, measured

Gidney and Ekerå bound the deviation per addition by $n/(c_{\text{sep}}2^{c_{\text{pad}}})$, subadditive under composition, so a sequence of A additions deviates by at most $A\,2^{-c_{\text{pad}}}$ when there are no carry runways. Measured, with $A = 8$ random additions:

N	$2^n - N$	c_{pad}	measured error	bound $A/2^{c_{\text{pad}}}$
15	1	2	0.749	2.000
		3	0.373	1.000
		4	0.129	0.500
		5	0.031	0.250
		6	0.000	0.125
31	1	2	0.507	2.000
		4	0.061	0.500
		6	0.000	0.125

The error halves with each padding qubit, exactly as advertised, and every measurement sits below the bound — which is loose by 2–4×, as a worst-case bound should be.

Where the approximation actually bites

A first attempt to measure this at $N = 21$ returned *zero* error at every padding size, which looks like a broken test and is not. The register holds $m = n + c_{\text{pad}}$ bits, and the headroom above the top branch is $H = 2^{c_{\text{pad}}}(2^n - N) + N - 1 - k$. For $N = 21$, $2^n - N = 11$, so the headroom is enormous and no branch ever wraps. The approximation only bites when the accumulated offset exceeds H, i.e. when N sits close to 2^n — which is why the table above uses $N = 15$ and $N = 31$, where $2^n - N = 1$. **Choosing a friendly modulus hides the very effect you are trying to measure.**

22.2 What it saves

One addition, coset versus the seven-block modular adder:

adder	$N = 15$	$N = 63$	$N = 255$	$N = 1023$
Fourier (CNOT ratio)	1.7×	2.2×	2.5×	2.8×
ripple-carry (Toffoli ratio)	3.0×	3.1×	3.5×	3.9×

Less than the naive 5.3×, because the coset register is c_{pad} qubits wider and the adder must pay for that width; and better with ripple-carry, whose cost is linear in width, than with Fourier, whose QFT sandwich is quadratic. The ripple-carry figure of $4n$ Toffoli for the plain addition matches [19] exactly. Both ratios improve with n, since c_{pad} grows only logarithmically: $c_{\text{pad}} = 2\lg n + \lg n_e + \delta$ with $\delta \in [2, 10]$.

22.3 Multiplying in the coset representation

There is a subtlety that looks fatal and is not. In branch j the register holds $v = jN + x$, so its *bits differ between branches* — and shift-and-add multiplication uses those bits as controls. It works anyway:

$$\sum_i v_i \,(A2^i \bmod N) \;\equiv\; Av \;=\; A(jN + x) \;\equiv\; Ax \pmod N,$$

so every branch accumulates something congruent to Ax, whatever its own j. Verified at $N = 7$ with both registers coset-encoded:

c_{pad}	m	qubits	worst error	bound $A/2^{c_{\text{pad}}}$
3	6	13	0.0760	0.7500
4	7	15	0.0135	0.4375

What is built, and what is not

Built and verified: the coset encoding, the plain-adder modular addition, the deviation law, and the coset multiplier. **Also built:** the full coset order-finding circuit (`coset_order.py`) and oblivious carry runways [20] (`runway.py`). The encoder is a circuit (pad qubit in $|+\rangle$, add $2^h N$, uncompute the pad by comparison) and matches the ideal coset state exactly. Level 4 runs in coset form — the accumulator starts in $|0\rangle_{\text{coset}}$ and the un-multiply returns it there — with runways attached to the accumulator only for the duration of each accumulation (encode, add piecewise, remove). One runway of length m deviates by at most 2^{-m} per addition, and `tests/test_runway.py` measures 0.2513 against 0.25 for $m = 2$. At toy sizes the whole multiplication is far from the ideal state, as the bound $2m \cdot 2^{-c_{\text{pad}}}$ predicts: for $N = 7$, fidelity 0.43/0.63/0.76 at $c_{\text{pad}} = 3/4/5$. End to end ($N = 15$, $A = 7$, $t = 3$) 50/60 shots land on the exact-phase outcomes without runways and 47/60 with them, and both recover $r = 4$.

There is also a change of character worth stating plainly. Every other construction in this note is *exact*, and its tests assert exact equality with ancillas provably clean. The coset representation is approximate by design, so its tests measure an error *rate* against a padding budget instead. That is not a weaker standard, but it is a different one, and it propagates: once the arithmetic is approximate, every claim above it becomes probabilistic.

23 Where to go from here

Two more upgrades, both easy and worth doing immediately.

Approximate QFT (easy, big win). Already built: pass $d = \lceil \log_2 n \rceil + 2$ as `approx` to `qft` (§11) and the count drops from $O(n^2)$ to $O(n \log n)$ with negligible error [11] — measured, 144 → 99 gates at $n = 16$, and the gap widens with n. Make it the default once your tests pass exactly.

One counting qubit — built and measured in §21.

Ripple-carry arithmetic — built and measured in §18. Do this one if you are heading anywhere near fault tolerance.

A fuller Qiskit implementation of the plain (unwindowed) algorithm, with both adders and the one-control circuit, is [25].

Windowing — built and measured in §19. Combined with the control savings it is what takes published Toffoli counts from $\approx 40n^3$ to $\approx 0.3n^3$.

What this does not demonstrate

Simulation cost grows exponentially in qubit count, so noise-free demos stop at a few bits. On real 2020s hardware the binding constraint is not qubit count but noise: even the $N = 15$ circuit's two-qubit depth far exceeds current coherence. What you have built is the algorithm, correct and testable — not a factoring machine. The gap between them is fault tolerance, and at $n = 2048$ it is about 10^9 Toffolis.

Part VI — Reading someone else's implementation

24 The package at a glance

Everything in Part II exists as running code: the `qiskit-shor` package [25] (see also the accompanying walkthrough [26]) implements the full tower on top of the Qiskit SDK and factors integers end to end on a simulator. It is worth reading against the note for two reasons. First, it realises the tower *twice*, over two interchangeable Level-1 backends — the CDKM ripple-carry adder of Part II [9], and a Fourier-space adder [7, 8] that Part II did not need but that dominates practical small-scale demonstrations. Second, everything above Level 1 is written *once*, generically over the adder: the code is organised exactly along the levels of §1.5, and the class hierarchy is the proof of the note's claim that Levels 2–5 use nothing about addition except its interface.

Module	Contents	Level(s)
`gate_utils.py`	`MAJGate`, `UMAGate`, controlled variants, QFT synthesis	0
`rc_adder.py`	`RCAdderCircuit`: ripple-carry additions [9]	1
`qft_adder.py`	`QFTAdderCircuit`: Fourier-space additions [7, 8]	1
`adder.py`	`AdderCircuit` (abstract): modular add / multiply / exponentiate	2–5
`shor.py`	order-finding circuits, `find_order`, `find_factor`	5 + classical

The class `AdderCircuit` is an abstract subclass of Qiskit's `QuantumCircuit`. It declares two primitives, `add_classical` and its controlled version — "add a known constant to a register, optionally under a control". The two backends supply those primitives; the abstract class builds Levels 2–5 out of them, verbatim from Theorems 5.1–7.1.

Dictionary: the note's registers, the code's names

The code follows Qiskit's little-endian convention: qubit k of a register carries bit x_k of $x = \sum_k x_k 2^k$, so qubit 0 is the least significant. The cast of §5 maps as:

In the note	In the code	Width
value register $\lvert x\rangle_{n+1}$	`y_reg` + `overflow_bit`	$n+1$
the sign qubit (top bit)	`overflow_bit`	1
the flag t	`ancilla_bit`	1
multiplicand register $\lvert x\rangle$	`x_reg`	n
counting register	`control_register`	$2n$ (or 1, §27.1)
ripple-carry scratch	`a_reg` (RC backend only)	$n+2$

25 Level 1 in code, twice

25.1 The ripple-carry backend

`gate_utils.py` defines MAJ and UMA exactly as in §4, as reusable Qiskit gates (listings are lightly abridged — docstrings and boilerplate trimmed — throughout Part IV):

```
class MAJGate(SingletonGate):
    """|c,b,a> -> |c+a, b+a, MAJ(a,b,c)>   (addition mod 2)"""
    def _define(self):
        q = QuantumRegister(3, "q")        # q0=c, q1=b, q2=a
        qc = QuantumCircuit(q, name="maj")
        qc.cx(q[2], q[1])                  # CNOT(a -> b)
        qc.cx(q[2], q[0])                  # CNOT(a -> c)
        qc.ccx(q[0], q[1], q[2])           # TOF(c, b -> a)
        self.definition = qc

class UMAGate(SingletonGate):
    """UMA . MAJ |c,b,a> = |c, a+b+c, a>"""
    def _define(self):
        q = QuantumRegister(3, "q")
        qc = QuantumCircuit(q, name="uma")
        qc.ccx(q[0], q[1], q[2])           # TOF(c, b -> a)
        qc.cx(q[2], q[0])                  # CNOT(a -> c)
        qc.cx(q[0], q[1])                  # CNOT(c -> b)
        self.definition = qc
```

and `RCAdderCircuit.add_quantum` chains them precisely as in the diagram of Theorem 4.2 — MAJ up the bits, one CNOT into the overflow qubit at the single instant the final carry sits in the clear, UMA back down:

```
def add_quantum(self, x_reg, y_reg, ancilla_bit, overflow_bit=None):
    """|x>_n |y>_n |0> -> |x>_n |y+x>_{n+1} |0>"""
    n = len(y_reg)
    maj, uma = MAJGate(), UMAGate(RC_version=True)
    self.append(maj, [ancilla_bit, y_reg[0], x_reg[0]])
    for i in range(n - 1):
        self.append(maj, [x_reg[i], y_reg[i + 1], x_reg[i + 1]])
    if overflow_bit:
        self.cx(x_reg[n - 1], overflow_bit)
    for i in range(n - 1, 0, -1):
        self.append(uma, [x_reg[i - 1], y_reg[i], x_reg[i]])
    self.append(uma, [ancilla_bit, y_reg[0], x_reg[0]])
```

Two departures from the bare Theorem 4.2 are worth pausing on.

Remark (the 3-CNOT UMA). The flag `RC_version=True` selects Cuccaro et al.'s alternative six-gate UMA ($X\,b$; CNOT $a \to b$; TOF $a, b \to c$; $X\,b$; CNOT $c \to a$; CNOT $c \to b$) [9]. It computes the same permutation as the three-gate version — the package's unit tests check them against each other — but its final two CNOTs share no qubit with the *next* UMA's first gates, so consecutive blocks overlap in time and the adder's depth drops by a constant factor. Same matrix, better schedule: a first taste of the fact that gate count and depth are different currencies.

Remark (adding a constant with an ancilla register). CDKM adds two *quantum* integers, but Level 2 needs $y \mathrel{+}= X$ for a *classical* X. The RC backend does the simplest correct thing: it writes X into n clean ancillas with X gates (`bitwise_AND_classical`), runs `add_quantum`, and unwrites X with the same X gates — which is why this backend carries the extra `a_reg` of $n+2$ qubits through every signature, and why its total budget is $5n+4$ qubits against the Fourier backend's $4n+2$ (§27.2). Subtraction, needed for blocks (2) and (5) of Theorem 5.1, is the same routine with the two's-complement of X: the code reduces X mod 2^{n+1} first, so `add_classical(X - N, ...)` just works.

The controlled adder is where the code quietly reuses the control lemma of §7. Rather than controlling all six gates of every block, `gate_utils.py` defines `MCXMAJGate` and `MCXUMAGate`, in which *only the CNOT targeting the b wire* acquires the outer control:

$$\begin{aligned}\mathrm{MAJ}_c:&\ c\text{-CNOT}(a\to b),\ \mathrm{CNOT}(a\to c),\ \mathrm{TOF}(c,b\to a);\\ \mathrm{UMA}_c:&\ \mathrm{TOF}(c,b\to a),\ \mathrm{CNOT}(a\to c),\ c\text{-CNOT}(c\to b).\end{aligned}$$

With the control off, what remains of MAJ is $\big[\mathrm{CNOT}(a\to c),\ \mathrm{TOF}(c,b\to a)\big]$ and what remains of UMA is its exact mirror $\big[\mathrm{TOF}(c,b\to a),\ \mathrm{CNOT}(a\to c)\big]$; since the UMA chain retraces the MAJ chain in reverse order, the whole residue telescopes to the identity — the *conjugation* pattern of §7, found in the wild. One controlled gate per block, $2n{+}1$ in total (the overflow CNOT is controlled too), instead of 6 per block.

25.2 The Fourier backend

The default backend does no carry propagation at all. Draper's observation [7] is that the QFT turns addition into phase accumulation. Write the full QFT on n qubits, in the code's little-endian convention, as

$$F|y\rangle = \bigotimes_{i=0}^{n-1} \frac{|0\rangle + e^{2\pi i\, y\, 2^{\,i-n}}|1\rangle}{\sqrt{2}}.$$

Each qubit carries the value y in the *period* of a phase, at a different scale — in Draper's reading, qubit i holds the binary fraction $e^{2\pi i\,(0.y_{n-i-1}\cdots y_1 y_0)}$ of the $n{-}i$ low bits of y: the value is spread across scales, not across carries. Adding a classical constant X is then n *parallel, uncontrolled* phase gates — qubit i gets $P(2\pi X\, 2^{\,i-n})$, turning $e^{2\pi i y 2^{i-n}}$ into $e^{2\pi i (y+X) 2^{i-n}}$ — and an inverse QFT returns to the computational basis holding $|(y+X) \bmod 2^n\rangle$. That is the entire implementation:

```
def add_classical(self, X, y_reg, a_reg=None, include_QFT=True):
    """|y> -> |y + X mod 2^n>   (no ancillas needed)"""
    n = len(y_reg)
    qft = QFTFullGate(n, approximation_degree=self.qft_approx_degree(n))
    if include_QFT:
        self.compose(qft, y_reg, inplace=True)
    for i in range(n):
        self.p(2 * np.pi * X * 2 ** (i - n), y_reg[i])  # addition in Fourier space
    if include_QFT:
        self.compose(qft.inverse(), y_reg, inplace=True)
```

No carries, no ancillas, and the controlled version (`c_add_classical`) simply upgrades each phase gate to a controlled phase gate — the QFT layers keep no control, being the conjugating $A\cdots A^\dagger$ of the control lemma, §7. This is why the Fourier route needs two registers fewer than the ripple-carry route, and why Beauregard [8] could squeeze the whole algorithm into $2n+3$ qubits.

Remark (what the Fourier route buys in depth). Two structural facts, one from each source paper, sharpen the economics. *Beauregard [8]:* because X is classical, all the rotations that land on one qubit multiply into a *single* phase gate — the loop above emits exactly one P per qubit, so the addition layer between the QFTs is n parallel one-qubit gates at **depth 1**. *Draper [7]:* even in the quantum–quantum adder (`add_quantum`, $O(n^2)$ controlled rotations) all rotations *commute* — unlike the QFT itself, whose Hadamards impose an ordering — so they schedule into $n{+}1$ time slices, or $O(\log n)$ slices under the small-angle cutoff below. The depth bottleneck of the Fourier route is therefore the QFT sandwich, at depth $O(n)$, not the addition it encloses. Finally, the same gate run backwards subtracts: $\phi\mathrm{ADD}(a)^\dagger$ sends $\phi(b)$ to $\phi(b-a)$ if $b \ge a$ and to $\phi(2^{n+1}-(a-b))$ otherwise,

so the top qubit is the comparator $[b < a]$ — §4's "comparison is free", relocated to Fourier space, and the workhorse of the sign tests in §26.

Doesn't this break "no complex number below Level 5"?

Inside the sandwich, yes: the intermediate states have irrational phases, and a single $P(\theta)$ is certainly not a permutation matrix. But the *composite* $F^\dagger \cdot (\text{phases}) \cdot F$ is exactly the same 0/1 permutation matrix $|y\rangle \mapsto |y + X \bmod 2^n\rangle$ as the CDKM circuit. The slogan of §1.3 survives in a sharper form: the two backends are two factorisations of one permutation — one through $\mathbb{F}_2$ carry algebra, one through the Fourier dual — and everything above Level 1 cannot tell them apart. The trade is not conceptual but economic: the Fourier route pays in *precision* (doubly-controlled rotations by angles as small as $2\pi/2^{n+1}$) instead of in *qubits*, which is why the package also offers the RC backend, whose gate set $\{X, \text{CNOT}, \text{TOF}\}$ is the one fault-tolerant hardware actually wants.

Remark (the approximate QFT). Rotations by $2\pi/2^k$ for large k are far below any hardware's resolution, and dropping them barely disturbs the result: keeping only angles at least $\pi/2^d$ with $d = \lceil \log_2 n \rceil + 2$ costs an error that stays negligible while cutting the QFT from $O(n^2)$ to $O(n \log n)$ gates [11, 12]. The package exposes exactly this cutoff via the flag `approx_QFT`; it is the "approximate QFT" priced in Part III's bill.

26 Levels 2–4: written once, above the interface

26.1 The modular adder

`AdderCircuit.add_classical_modulo` is Theorem 5.1 with one refinement. Since a *and* N are both classical, blocks (1) and (2) — add a, subtract N — merge into a single addition of the constant $a - N$:

```
def add_classical_modulo(self, X, y_reg, ancilla_bit, overflow_bit, N,
                         a_reg=None, reset_ancilla=True):
    """|y>|0> -> |(X + y) mod N>|0>,  assuming 0 <= X < N and 0 <= y < N."""
    y_bits = y_reg[:] + [overflow_bit]        # the (n+1)-wide value register
    n = math.ceil(math.log2(N))

    self.add_classical(X - N, y_bits, a_reg)  # blocks (1)+(2): y + X - N
    self.cx(y_bits[n], ancilla_bit)           # block (3): sign -> t
    self.c_add_classical(ancilla_bit, N, y_bits, a_reg)   # block (4)

    if reset_ancilla:                         # blocks (5)-(7): clear the flag
        self.add_classical(-X, y_bits, a_reg)
        self.cx(y_bits[n], ancilla_bit)
        self.x(ancilla_bit)
        self.add_classical(X, y_bits, a_reg)
```

Read it against the table in §5 line by line: `y_bits[n]` is the sign qubit, `ancilla_bit` is t, and the last four lines are blocks (5), (6), (7) — recompute the flag from the answer, XOR it against itself, restore. (The code performs the CNOT before the X where the note performs the X before the CNOT; both compute $t \mapsto \neg(t \oplus s)$.) The merge means the uncontrolled modular adder costs *four* constant additions rather than five — compile-time constants at work, as promised in the remark of §4.

The controlled version cannot merge blocks (1) and (2), because block (1) must switch with the control while block (2) must always fire. It is also where the code makes a different — and instructive — choice of which blocks carry the control:

```
def c_add_classical_modulo(self, control_reg, X, y_reg, ancilla_bit,
                           overflow_bit, N, a_reg=None, reset_ancilla=True):
    y_bits = y_reg[:] + [overflow_bit]
    n = math.ceil(math.log2(N))

    self.c_add_classical(control_reg, X, y_bits, a_reg)  # (1), controlled
    self.add_classical(-N, y_bits, a_reg)                # (2), uncontrolled
    self.cx(y_bits[n], ancilla_bit)                      # (3)
    self.c_add_classical(ancilla_bit, N, y_bits, a_reg)  # (4), controlled on t

    if reset_ancilla:
        self.add_classical(-X, y_bits, a_reg)            # (5), uncontrolled!
        self.mcx(control_reg + [y_bits[n]], ancilla_bit) # (6), CNOT -> Toffoli
        self.x(ancilla_bit)
        self.add_classical(X, y_bits, a_reg)             # (7), uncontrolled!
```

Two of seven, not three of seven

Where §7 controlled blocks (1), (5), (7), the code controls block (1) and, instead of controlling the additions of blocks (5) and (7), upgrades block (6)'s flag-clearing CNOT to a Toffoli. Both choices are correct, and both are licensed by the same lemma; only the residue argument changes. Control off, code version: block (2) drives the register to $y - N < 0$, sign set, t flips on, block (4) adds N back — the familiar conspiracy. Then block (5) subtracts X *unconditionally*, the Toffoli does nothing (its outer control is off), the X gate undoes the earlier flip of t, and block (7) adds X back: the residue $(-X)$ then $(+X)$ cancels as pure conjugation. Control on: after block (5) the sign qubit holds $\neg t$ (Theorem 5.1's proof), so the Toffoli sets $t \mapsto t \oplus \neg t = 1$ and the X clears it. The bargain: two additions run uncontrolled — saving $O(n)$ controlled gates each — at the cost of upgrading one CNOT to a Toffoli, which is $O(1)$.

Remark (the hidden QFT sandwiches). Beauregard's drawn circuit (his Fig. 5 [8]) keeps the value register in Fourier space for the whole modular adder and drops out only twice — a $\mathrm{QFT}^{-1} \cdots \mathrm{QFT}$ pair around each read of the sign qubit — because a two's-complement sign is a computational-basis fact, invisible to a register that stores its value in phases. The code contains the same structure without ever writing it: every `add_classical` of the Fourier backend closes with an inverse QFT and the next opens with a QFT, so wherever two additions abut, the pair is redundant (Beauregard draws nothing there, and a transpiler cancels it), and wherever a CNOT to the flag intervenes — blocks (3) and (6) — the pair survives as exactly his sandwich. The RC backend has no such subtlety: its register never leaves the computational basis, which is one more reason its gate count is a factor of n better and its qubit count $n{+}2$ worse.

26.2 The multiplier, out of place and then in place

Level 3 is a four-line loop — Theorem 6.1 transcribed. Note the constant `((A % N) * 2**i) % N` is the shifted multiple $2^i A \bmod N$, computed in Python at circuit construction time — "compile time" in the note's vocabulary:

```
def c_add_quantum_modulo(self, control_reg, x_reg, y_reg, ancilla_bit,
```

```
                        overflow_bit, N, a_reg=None, A=1):
    """|c>|x>|y>|0> -> |c>|x>|(y + c*A*x) mod N>|0>"""
    for i in range(len(x_reg)):
        self.c_add_classical_modulo(
            control_reg=control_reg + [x_reg[i]],   # doubly controlled
            X=((A % N) * 2**i) % N,                 # 2^i A mod N, classical
            y_reg=y_reg, ancilla_bit=ancilla_bit,
            overflow_bit=overflow_bit, N=N, a_reg=a_reg)
```

Level 4 — Theorem 7.1 — is the multiply/swap/uncompute sandwich, with the modular inverse handed to Python's built-in `pow`:

```
def c_multiply_modulo(self, control_reg, A, x_reg, y_reg, overflow_bit,
                      ancilla_bit, N, a_reg=None, ...):
    """|c>|x>|0> -> |c>|A^c x mod N>|0>"""
    # CMULT(A):  |x>|0> -> |x>|Ax mod N>
    self.c_add_quantum_modulo(control_reg, x_reg, y_reg,
                              ancilla_bit, overflow_bit, N, a_reg, A=A)
    # controlled SWAP: n Fredkin gates sharing the control
    for i in range(n):
        self.append(SwapGate().control(k), control_reg + [x_reg[i], y_reg[i]])
    # CMULT(A^{-1})^dagger: uncompute x from the product
    B = pow(A, -1, N)                       # extended Euclid, classical
    self.c_add_quantum_modulo(control_reg, y_reg, x_reg,
                              ancilla_bit, overflow_bit, N, a_reg, A=-B)
```

Remark (the dagger, implemented as a minus sign). Theorem 7.1 calls for $\mathrm{CMULT}(\bar{a})^\dagger$ — run the $\bar{a}$-multiplier backwards. The code never reverses a circuit: it passes $A = -B$ and lets the constant reduction fold $-\bar{a} \bmod N = N - \bar{a}$ into an ordinary forward multiplier. The two are the same operator, because the inverse of "add $\bar{a}x$ to this register mod N" is exactly "subtract $\bar{a}x$ from this register mod N". Adjoints of permutations are permutations, and for adders they are again adders — one more classical fact doing quantum work.

27 Level 5 and the two order-finding circuits

The ladder is the last, shortest loop — rung constants by repeated squaring, one controlled in-place multiplier per counting qubit:

```
def exponentiate_modulo(self, A, x_reg, y_reg, ancilla_reg, N, a_reg=None):
    """|x>_m |y>_n |0>_{n+2} -> |x>_m |(A^x y) mod N>_n |0>_{n+2}"""
    n = math.ceil(math.log2(N))
    for i in range(len(x_reg)):
        self.c_multiply_modulo(
            control_reg=x_reg[i],
            A=pow(A, 2**i, N),              # the rung constant a^{2^i} mod N
            x_reg=y_reg, y_reg=ancilla_reg[:n],
            overflow_bit=ancilla_reg[n], ancilla_bit=ancilla_reg[n + 1],
            N=N, a_reg=a_reg)
```

and the full order-finding circuit of §8 — Hadamards, ladder, inverse QFT, measurement — assembles in a dozen lines (`shor.py`):

```
def order_finding_circuit(A, N, precision=None, adder="qft_adder"):
    n = math.ceil(math.log2(N))
```

```
    m = precision if precision is not None else 2 * n   # t = 2n counting qubits

    control_register = QuantumRegister(m)
    target_register  = QuantumRegister(n)
    ancilla_register = QuantumRegister(n + 2)
    output_register  = ClassicalRegister(m, name="output_bits")
    qc = QFTAdderCircuit(control_register, target_register,
                         ancilla_register, output_register)

    for i in range(m):
        qc.h(control_register[i])           # Hadamard layer
    qc.x(target_register[0])                # target = |1>
    qc.exponentiate_modulo(A=A, x_reg=control_register,
                           y_reg=target_register,
                           ancilla_reg=ancilla_register, N=N)
    qc.compose(QFTGate(m).inverse(), qubits=control_register, inplace=True)
    qc.measure(control_register, output_register)
    return qc
```

Count the qubits: $2n$ counting + n target + $(n{+}2)$ scratch (n for the out-of-place product, the sign qubit, the flag t) $= 4n + 2$ — with the RC backend, $n{+}2$ more for `a_reg`, hence $5n + 4$.

27.1 One control qubit: the $2n + 3$ circuit

Beauregard's headline saving [8] is that the $2n$ counting qubits can be replaced by *one*, recycled $2n$ times. The idea, due to Mosca–Ekert [14] and Parker–Plenio [15], rests on the semiclassical Fourier transform of Griffiths–Niu [13]: in the inverse QFT, every two-qubit controlled rotation is controlled by a qubit that is *about to be measured anyway* — so measure it first, and replace the quantum control by a classical one conditioned on the outcome. The counting register then never needs to exist all at once. One qubit is Hadamarded, drives the controlled multiplication for its rung, receives phase corrections determined by all *previously measured* bits, is measured, and is reset for the next rung:

```
    c_bit = control_register[0]             # the one and only counting qubit
    for i in range(m):
        qc.h(c_bit)
        qc.c_multiply_modulo(               # rung i, MSB first
            control_reg=c_bit, A=pow(A, 2 ** (m - i - 1), N),
            x_reg=target_register, y_reg=ancilla_register[:n],
            overflow_bit=ancilla_register[n],
            ancilla_bit=ancilla_register[n + 1], N=N)
        for j in range(i):                  # semiclassical inverse QFT:
            with qc.if_test((output_register[j], 1)):     # classically
                qc.p(-math.pi / 2 ** (i - j), c_bit)      # conditioned phases
        qc.h(c_bit)
        qc.measure(c_bit, output_register[i])
        with qc.if_test((output_register[i], 1)):
            qc.x(c_bit)                     # reset for the next rung
```

The quantum inverse QFT has disappeared entirely, traded for mid-circuit measurement and feed-forward (Qiskit's `if_test` control flow — on hardware, "dynamic circuits"). Total: $1 + n + (n{+}2) = 2n + 3$ qubits, the count in the title of Beauregard's paper. The two variants are toggled by one argument, and they sample from the same distribution: nothing about the *arithmetic* changed, only

about where the counting register lives in time.

27.2 The bill, as the package pays it

With $n = \lceil \log_2 N \rceil$ and the package's own accounting [25] (gates counted in the Qiskit basis before transpilation — see the note in the README on why such counts are only comparative):

Backend	qubits	qubits (one-control)	gates	depth
Fourier [7, 8]	$4n+2$	$2n+3$	$O(n^4)$	$O(n^3)$
with approximate QFT [11, 12]	$4n+2$	$2n+3$	$O(n^3 \log n)$	$O(n^2 \log n)$
Ripple-carry [9]	$5n+4$	$3n+5$	$O(n^3)$	$O(n^3)$

The Fourier backend's extra factor of n is the QFT sandwich around every addition; the ripple-carry backend's extra n+2 qubits are the scratch register for constant encoding. The package also implements, as a standalone method (`c_add_quantum_optimized_depth`), the depth-$O(n)$ controlled Fourier addition of Pavlidis and Gizopoulos [10], which moves the control off the $O(n^2)$ rotation block — the rotations run uncontrolled, conjugated by controlled bit flips of the operand — but this trick does not compose with the modular reduction of §26, so the order-finding circuit does not use it; their full architecture reworks the modular level too, and is beyond both the package and this note.

Beauregard's own ledger [8] prices each level of the Fourier route, with $k_{\max}$ the approximate-QFT cutoff: ϕADD(a) is n+1 qubits at depth $O(1)$; the doubly controlled modular adder n+4 qubits, $O(n\,k_{\max})$ gates, depth $O(n)$; CMULT and c-U_a are $2n$+3 qubits at depth $O(n^2)$; the whole circuit $2n$+3 qubits, $O(n^3 \lg n)$ gates, depth $O(n^3)$. His $2n+3$ was, at the time, the last word in a decade-long qubit race:

Year	Construction	Qubits	Notes
1996	Vedral–Barenco–Ekert [4]	$7n+1$	$5n+2$ with basic optimisation
1996	Beckman–Chari–Devabhaktuni–Preskill [5]	$5n+1$	$4n+1$ with unbounded Toffolis
1998	Zalka [6]	$3n + O(\lg n)$	elementary gates only
2003	Beauregard [8]	$2n+3$	the one-control circuit of §27.1

At $N = 15$ the $2n+3$ circuit is eleven qubits. The 2001 NMR experiment that factored 15 [23] used seven — by compiling for that one modulus, whose rung constants $7, 4, 1, 1, \ldots$ (§8) make most of the ladder the identity. General-purpose and special-cased circuits are different objects, and only the former is priced by the table above.

28 Cross-check: factoring 15 from scratch

The same two papers admit more than one faithful implementation, and it is instructive to read a second. *Building Shor's algorithm from scratch: factoring 15* [24] is an independent Qiskit notebook that assembles the identical Draper–Beauregard stack — its QPE preamble follows Nielsen–Chuang [22] — one class per level of the tower, each verified on deterministic test vectors before the next is built (5 through the QFT and back; (5+3) mod 9 = 8 through the modular adder; (7+3·3) mod 7 = 2 through the multiplier). The correspondence is exact:

Object	`qiskit-shor`	notebook [24]
QFT (Level 0/1)	`QFTFullGate`	`QFTBlock`
ϕADD(a) (§25.2)	`add_classical`	`QAdderClassicalBlock`
modular adder (Thm. 5.1)	`add_classical_modulo`	`add_mod_N_classic`
CMULT (Thm. 6.1)	`c_add_quantum_modulo`	`CMULTaMODN_c`
c-U_a (Thm. 7.1)	`c_multiply_modulo`	`Controlled_U_a`
ladder + QPE (§8)	`order_finding_circuit`	final assembly cell

Three divergences are worth reading closely, because each lands on a point the note has already made.

Remark (three of seven, again). The notebook builds its doubly controlled additions by calling Qiskit's `.control(2)` on the ϕADD gates of blocks (1), (5) and (7) — Beauregard's original prescription, and precisely the 3-of-7 choice of §7. So the two codebases realise the *two* correct control placements side by side: blocks (1), (5), (7) in the notebook; block (1) plus the Toffoli'd flag-clear in `qiskit-shor` (§26). Diff them and the residue argument of the control lemma is the only invariant.

Gift 1, forgone

The notebook's ladder does *not* precompute rung constants. It applies the same c-U_a gate 2^i times for counting qubit i:

```
for j in range(2**i):  qc.append(Ctrl_U_a_gate, ...)
```

— $1+2+4+8 = 2^t - 1 = 15$ multiplier applications where `qiskit-shor` uses $t = 4$ rungs with `pow(A, 2**i, N)`. This is the exponential sum of §1.1, live in running code. At $N = 15$ it costs a factor of four and nobody notices; at any cryptographic size it is the difference between an algorithm and a demonstration. Gift 1 is two lines of Python — and it is still a gift you must actually accept.

Remark (an undersized counting register). The notebook sets $t = n = 4$ rather than $t = 2n = 8$, noting the shortcut. For this instance it is sound: the order $r = 4$ divides 2^t, so the phase s/r is *exactly* representable in 4 bits and no rounding occurs. For general N the order does not divide 2^t, the measured value only approximates s/r, and the $2n$ bits are what guarantees the continued-fraction step converges to the right denominator [1, 3]. Another instance of 15's exceptional friendliness (§8) — and of why it proves so little.

Read the histogram, not the factors

With $a = 7$, $N = 15$, $r = 4$, $t = 4$, theory owes four sharp peaks: the measurement should return $y = 2^t s/r \in \{0, 4, 8, 12\}$, a quarter of the shots each. The run recorded in the notebook shows instead a *flat* distribution — all sixteen outcomes near 512 of 8 192 shots, and measured phases like 3/16 and 5/16 that no $s/4$ can produce. That flatness is a diagnosis, not statistics: it is the uniform-noise signature §1.4 derives for a counting register left entangled with something — interference killed by $\langle g_{x'}|g_x\rangle = 0$ — somewhere in the final assembly, even though every gate-level unit test passed deterministically. The notebook's retry loop still ends by printing 3×5, because its classical tail — continued fractions, then *verify* $\gcd(a^{r/2} \pm 1, N)$ — filters random candidates until one passes, which uniform noise eventually supplies. Two lessons, both cheap. End-to-end correctness lives in the *distribution*, so test the distribution (the seeded `find_order` test of §29 is exactly such a test). And a pipeline whose classical tail can succeed on white noise will not tell you when its quantum head has failed — the algorithm's own robustness is a bug-mask.

29 Running it

What remains of Shor's algorithm [1] outside the circuit is classical, and `shor.py` contains all of it. `find_order` builds either circuit, transpiles it, samples it, and reads the order out of the measurement distribution — the continued-fraction step is Python's `Fraction`:

```
x = int(best_output, 2)                          # measured phase, t bits
r = Fraction(x / 2**precision).limit_denominator(N - 1).denominator
if pow(A, r, N) == 1:                             # verify classically
    return r
```

— exactly the "continued fractions recover r" of Part III, with the candidate checked by one classical exponentiation. `find_factor` wraps it in the standard loop: strip factors of 2 and prime powers, pick a random base, gcd-check it (the lucky case has already factored N), find the order, and try $\gcd(a^{r/2} - 1, N)$ when r is even. End to end, on a noise-free simulator:

```
from qiskit_aer import AerSimulator
from qiskit_aer.primitives import SamplerV2 as AerSampler
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager
from qiskit_shor.shor import find_factor, find_order

sim = AerSimulator()
pm = generate_preset_pass_manager(backend=sim, optimization_level=1)

order, dist = find_order(7, 15, AerSampler(), pm, num_shots=100,
                         one_control_circuit=True)   # -> 4
factor = find_factor(15, AerSampler(), pm, num_tries=3,
                     num_shots_per_trial=100)        # -> 3 or 5
```

What the package machine-checks

The note's "Verified" boxes have counterparts in the repository's `pytest` suite: unit tests drive every arithmetic method on all (or sampled) basis states through Qiskit's simulator — additions, modular additions, multipliers, for both backends, including that every ancilla returns to $|0\rangle$ — and the end-to-end tests recover the order $r = 4$ of both $A = 2$ and $A = 7$ modulo 15, with both circuit variants, on a seeded `AerSimulator`. The same discipline at two scales: trust nothing you have not enumerated.

What this does not demonstrate

Simulation cost grows exponentially in qubit count, so noise-free demonstrations stop around N of a few bits — and on 2025-era hardware, noise, not qubit count, is the binding constraint: a few hundred physical qubits exist, but the $\sim 2\,000$-gate two-qubit-depth of even the $N = 15$ one-control circuit far exceeds current coherence [25]. The package is the tower of Part II made executable and testable, not a factoring machine; the gap between the two is the fault-tolerance overhead priced in Part III's bill.

Part VII — The other Shor: elliptic-curve discrete logarithms

Why a second half

Everything so far factors integers. The same three layers — Hadamards, an oracle ladder, an inverse QFT — also break elliptic-curve cryptography, and the outer two layers are unchanged. What changes is the middle, and it changes completely.

For factoring, the group is $(\mathbb{Z}/N)^\times$ and the oracle multiplies by a *classical* constant. Multiplication by a constant is cheap: Part II built it out of shifted additions. For the discrete logarithm on a curve, the group is $E(\mathbb{F}_p)$ and the oracle adds a classical *point* — and the chord-and-tangent law needs a division, i.e. a modular *inverse*, of a quantum value. That single fact is the whole story of this part: an inversion costs an order of magnitude more than the multiplication it replaces, and every optimisation below is an attempt to make it cheaper, do it less often, or avoid it altogether.

30 What changes, and what does not

Let E be an elliptic curve over $\mathbb{F}_p$, P a point of prime order r, and $Q = [k]P$ for an unknown k. Recovering k is the *elliptic-curve discrete logarithm problem* (ECDLP), and it is what ECDSA, ECDH and every deployed curve-based scheme rest on.

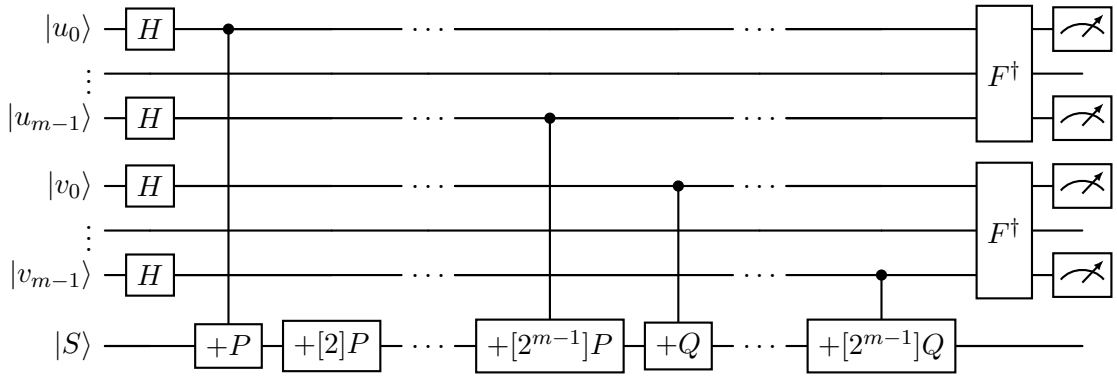
Shor's algorithm for it [27] uses two counting registers instead of one:

$$\frac{1}{2^m}\sum_{u,v}|u\rangle|v\rangle|\mathcal{O}\rangle \xrightarrow{\text{oracle}} \frac{1}{2^m}\sum_{u,v}|u\rangle|v\rangle|[u]P+[v]Q\rangle \xrightarrow{F^\dagger\otimes F^\dagger} \text{measure } (j_1,j_2).$$

The third register depends on (u,v) only through $u+kv \bmod r$, so its level sets are cosets of the lattice generated by $(-k,1)$ and $(r,0)$. The Fourier transform concentrates on the dual lattice, and a measured pair obeys, with $a_i = \mathrm{round}(j_i r/2^m)$,

$$a_2 \equiv k\,a_1 \pmod r, \qquad \text{so} \qquad k = a_2\,a_1^{-1} \bmod r \tag{6}$$

whenever a_1 is invertible — which discards $j_1 = 0$ automatically. One measurement suffices with constant probability; §38 shows the distribution this produces on a toy curve, and §38.1 the search that [34] adds when 2^m is not a multiple of r.



Each rung adds a *classical* point — the multiples $[2^i]P$ and $[2^i]Q$ are precomputed on a laptop — controlled by one counting qubit. There are $2m \approx 2n$ of them, against the $2n$ rungs of Part II's ladder, so at the level of *counting* nothing has changed. The cost is entirely inside the box marked $+P$.

The accumulator never starts at infinity

The diagram starts the third register at a fixed point S, not at the identity $\mathcal{O}$. Affine addition formulas cannot represent the point at infinity at all: there is no (x, y) for it, and the slope λ is undefined the moment the accumulator equals $\pm$ the addend. Starting at a constant S and reading the answer relative to it keeps the accumulator on an ordinary point throughout, and shifts the computed coset without touching the interference. The same device appears as the offset T in the windowed constructions of §36.8, and as `P0` in the Classiq tutorial [37].

31 Why it works

Section 30 asserted (6). This section derives it. The argument is short, it is the only place in this part where the *quantum* content lives — everything after §32 is reversible arithmetic — and it is checked numerically in `bench/ec_ablation.py`, which evaluates the amplitudes by brute force and compares them against the closed form below.

31.1 The state, in four lines

Write $q = 2^m$ for the size of each counting register and $\omega = e^{2\pi i/q}$. Start from a uniform superposition and apply the oracle:

$$\frac{1}{q}\sum_{u,v}|u\rangle|v\rangle|S\rangle \;\longrightarrow\; \frac{1}{q}\sum_{u,v}|u\rangle|v\rangle\,\big|\,S + [u]P + [v]Q\,\big\rangle = \frac{1}{q}\sum_{u,v}|u\rangle|v\rangle\,\big|\,S + [u + kv]P\,\big\rangle,$$

the last equality being the only place $Q = [k]P$ is used. The third register therefore depends on (u, v) *only* through

$$w \;=\; u + kv \bmod r,$$

so group the terms by w:

$$\frac{1}{q}\sum_{w\in\mathbb{Z}_r}\left(\sum_{(u,v)\,:\,u+kv\equiv w}|u\rangle|v\rangle\right)\big|\,S + [w]P\,\big\rangle. \tag{7}$$

The third register is never measured and never needs to be: (7) is already a sum of orthogonal branches, so the reduced state on (u, v) is a mixture over w, and it is enough to analyse one branch.

Why the offset S cannot matter

S appears only inside the third register's label. It relabels which basis state each branch attaches to; it does not touch the (u, v) amplitudes at all, so it cannot change the measured distribution. The same argument covers w: every branch of (7) gives the same |amplitude|, which the benchmark confirms directly by computing two different branches and comparing (result: `yes`). S is there for the arithmetic — to keep the accumulator off the point at infinity — and for nothing else.

31.2 Two lattices

Fix a branch w. Its (u, v) set is a coset of

$$L \;=\; \{(u,v) \in \mathbb{Z}^2 \;:\; u + kv \equiv 0 \pmod r\} \;=\; \big\langle\, (r,0),\ (-k,1) \,\big\rangle,$$

the *period lattice*. The two generators say the two things that can be done without changing the answer: take r steps in u, or take one step in v and k steps back in u. Its determinant is r, so $[\mathbb{Z}^2 : L] = r$ and the plane splits into exactly r cosets — one per value of w, as it must.

The discrete logarithm is the slope of the second generator. Finding L is finding k; that is the whole problem, restated.

A Fourier transform does not see L, it sees the *dual*

$$L^* \;=\; \{\, y \in \mathbb{Q}^2 \;:\; \langle y, x\rangle \in \mathbb{Z} \;\; \forall x \in L \,\} \;=\; \big\langle\, \tfrac{1}{r}(1,k),\ (0,1) \,\big\rangle,$$

obtained as $(B^{-1})^{\mathsf{T}}$ from L's basis B. Writing a point of L^* as $(a_1/r,\, a_2/r)$ and clearing denominators, membership says exactly

$$a_2 \equiv k\, a_1 \pmod r,$$

which is (6). It is not a consequence of the dual lattice — it *is* the dual lattice, written in coordinates.

31.3 The amplitude

Take $r \mid q$ for a moment and write $q = rM$. Then for each v the admissible u in $[0, q)$ are $u_0(v) + tr$ with $u_0(v) = (w - kv) \bmod r$ and $t = 0, \ldots, M-1$ — exactly M of them. The amplitude at (j_1, j_2) after the transform is, up to normalisation,

$$A(j_1, j_2) \;\propto\; \sum_{v=0}^{q-1} \omega^{v j_2}\, \omega^{u_0(v) j_1} \underbrace{\sum_{t=0}^{M-1} \omega^{t r j_1}}_{\text{(i)}}.$$

Sum (i) is geometric with ratio $\omega^{r j_1}$: it equals M when $r j_1 \equiv 0 \pmod q$, i.e. when $M \mid j_1$, and *exactly zero* otherwise. So only $j_1 = a_1 M$ survives. For those, $\omega^{u_0 j_1} = e^{2\pi i\, u_0 a_1 / r}$ depends on u_0 only mod r — which is why reducing u_0 into $[0, r)$ above was harmless — and $u_0 \equiv w - kv$, leaving

$$A \;\propto\; e^{2\pi i\, w a_1 / r} \sum_{v=0}^{q-1} e^{2\pi i\, v\,(j_2 - k a_1 M)/q}.$$

That second sum is q when $j_2 \equiv k a_1 M \pmod q$ and zero otherwise. Writing $j_2 = a_2 M$ and dividing through by M:

$$\boxed{\; a_2 \equiv k\, a_1 \pmod r \;}$$

and every surviving outcome has $|A|^2 = 1/r$. The distribution is *uniform on the r points of the dual lattice*, and nothing else has any amplitude at all.

r	k	q	outcomes	each	closed form exact	usable $\varphi(r)/r$
4	3	16	4	1/4	✓	0.50
8	3	32	8	1/8	✓	0.50
4	1	64	4	1/4	✓	0.50

31.4 Reading off k, and what it costs

Given (a_1, a_2) with $a_2 \equiv ka_1$, recover $k = a_2 a_1^{-1} \bmod r$ — possible exactly when a_1 is invertible. For prime r that means $a_1 \neq 0$, so $r-1$ of the r outcomes work and one run succeeds with probability $(r-1)/r$: overwhelming, and it does not improve with more qubits because the $a_1 = 0$ outcome is a fixed $1/r$ of the mass. Note r is not secret — it is the curve's published group order — so this is a division, not a search.

The clean case never actually happens

Everything above assumed $r \mid q$. For ECDLP r is *prime*, and q is a power of two, so $r \mid q$ only when $r = 2$. The exact calculation is a device for seeing the structure; the real situation is always the other one.

What goes wrong is only sum (i): with $r \nmid q$ it no longer vanishes, it becomes a Dirichlet kernel. Amplitude leaks onto the integers either side of $a_1 q/r$, which is why the measured peaks of §38.1 sit at $(6, 5)$ and $(2, 3)$ rather than at the exact dual-lattice points $(1.6, 3.2)$ and $(3.2, 6.4)$, and why $a_i = \text{round}(j_i r/q)$ has a rounding in it at all. The leak shrinks as q grows past r. Entries are $P(\text{usable and correct})$, i.e. the chance of an outcome with a_1 invertible *and* $a_2 \equiv ka_1$; the last column is the irreducible $a_1 = 0$:

	$m - \lceil \log_2 r \rceil$				
r	0	1	2	3	$P(a_1{=}0)$
5	60%	67%	74%	77%	20%
7	48%	67%	77%	81%	15%
11	69%	77%	84%	88%	9%
13	63%	75%	84%	88%	8%

At $m = \lceil \log_2 r \rceil$ the answer is barely better than a coin; three extra qubits per register take it to 88%, with the residual 8 being the irreducible $a_1 = 0$ outcome. This is the whole content of [34]'s analysis for the discrete logarithm, and the reason the post-processing of §38.1 searches a small neighbourhood of the rounded value rather than trusting it.

32 The group law, and why it resists reversibility

For $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$ on $y^2 = x^3 + ax + b$ with $x_1 \neq x_2$,

$$\lambda = \frac{y_1 - y_2}{x_1 - x_2}, \qquad x_3 = \lambda^2 - x_1 - x_2, \qquad y_3 = \lambda(x_2 - x_3) - y_2. \tag{8}$$

Three subtractions, one squaring, one multiplication — and one division. In Part II every arithmetic step was addition or multiplication by a compile-time constant. Here the divisor $x_1 - x_2$ is a quantum value, so the inverse must be computed *by circuit*, on a superposition, and then uncomputed. Section 34 does that, and §37 shows it dominating everything else by a factor of three to five.

Level	Object	Made of	In code
5	the two ladders	$2m$ rungs, one per counting qubit	`ec_shor.py`
4	c-$(+P_i)$	4 sub, 1 add, 2 div, 2 mul, 1 sqr, 1 neg	`ec_pointadd.py`
3	$x^{-1} \bmod p$	$2n$ Kaliski rounds + correction	`ec_kaliski.py`
3′	$x \cdot y \bmod p$	n modular doublings + n additions	`ec_mult.py`
2	$\pm \bmod p$, $2x \bmod p$	plain add/sub plus one reduction	`ec_modarith.py`
1	ADD	CDKM chain, or Gidney's AND chain	`ec_adders.py`
0	AND	$4T$ to compute, $0T$ to uncompute	`ec_gates.py`

Level 3 is new and it is the expensive one. Everything below it is Part II's tower with the modulus renamed.

33 Level 2: modular arithmetic over $\mathbb{F}_p$

Two blocks carry everything above them, and both are from Roetteler, Naehrig, Svore and Lauter [28]. Neither is hard; both are worth seeing, because §36.3 makes its savings by attacking exactly the parts described here as "the awkward bit".

Modular doubling. Shift, subtract p, add it back if that went negative:

$$2x \;\longrightarrow\; 2x - p \;\longrightarrow\; \begin{cases} 2x - p & 2x \geq p \\ 2x & \text{otherwise.} \end{cases}$$

The shift is free — it renames wires. The awkward bit is the borrow flag, which must return to $|0\rangle$ or it is garbage in the sense of §1.4. It clears for one CNOT: $2x$ is even and p is odd, so *the answer is even exactly when no reduction happened*, and the answer's own low bit is the flag.

Modular addition. The same shape, but the flag-clearing trick is prettier. After adding and subtracting p, the flag holds $[\,x + y < p\,]$, i.e. "no reduction". And

- no reduction means the answer is $x + y$, which is $\geq x$;
- a reduction means the answer is $x + y - p$, which is $< x$ because $y < p$.

So $[\,\text{answer} < x\,]$ is exactly the complement of the flag, and one comparison against an operand that is still lying there clears it. No extra ancilla, no second copy. In code:

```
def cmodadd(m, ctrl, x, y, p):
    """y <- (y + x) mod p when ctrl (or unconditionally if ctrl is None).
    ..."""
    ctx, n = m.ctx, len(y)
    assert len(x) == n
    anc_x, anc_y = m.anc(1, "ax"), m.anc(1, "ay")
    xe, ye = x + anc_x, y + anc_y
    cp, a = m.anc(n + 1, "cp"), m.anc(n + 1, "a")

    if ctrl is None:
        A.add(ctx, xe, ye, a)
    else:
        A.cadd(ctx, ctrl, xe, ye, cp, a)
    A.sub_const(ctx, ye, p, cp, a)
    A.cadd_const(ctx, anc_y[0], y, p, cp[:n], a)
```

```
    if ctrl is None:
        A.lt_uint(ctx, y, x, anc_y[0], a)
    else:
        A.clt_uint(ctx, ctrl, y, x, anc_y[0], a, cp)
    ctx.x(anc_y[0])
    m.free(cp, a, anc_x, anc_y)
```

The doubling is the same shape and its flag-clearing is one CNOT — the last two lines below, where `xe[0]` is the answer's own low bit:

```
def moddbl(m, x, p):
    """x <- 2x mod p, in place: the register handle stays valid.
    ..."""
    ctx, n = m.ctx, len(x)
    z = m.anc(1, "z")
    xe = A.shift_up(x, z[0])                    # n+1 bits, holds 2x
    _moddbl_body(m, xe, p)
    # value bits now sit at xe[0..n-1]; xe[n] (the old MSB) is clean
    for i in range(n - 1, -1, -1):
        ctx.swap(xe[i], xe[i + 1])
    m.free(z)
```

34 Level 3: inversion by Kaliski's algorithm

The binary extended Euclidean algorithm branches four ways per step: u even, v even, both odd with $u > v$, both odd with $u \le v$. A quantum circuit cannot branch — every branch must run on every input, on a superposition. Kaliski's formulation as reworked by Häner, Jaques, Naehrig, Roetteler and Soeken [29] makes all four cases the *same* straight line:

repeat $2n$ times:

1.	**if** v odd **and** (u even **or** $u > v$):	swap (u, v), swap (r, s); remember it
2.	**if** u, v both odd:	$v \mathrel{-}= u$; $\quad s \mathrel{+}= r$
3.		$v \mathrel{/}= 2$; $\quad r \mathrel{\times}= 2$ (the doubling only if the round fired)
4.	**if** it swapped:	swap back

Started at $(u, v, r, s) = (p, x, 0, 1)$ this terminates with $u = 1$, $v = 0$, $s = p$ and

$$r \;=\; -x^{-1}\, 2^{k} \bmod p, \tag{9}$$

where k counts the rounds that did anything. That is a *pseudo*-inverse: right up to a sign and a power of two, both of which the caller fixes — one modular negation, and k modular halvings driven by a counter. Equation (9) is asserted directly in `tests/test_ec_kaliski.py` for every invertible x and every p tested, alongside $u = 1$, $v = 0$, $s \equiv 0$ and the counter itself.

Three bits per round, and why none can be dropped

The round is a permutation of (u, v, r, s) chosen by its decision bits, so those bits must survive or the round is not reversible. Three are kept:

- f — *did this round do anything* ($v \neq 0$ on entry). Not recoverable afterwards: an inert round and the last active round both leave $v = 0$.
- a — *did it swap.*
- b — *were both odd*, so the subtract-and-add fired. Not implied by (f, a): the case $f = 1, a = 0$ covers both "v was even" and "both odd with $u \leq v$".

The comparison $u > v$ is *not* kept. It is consumed while building a and then uncomputed by running the comparator a second time — before u and v change, which is the only window in which that is possible.

In code, with the decision bits built first and the comparison uncomputed before u and v move:

```
def kaliski_round(m, u, v, r, s, p, rec, conditional=True):
    ctx, n = m.ctx, len(u)
    f, a, b = rec

    # --- f: is this round active? ---------------------------------------
    if conditional:
        MA.is_nonzero(m, v, f[0])
    else:
        ctx.x(f[0])                       # [106] Sec 3.3: never test v != 0

    # --- decision bits, from the pre-round u and v ----------------------
    ...

    # --- the straight-line body -----------------------------------------
    for i in range(n):
        ctx.cswap(a[0], u[i], v[i])
    for i in range(len(r)):
        ctx.cswap(a[0], r[i], s[i])

    cp, sc = m.anc(n, "cp"), m.anc(n, "sc")
    A.csub(ctx, b[0], u, v, cp, sc)       # v -= u  (no borrow: u <= v here)
    m.free(cp, sc)
    MA.cmodadd(m, b[0], r, s, p)          # s += r  mod p

    _shift_down(ctx, v)                   # v /= 2  (v is even by now)
    MA.cmoddbl(m, f[0], r, p)             # r *= 2  mod p

    for i in range(n):
        ctx.cswap(a[0], u[i], v[i])
    for i in range(len(r)):
        ctx.cswap(a[0], r[i], s[i])
```

r and s are reduced mod p every round. They have to be: r doubles each round, and the tests confirm it runs past p within a few rounds if left alone. u and v are plain n-bit integers, staying in $[0, p]$ throughout.

Division, and where the garbage goes. What a point addition actually wants is $\lambda = y/x$, and the inversion above leaves $6n$ qubits of per-round records behind. Both are solved at once by the shape of [28]: invert, multiply, copy the answer out, then *run the whole thing backwards*. Only the copy survives; records, counter, u, v and s all return to $|0\rangle$, which the simulator checks at the exact instruction each is freed.

```
def mod_div(m, ctrl, x, y, out, p, inverse_builder=None,
            multiply_builder=None):
    """out ^= ctrl * (y / x mod p).  [106] Fig. 7, the modular division.
    ..."""
    n = len(x)
    lam = m.anc(n, "lam")

    def build():
        inv = m.anc(n, "inv")
        if inverse_builder is None:
            recs, cnt = mod_inv(m, x, inv, p, ctrl=ctrl)
            garbage = [bit for record in recs for bit in record]
            if cnt is not None:
                garbage.append(cnt)
        else:
            garbage = inverse_builder(m, x, inv, p, ctrl)
        if multiply_builder is None:
            import ec_mult as MU
            MU.modmul(m, y, inv, lam, p)
        else:
            multiply_builder(m, y, inv, lam, p)
        return inv, garbage

    # forward: lam = ctrl * y * x^{-1}
    start = len(m.qc.data)
    inv, garbage = build()
    body = list(m.qc.data[start:])

    # lam is already 0 on the ctrl = 0 branch, so the copy needs no control
    for i in range(n):
        m.ctx.cx(lam[i], out[i])

    for ci in reversed(body):                          # undo everything else
        m.qc.append(ci.operation.inverse(), ci.qubits, ci.clbits)
    m.free(lam, inv, *garbage)
```

A control that has to reach all the way down — here, though not everywhere

The division is controlled by the counting qubit q, and for the inversion built above it is not enough to gate only the final copy. On the $q = 0$ branch the x register does not hold $x_1 - x_2$; by that point in the point addition it holds $x_1 + 2x_2$, which vanishes for perfectly ordinary accumulators, so the circuit would be inverting zero — and Kaliski on 0 terminates with $u = p$ and $s = 1$, not the $u = 1$, $s \equiv 0$ that the cleanup here relies on to return those registers to $|0\rangle$.

The fix costs almost nothing. Load the inversion's *setup* under the control — $u \leftarrow p$, $v \leftarrow x$, $s \leftarrow 1$ — and when $q = 0$ every register starts at zero, so $v = 0$ makes all $2n$ rounds inert, r stays 0, and the correction does nothing. The rounds themselves are never controlled; n ANDs and a handful of CNOTs buy the whole thing, and it is checked directly: with $q = 0$ the output register is 0 for every input.

[30] does *not* do this, and does not need to. Its Fig. 7 runs the inversion and the multiplication unconditionally and gates only the copy-out of λ, which is sound there because the inversion is undone by its literal inverse — whatever Inv did to the registers on a degenerate input, $\text{Inv}^\dagger$ undoes. The gating above is the price of *this* construction's cheaper cleanup, not a correction to the paper.

35 Level 4: the eleven-step point addition

The accumulator (x_1, y_1) is quantum, the addend (x_2, y_2) classical, and the control q says whether to add at all. In place, no garbage:

	operation	after it, on the $q = 1$ branch:	
		x register	y register
1	$x \mathrel{-}= x_2$	$x_1 - x_2$	y_1
2	$y \mathrel{-}= q\, y_2$		$y_1 - y_2$
3	$\lambda \mathrel{\oplus}= q \cdot y/x$		
4	$y \mathrel{\oplus}= x\lambda$		$\mathbf{0}$
5	$x \mathrel{+}= 3x_2$	$x_1 + 2x_2$	
6	$x \mathrel{-}= \lambda^2$	$x_2 - x_3$	
7	$y \mathrel{+}= x\lambda$		$\lambda(x_2 - x_3)$
8	$\lambda \mathrel{\oplus}= q \cdot y/x$		
9	$x \mathrel{-}= (2 - q)x_2$	$-x_3$	
10	$y \mathrel{-}= q\, y_2$		y_3
11	$x \leftarrow -x$ if q	x_3	

The circuit is the table, line for line:

```
def point_add_ctrl(m, q, x1, y1, x2, y2, p, multiplier="reference",
                   inversion="kaliski", word_size=None):
    """(x1, y1) <- (x1, y1) + (x2, y2) when q; unchanged when q = 0.
    ..."""
    n = len(x1)
    lam = m.anc(n, "lam")

    MA.modsub_const(m, x1, x2, p)                    # 1  x <- x1 - x2
    MA.cmodsub_const(m, q, y1, y2, p)                # 2  y <- y1 - q y2
    _moddiv(m, q, x1, y1, lam, p, multiplier, word_size, inversion)  # 3
    _modmul_xor(m, x1, lam, y1, p, multiplier, word_size)             # 4
    MA.modadd_const(m, x1, 3 * x2 % p, p)            # 5  x <- x + 3 x2
```

```
    _modsqr_sub(m, lam, x1, p, multiplier, word_size)                  # 6
    _modmul_add(m, x1, lam, y1, p, multiplier, word_size)              # 7
    _moddiv(m, q, x1, y1, lam, p, multiplier, word_size, inversion)    # 8
    _csub_2x2_or_x2(m, q, x1, x2, p)              # 9  x <- -x3
    MA.cmodsub_const(m, q, y1, y2, p)             # 10 y <- y3
    MA.cmodneg(m, q, x1, p)                       # 11 x <- x3

    m.free(lam)
```

Three things in that table are worth pausing on.

Step 4 is an XOR, not a subtraction. At that moment y holds $y_1 - y_2$ and $x\lambda$ is by definition $y_1 - y_2$, so XOR-ing the product in zeroes the register, and the algebra guarantees it. What is saved is the *accumulation* — CNOTs instead of a modular subtraction. The product $x\lambda$ still has to be computed and uncomputed around it, which is two modular multiplications: the step is cheaper than the alternative, not cheap.

Step 8 is the same circuit as step 3, and it clears λ. By then the registers hold $x = x_2 - x_3$ and $y = \lambda(x_2 - x_3)$, so y/x is again λ, and the XOR-accumulating division undoes what step 3 wrote. The whole eleven-step arrangement exists to make that identity true; without it λ would be n qubits of garbage per addition.

Step 9 is two of [29]'s steps merged, following [30]. The original does $x \mathrel{-}= 2x_2$ then a controlled $x \mathrel{+}= x_2$; the merged form subtracts $(2 - q)x_2$ once. Building that operand is free: $2x_2 \bmod p$ and $x_2 \bmod p$ are both classical, so the register is X-gated to one of them and CNOTs from q patch the bits where they differ.

Three exceptional cases, not one

The circuit is correct except on a set of density $O(1/p)$, and it does not detect the exceptions — a check would cost more than it saves. There are three, and only the first is the one usually quoted:

- $x_1 = x_2$ — step 3 divides by $x_1 - x_2$. Doubling, adding an inverse, or infinity.
- $x_3 = x_2$, i.e. $P_1 = -[2]P_2$ — step 8 divides by $x_2 - x_3$. This one is invisible in (8) and appears only once you follow what the *registers* hold. The in-place multiplier of §36.6 reaches the same wall from the other direction, where it would be asked to multiply by zero.
- for the *windowed* circuit of §36.8 adding $\mathcal{O}$: $x_R = 0$. This one is not about the table above — it belongs to [31]'s Algorithm 1, whose own step 6 divides by $x_R - x_{P_i}$, and that is x_R when $P_i = \mathcal{O}$ is encoded as $(0, 0)$. Adding the identity looks like it could never fail.

`ec_classical.point_add_exceptional` decides the first two; the tests exclude them explicitly and report how many, rather than passing quietly on inputs they never exercise.

36 Making it fast

Everything from here on is measured. Each subsection fixes a *task* and varies one implementation choice; comparing a modular doubling against a point addition would say nothing, so nothing here does. Every variant is re-checked against the classical model *inside* the benchmark before its cost is recorded — a cheaper circuit that is wrong is not a datapoint — and variants that are deliberately approximate record a measured failure rate instead. All of it comes from `bench/ec_ablation.py`,

and the tables below are generated from its output rather than typed.

Which Toffoli count, and why it matters here more than in Part V

Both source papers count "Toffolis" as CCX, CCZ and AND gates together [31]. A measurement-based AND† contains *no* Toffoli and no T gate, so it is not counted. That is the convention used in every table below, and it is the one to compare against published figures.
It is not the only defensible convention — one could count every three-qubit operation performed, uncomputes included — and the two differ by nearly 2× on Gidney-style circuits, which is most of what this part builds. `ec_cost` reports both (`toffoli_paper` and `toffoli_equiv`) precisely so the choice is visible rather than buried. Quoting the wrong one would make every optimisation here look roughly half as good as the literature says it is.

36.1 Level 1: which adder, and how to control it

Two adders, chosen per call site by how many ancillas happen to be free — as [31] puts it, the ancilla-hungry one where ancillas are available and the ancilla-free one where they are not.

	addition		controlled addition			qubits	
				copy-then-add, with			
n	CDKM	Gidney	reconstruct	CDKM	Gidney	CDKM	Gidney
8	16	7	32	24	15	17	23
16	32	15	64	48	31	33	47
32	64	31	128	96	63	65	95

Plain addition is the $2n$-versus-n of the textbook comparison, and lands exactly there: $64 = 2n$ against $31 = n - 1$ at $n = 32$. CDKM spends two Toffolis per bit, Gidney one AND per bit, and the difference is that Gidney's targets start clean, so they uncompute for free — the $n - 1$ rather than n being the carry into the top bit, which no adder has to compute.

Controlled addition is where [30] makes its choice. Promoting the adder's CNOTs to Toffolis costs 128 at $n = 32$; copying the operand into clean ancillas *under the control* and then adding unconditionally costs 63, a factor 2.0. The copies all commute, so they also parallelise, which the reconstructed version cannot.

```
def cadd(ctx, ctrl, x, y, copy, anc):
    """y += x if ctrl, via [106] Fig. 4(b): copy under control, then add.
    ..."""
    assert len(copy) == len(x)
    for i, q in enumerate(x):
        ctx.and_(ctrl, q, copy[i])
    add(ctx, copy, y, anc)
    for i, q in enumerate(x):
        ctx.and_dg(ctrl, q, copy[i])
```

The middle column is there to keep that factor honest. Copy-then-add and the Gidney adder are two separate decisions, and the 2.0× bundles them; hold the adder fixed at CDKM and the control style alone is worth 1.33× (96 against 128). The rest is the adder.

What it costs is qubits. The last two columns are the plain adders' widths, $2n+1$ against $3n-1$; for the controlled pair the gap is wider still, 66 against 128 at $n = 32$, since the copy needs a register of its own.

For a classical constant offset, the separate module `gidney_dirty_adder` implements Gidney's dynamic $3n \pm O(1)$ construction with two clean and $n-2$ dirty ancillas. It vents carries by mid-circuit measurement and repairs their phases by classical feed-forward. This path is not composed into the unitary modular adders above: it has no unitary inverse and requires a backend that supports dynamic circuits.

36.2 Level 0: temporary ANDs

Part V measured this for factoring; it applies unchanged, and matters more here because there is so much more arithmetic to apply it to. Over one whole modular inversion:

n	ANDs	AND†s	T (all Toffoli)	T (temporary AND)	saving
5	912	892	14 938	5 958	2.51×
8	2 229	2 197	36 694	14 628	2.51×
12	4 785	4 737	79 254	31 740	2.50×

A flat 2.50×, and the arithmetic says why: ANDs and AND†s arrive in near-equal numbers, so the average is $(4+0)/2 = 2$ against 7, i.e. 3.5× on the ANDs alone, diluted by the Fredkins and dirty-target Toffolis that cannot use the trick.

36.3 Level 2: approximate and pseudo-Mersenne reduction

Shor's circuit only has to work with large constant probability, and [31] spends that freedom on the reductions. Two ideas.

Compare only the top bits. "Is this at least p?" is answered correctly unless the top few bits tie, which happens with probability about $2^{-\text{msbs}}$. That shrinks an n-bit comparison to an msbs-bit one and — more importantly — lets the flag be produced *before* the subtraction rather than recovered after it, collapsing the exact two-step reduction of §33 into a single controlled subtraction.

Exploit the shape of the prime. secp256k1 uses $p = 2^u - f$ with $f = 2^{32} + 977$, tiny against 2^{256}. Then "at least p?" is almost "is bit u set?", and subtracting p becomes "clear bit u, then add f" — and since f is small, adding it need only touch the bottom few bits, because the carry cannot travel far. An n-bit constant subtraction becomes a short one.

The pseudo-Mersenne doubling is the whole idea in eight lines — note that nothing compares anything, and that clearing the overflow bit *is* the subtraction of 2^u:

```
def moddbl_pm(m, x, q, lsbs=None):
    pm = pseudo_mersenne(q)
    assert pm, f"q = {q} is not pseudo-Mersenne"
    u, f = pm
    ctx, n = m.ctx, len(x)
    assert u == n, "expected q just below 2^n"
    lsbs = lsbs or min(n, 2 * max(1, f.bit_length()) + 8)

    z = m.anc(1, "z")
    xe = A.shift_up(x, z[0])                        # n+1 bits; xe[n] is the overflow
    cp, sc = m.anc(lsbs, "cp"), m.anc(lsbs, "sc")
    A.cadd_const(ctx, xe[n], Reg(list(xe[:lsbs])), f, cp, sc)
    m.free(cp, sc)
    ctx.cx(xe[0], xe[n])                            # clears the bit == subtracts 2^u
    for i in range(n - 1, -1, -1):
        ctx.swap(xe[i], xe[i + 1])
    m.free(z)
```

		doubling		controlled addition	
p	reduction	Toffoli	failure	Toffoli	failure
251	exact (Alg. 5 / Alg. 8)	17	0.0%	49	0.0%
251	top 4 bits only (Alg. 6 / Alg. 9)	13	2.4%	39	7.0%
251	pseudo-Mersenne (Alg. 7 / Alg. 10)	5	3.2%	33	3.7%
1021	exact (Alg. 5 / Alg. 8)	21	0.0%	61	0.0%
1021	top 4 bits only (Alg. 6 / Alg. 9)	15	3.0%	45	9.7%
1021	pseudo-Mersenne (Alg. 7 / Alg. 10)	4	3.1%	41	1.3%
4093	exact (Alg. 5 / Alg. 8)	25	0.0%	73	0.0%
4093	top 4 bits only (Alg. 6 / Alg. 9)	17	3.1%	51	9.7%
4093	pseudo-Mersenne (Alg. 7 / Alg. 10)	4	3.1%	49	1.0%

Read the failure column as a rate, not a defect: these circuits are *supposed* to be wrong sometimes. The question is whether the rate tracks $2^{-\text{msbs}}$, and it does — at $p = 4093$, four MSBs give 3.1% and eight give 0.17%, a factor 18 for four bits. The pseudo-Mersenne doubling is the headline: 4 Toffolis against 25 exact, a 6.2× reduction, for a 3.1% failure rate at this width.

Failure compounds, which is why the paper's budget is so large

A single operation failing a few per cent of the time sounds cheap. Chained through the $\approx 1.4n$ iterations of an in-place multiplication it is not. Measured end to end at $p = 31$: a two-bit comparison, which fails about 12% per operation, turns a working multiplier into one that fails 81% of the time, while the full-width comparison stays exact. That is the calculation behind [31]'s choice of 40 to 50 bits at $n = 256$ — per-operation failure near 2^{-40}, so even several hundred iterations stay negligible. A cheap approximation is only cheap once you count how often you use it. The measurement is in `tests/test_ec_opt1128.py`, not an estimate.

One case is not rare. Algorithm 10 cannot reduce $x + y = p$: the sum does not reach bit u. Inside the in-place multiplier that input is certain. r must end the Bézout replay at 0 and only a swap writes r, so every multiplication with $y \neq 0$ presents $r + s = p$ exactly once, at the replay step mirroring the dialog's first swap — near the *end* of the replay, not the start. Algorithms 7 and 10 composed into the multiplier therefore fail on every such input. `ec_approx.cmodadd_pm_q` is Algorithm 11, which catches the case with an all-ones test on the top bits and XORs p out. The all-zeros test that clears its flag is gated here on $ctrl \wedge [y < x]$: the replay's opening iterations make exactly the call ($ctrl = 0$, $y = 0$) on which the ungated test misfires.

36.4 Level 3: Montgomery multiplication over a carry-save tree

[30] rewrites word-level Montgomery so that its two halves have the same shape. The textbook reduction computes $M = c_w p'_w \bmod 2^w$ and then $c \mathrel{+}= Mp$; substituting M gives $c \mathrel{+}= c_w\,(p'_w \bmod 2^w)\,p$, where the bracket depends only on the modulus. So $d = (p'_w \bmod 2^w)\,p$ is a compile-time constant and the reduction becomes a second "$c \mathrel{+}= c_w \cdot d$", structurally identical to the multiplication step. One carry-save tree then serves both. Its 3:2 compressor turns three addends into two with no carry propagation at all,

$$\begin{aligned} S_i &= A_i \oplus B_i \oplus C_i && \text{(three CNOTs)},\\ M_i &= \mathrm{MAJ}(A_i, B_i, C_i) = A_i \oplus \big((A_i \oplus B_i) \wedge (A_i \oplus C_i)\big) && \text{(one AND)}, \end{aligned}$$

with $A + B + C = S + 2M$, so reducing w addends to two costs $O(\log w)$ depth and only the final addition has to propagate a carry.

```
def csa_compress(m, A, B, C):
    """(A,B,C) -> (S, M) with A + B + C == S + M, inputs preserved.
    ..."""
    ctx, W = m.ctx, len(A)
    assert len(B) == len(C) == W
    S, M = m.anc(W, "csaS"), m.anc(W, "csaM")
    for i in range(W):                          # S = A ^ B ^ C
        ctx.cx(A[i], S[i])
        ctx.cx(B[i], S[i])
        ctx.cx(C[i], S[i])
    for i in range(W - 1):                      # M = MAJ, written one place up
        ctx.cx(A[i], B[i])
        ctx.cx(A[i], C[i])
        ctx.and_(B[i], C[i], M[i + 1])
        ctx.cx(A[i], M[i + 1])
        ctx.cx(A[i], B[i])
        ctx.cx(A[i], C[i])
    return S, M
```

multiplier	Toffoli	qubits	depth	depth vs. schoolbook
$n = 8$, $w = 2$, $p = 251$				
schoolbook double-and-add	528	46	3 211	—
Montgomery over a carry-save tree	434	88	2 600	1.2×
Montgomery over a QROM lookup	312	121	1 676	1.9×
$n = 12$, $w = 4$, $p = 4093$				
schoolbook double-and-add	1 176	66	7 311	—
Montgomery over a carry-save tree	1 078	268	4 994	1.5×
Montgomery over a QROM lookup	699	261	3 315	2.2×
$n = 16$, $w = 4$, $p = 65521$				
schoolbook double-and-add	2 080	86	13 043	—
Montgomery over a carry-save tree	1 566	324	6 669	2.0×
Montgomery over a QROM lookup	1 054	347	4 780	2.7×

What this table does *not* show, and why

The carry-save multiplier is a *depth* optimisation, and depth is the column where it wins: 2.0× over schoolbook at $n = 16$. It does not beat the QROM-lookup multiplier of [29] on gate count, and at these widths it should not be expected to — a tree over two or four words has almost nothing to flatten, and the rewrite costs real width. The two differ by $2^w \lfloor c_w p'_w / 2^w \rfloor\, p$, which with c_w and p'_w both below 2^w reaches about $2^{2w} p$ — so the reduction over-adds by that much and the accumulator settles at $n + 2w$ bits rather than the $n + 2$ of textbook Montgomery, with a final reduction to mop up. The paper does not dwell on this; the tests measure the width rather than trusting it.

36.5 Level 3: dropping the counter and postponing the reduction

Three changes to §34, all from [30] Sec 3.3. The first two compound and carry the saving; the third is a scheduling change that buys depth with qubits.

Run every round unconditionally. Drop the "is v still nonzero?" test. An inert round does nothing except double r, so instead of stopping at $-x^{-1}2^k$ the register runs on to $-x^{-1}2^{2n}$ — the

same value for *every* input, with no counter to keep and no $2n - k$ corrective halvings to perform. And 2^{2n} is exactly right: fed $x = X2^n$ in Montgomery form the answer is $-X^{-1}2^n$, the Montgomery form of $-X^{-1}$, so the power of two has corrected itself and only the sign is left — one modular negation, against [29]'s counter and its $2n - k$ controlled halvings. It also removes one of the three recorded bits per round, since f existed only to say whether the round fired.

Postpone the modular reductions. Do not reduce r and s mod p each round; let them run into a $2n$-bit register and reduce once at the end. This changes what a round *is*: a modular doubling (compare, conditionally subtract, clear the flag) becomes a plain left shift, which is a renaming of wires and costs no Toffoli at all, and a modular addition becomes a plain addition.

	Toffoli			depth			qubits	
n	[29]	[30]	saved	[29]	[30]	saved	[29]	[30]
5	1 242	817	34%	4 846	2 664	45%	83	84
7	2 328	1 593	32%	9 358	5 342	43%	109	116
8	3 045	2 077	32%	12 271	7 019	43%	125	132
10	4 647	3 237	30%	19 021	11 081	42%	151	164
12	6 585	4 653	29%	27 239	16 059	41%	177	196

At $n = 12$: 6 585 Toffolis become 4 653, a 29% saving, and depth falls by 41%, for about 11% more qubits.

[30] reports 58–60% for division at $n = 192$–521, but on a different metric: the *product* of qubit count and T-depth, which is the trade-off measure that paper uses throughout. The columns above are plain counts, which is why they are not directly comparable — a construction that spends qubits to buy depth looks better under a product metric than under either factor alone. The direction agrees; the magnitudes should not be read against each other.

Fan the swap control out. Each round swaps (u, v) and (r, s) under the decision bit a, twice — $6n$ controlled swaps hanging off a single qubit, which must therefore run one after another. `kaliski_round_opt` copies a into k clean ancillas (`fanout`, default $k = 4$) and controls swap i on copy i mod k, so k swaps can run in the same layer; the copies are cleared with the same CNOTs at the end of the round. The Toffoli count does not move, since $2k$ CNOTs are free in this accounting, and the width grows by $k - 1$ — the 84 qubits in the $n = 5$ row below would be 81 at $k = 1$. What the fan-out does *not* reliably buy at these sizes is depth. Within one round it helps (281 against 288 layers at $n = 7$, $k = 4$ against $k = 1$). Across the whole inversion, though, the k ancillas are freed last and recycled as the next round's comparator scratch, so the next round queues behind the un-fan-out and the total grows: 5 342 layers at $k = 4$ against 4 793 at $k = 1$ and 4 708 at $k = 2$, for $n = 7$. A doubling-tree fan-out (depth $\lceil \log_2(k + 1) \rceil$) changes this by under 1%, so the cost is the allocation order rather than the copying. The table keeps $k = 4$ to stay comparable with [30], whose fan-out draws on the following multiplier's idle scratch rather than fresh qubits.

The $2n$-bit budget is not a comfortable margin, it is the exact requirement — r reaches 6 bits for $p = 7$ and 32 bits for $p = 65521$, both exactly $2n$. The test measures it rather than assuming it.

36.6 Level 3, rebuilt: the Euclidean dialog

Everything so far treats inversion and multiplication as two jobs. Schrottenloher [31], following [33], observes that they are one, and this is the single largest structural saving in this part.

Run the binary GCD on (u, v) alone and *do not* maintain the Bézout coefficients. Just record the two decision bits per iteration — call the record a *dialog*. It is 1.5 bits per iteration on average, against the $2n$ qubits carrying (r, s) would cost:

repeat $\approx 1.413n$ times:
1. $b_0 := v \bmod 2$; $b_1 := [\,u > v\,]$ record b_0 and $b_0 b_1$
2. **if** $b_0 b_1$: swap (u, v)
3. **if** b_0: $v \mathrel{-}= u$
4. $v \mathrel{/}= 2$

The iteration count is not arbitrary. Halving v always removes a bit; the subtraction removes a second one half the time. [31] averages the shrink factor of uv over those two cases, $\frac{1}{2} \cdot \frac{1}{2} + \frac{1}{2} \cdot \frac{1}{4} = \frac{3}{8}$, and reads off $2n/\log_2(8/3) \approx 1.413n$ iterations. That step is heuristic — it averages the factor rather than its logarithm, which is not the same thing — but the conclusion holds: measured on this implementation's own dialog, the mean iteration count is $1.383n$ at $n = 32$ and $1.409n$ at $n = 256$, climbing towards $1.413n$. The spread is $\approx 0.6\sqrt{n}$, which is what the $+c\sqrt{n}$ safety margin is for.

```
def dialog_round(m, u, v, rec):
    """One iteration of [1128] Algorithm 2.  `rec` = (b0, b0b1), kept.
    ..."""
    ctx, n = m.ctx, len(u)
    b0, b0b1 = rec
    ctx.cx(v[0], b0[0])                                  # b0 = v mod 2

    t, sc = m.anc(1, "t"), m.anc(n, "sc")
    A.gt_uint(ctx, u, v, t[0], sc)                       # t = [u > v]
    ctx.and_(b0[0], t[0], b0b1[0])                       # b0b1 = b0 AND b1
    A.gt_uint(ctx, u, v, t[0], sc)
    m.free(t, sc)

    for i in range(n):
        ctx.cswap(b0b1[0], u[i], v[i])                   # if b0b1: swap u, v

    cp, sc = m.anc(n, "cp"), m.anc(n, "sc")
    A.csub(ctx, b0[0], u, v, cp, sc)                     # if b0: v -= u
    m.free(cp, sc)

    for i in range(n - 1):                               # v /= 2 (v is even now)
        ctx.swap(v[i], v[i + 1])
```

Now the second idea. The updates to (r, s) that the full extended algorithm would have performed are all *linear* and depend only on those two bits, so they can be replayed afterwards, onto any starting pair:

$$s \leftarrow 2s \bmod p; \qquad \text{if } b_0: \; s \mathrel{+}= r; \qquad \text{if } b_0 b_1: \; \mathrm{swap}(r, s)$$

Read *forwards*, alongside the Euclidean algorithm as the textbook extended version would, this maps $(1, 0) \mapsto (0,\, x^{-1})$ — an inversion — and more generally $(y, 0) \mapsto (0,\, yx^{-1})$, which is an inversion and a multiplication for the price of one pass. That is [31]'s observation, and it is already the saving.

But the circuit reads the record *backwards*, and gets a different map: $(y, 0) \mapsto (0,\, yx)$. Multiplication by x, not by x^{-1}. That is not a defect but the reason the backwards reading is chosen: replayed in reverse the updates need only *doubling* and addition, where the forward reading needs modular *halving*. And since the whole thing is a circuit, running it in the other direction recovers the division. One component, both of the point addition's multiplicative steps.

The replay is seven lines, and the whole saving is that none of them looks at x:

```
def bezout_replay(m, r, s, recs, q, dbl=None, cadd=None, compressed=False):
    """(r, s) updated by [1128] Algorithm 3, reading the record in reverse.
    ..."""
```

```
    if dbl is None:
        state = {"s": s}

        def do_dbl():
            state["s"] = MA.moddbl_rotate(m, state["s"], q)
    else:
        state = {"s": s}

        def do_dbl():
            dbl(m, state["s"])
    cadd = cadd or (lambda mm, c, a, b: MA.cmodadd(mm, c, a, b, q))

    def replay_round(b0, b0b1):
        do_dbl()                                        # s <- 2s mod q
        s = state["s"]
        cadd(m, b0[0], r, s)                            # if b0: s += r mod q
        for i in range(len(r)):
            m.ctx.cswap(b0b1[0], r[i], s[i])            # if b0b1: swap r, s

    if compressed:
        full = len(recs) - len(recs) % 3
        for i in reversed(range(full, len(recs))):
            replay_round(*recs[i])
        for start in reversed(range(0, full, 3)):
            group_recs = recs[start:start + 3]
            group = _record_wires(group_recs)
            m.emit_inverse(compress_triple_circuit, m, group)
            for rec in reversed(group_recs):
                replay_round(*rec)
            compress_triple_circuit(m, group)
    else:
        for rec in reversed(recs):
            replay_round(*rec)
    return state["s"]
```

Assembling the three phases — dialog, replay, dialog backwards — gives the in-place multiplication, and the rebuild consumes the record on its way out, so nothing is left behind:

```
def inplace_mul(m, x, y, q, iters=None, dbl=None, cadd=None,
                register_sharing=False, c_pad=2.3, c_iter=2.4, compress=False):
    """|x, y> -> |x, y*x mod q>.  [1128] Algorithm 4.

    ..."""
    n = len(x)
    iters = iters or eea_iterations(n, c_iter)
    if register_sharing:
        return _inplace_mul_register_sharing(
            m, x, y, q, iters, dbl, cadd, c_pad, c_iter)
    if not compress:
        return _inplace_mul_plain(m, x, y, q, iters, dbl, cadd)

    recs, u, v = dialog(m, x, q, iters, compress=True)   # 1: x -> dialog
    clear_dialog_end(m, u, v)

    full = len(recs) - len(recs) % 3
```

```
    spares = [Reg([_record_wires(recs[i:i + 3])[5]])
              for i in range(0, full, 3)]
    m.reserve(*spares)
    s = m.anc(n, "bz")                                  # 2: replay onto (y, 0)
    s = bezout_replay(m, y, s, recs, q, dbl, cadd, compressed=True)
    for a, b in zip(y, s):                              # r is 0, s is the answer
        m.ctx.swap(a, b)
    m.free(s)

    hi = m.anc(1, "vhi")                                # 3: dialog -> x
    u2 = m.anc(n + 1, "u")
    v2 = Reg(list(x) + list(hi), "v")
    m.ctx.x(u2[0])                                      # the dialog ends at u = 1
    for i in reversed(range(full, len(recs))):
        m.emit_inverse(dialog_round, m, u2, v2, recs[i])
        m.free(*recs[i])
    for start in reversed(range(0, full, 3)):
        group_recs = recs[start:start + 3]
        group = _record_wires(group_recs)
        m.emit_inverse(compress_triple_circuit, m, group)
        for rec in reversed(group_recs):
            m.emit_inverse(dialog_round, m, u2, v2, rec)
        m.free(Reg(group))
    A.encode_const(m.ctx, u2, q)                        # and starts at u = q
    m.free(u2, hi)
```

Direction matters, and it is easy to get wrong

Applying Algorithm 3's update rules in the record's original order is *not* the inverse of applying them in reverse, and it is not the transpose either — it is simply a different linear map. Writing "$(1, 0)$ gives x^{-1}" next to "$(y, 0)$ gives yx" is self-contradictory, since the second is linear in y and contains the first at $y = 1$; an earlier draft of this section said exactly that, and the code is what caught it.

The classical model in `ec_classical.py` carries all three maps and the tests pin down which is which: `bezout_replay` (reversed) multiplies by x, `bezout_replay_inv` divides by it, and `bezout_inverse_classical` — an independent coefficient-tracking implementation that needs modular halving where the replay needs only doubling — yields x^{-1} directly. The third exists purely as a cross-check, which is why the direction claim is not taken on faith.

Compared against what it replaces — Kaliski inversion plus an out-of-place multiplication, which is what a point addition otherwise needs:

	invert-then-multiply		dialog + replay		saving	
p	Toffoli	qubits	Toffoli	qubits	Toffoli	qubits
7	1 095	64	612	39	44%	39%
11	1 910	83	957	48	50%	42%
31	2 889	99	1 378	57	52%	42%
61	4 068	115	1 875	66	54%	43%

At $p = 61$ that is 54% of the gates and 43% of the qubits, and the margin widens with n. The record itself is squeezed further by a trick worth its own paragraph: each pair $(b_0, b_0 b_1)$ can only be $(0, 0)$,

$(1,0)$ or $(1,1)$ — never $(0,1)$, since $b_0 b_1$ cannot be set unless b_0 is — so three pairs carry $3^3 = 27 < 32$ states and fit in five bits instead of six. One qubit freed per three iterations, taking the record from $2.83n$ to $2.355n$ asymptotically, and (unlike a shift-based encoding) leaving it a *fixed* size, which is what lets the rest of the circuit be laid out statically.

That packing is implemented (`compress_triple_circuit`, with its inverse used in the replay) but it is *standalone and opt-in*: `inplace_mul(..., compress=True)`. The multiplier measured in the table above, and used everywhere else in this part, is the unpacked one. At these sizes packing saves no peak width — the peak sits in the replay's scratch, not in the record — and the encode/decode network costs Toffolis: 672 against 612 at $p = 7$, and 1 975 against 1 875 at $p = 61$, at the same 39 and 66 qubits. The packing pays only where the record is the widest live thing, which is the cryptographic regime of §37.1.

36.7 Level 4, rebuilt: projective coordinates

The other way to make an inversion cheap is not to do it. In Jacobian coordinates $(X : Y : Z) \sim (x, y) = (X/Z^2, Y/Z^3)$ the addition law is inversion-free, and with the addend in affine form ($Z_2 = 1$, which it is, being a classical point) every term involving Z_2 drops out:

$$\begin{aligned} &U_2 = x_2 Z_1^2, && S_2 = y_2 Z_1^3, && H = U_2 - X_1, \quad R = S_2 - Y_1, \\ &X_3 = R^2 - H^3 - 2X_1 H^2, && Y_3 = R(X_1 H^2 - X_3) - Y_1 H^3, && Z_3 = Z_1 H. \end{aligned}$$

Eleven multiplications, no division — [30] reports this as the fewest of any quantum projective addition, against a previous best of twelve.

The catch is that projective coordinates are not a unique representation. $(P + Q) - Q$ returns a *different triple* for the same point, so the input cannot be erased and every addition leaves its accumulator behind. The fifteen intermediates *can* all be cleaned — each by reversing the step that made it, with that step's inputs still live — and this implementation cleans them, but the input point is $3n$ qubits of irreducible garbage per addition.

The eighteen steps, and then the unwinding — note the last two lines, which are the whole subtlety:

```
def jacobian_add(m, X1, Y1, Z1, x2, y2, X3, Y3, Z3, p):
    """(X3:Y3:Z3) <- (X1:Y1:Z1) + (x2, y2).  [106] Algorithm 4.
    ..."""
    n = len(X1)
    A = m.anc

    ...

    Z1sq = A(n, "Z1sq"); _, b1 = m.step(sqr, Z1, Z1sq)                    # 1
    U2 = A(n, "U2");     _, b2 = m.step(MU.modmul_const, m, Z1sq, x2, U2, p)   # 2
    Z1cu = A(n, "Z1cu"); _, b3 = m.step(mul, Z1sq, Z1, Z1cu)              # 3
    H = A(n, "H");       _, b4 = m.step(sub_into, U2, X1, H)              # 4
    ...
    def mk2V():
        for q, r in zip(V, twoV):
            m.ctx.cx(q, r)
        MA.modadd(m, V, twoV, p)
    _, b11 = m.step(mk2V)                                                 # 11

    T0 = A(n, "T0");     _, b12 = m.step(sub_into, Rsq, Hcu, T0)          # 12
    _, b13 = m.step(sub_into, T0, twoV, X3)                       # 13  OUTPUT
```

```
    T1 = A(n, "T1");      _, b14 = m.step(sub_into, V, X3, T1)              # 14
    T2 = A(n, "T2");      _, b15 = m.step(mul, R, T1, T2)                   # 15
    T3 = A(n, "T3");      _, b16 = m.step(mul, Y1, Hcu, T3)                 # 16
    _, b17 = m.step(sub_into, T2, T3, Y3)                            # 17  OUTPUT
    _, b18 = m.step(mul, Z1, H, Z3)                                  # 18  OUTPUT

    # unwind everything but the three outputs.  Note T0 goes back via step 12,
    # not step 13 -- step 13 wrote X3, which stays.
    for body in (b16, b15, b14, b12, b11, b10, b9, b8, b7, b6, b5, b4, b3, b2, b1):
        m.undo(body)
    m.free(T3, T2, T1, T0, twoV, V, Rsq, R, S2, Hcu, Hsq, H, Z1cu, U2, Z1sq)
```

The order of unwinding is not simply "backwards"

X_3 is an output and must survive, so the intermediate T_0 cannot be released by reversing the step that consumed it. It is released by reversing the step that *produced* it, from R^2 and H^3, which are still live at that moment. Getting this wrong destroys the answer; `Machine.undo` exists precisely so that a recorded step can be reversed out of order, and the tests sweep the projective representatives Z of every input point — $Z \in \{1, 2\}$ by default and all of $[1, p)$ under `SHOR_EC_FULL=1` — a sweep that catches representation bugs an affine-only test cannot see.

36.8 Level 5: windowing, zig-zag, and the semiclassical transform

Three counting arguments, none of which changes a single arithmetic circuit.

Windowing [**?**] groups w control qubits into one address and looks up P_i from a table of 2^w precomputed points, so $2n + 2$ additions become $2\lceil (n+1)/w \rceil$. A lookup costs 2^w Toffolis, an order of magnitude below an addition, so w is chosen large — both [31] and [32] use $w = 16$.

Zig-zag [36] attacks the projective garbage. Write ρ for the number of accumulator registers — not m, which above is the counting-register width. Additions are arranged in runs of $\rho, \rho - 1, \ldots, 1$: fill all ρ registers, then walk back un-computing all but the last of the run — legal, because each state's predecessor is still live — returning $\rho - 1$ registers, and repeat. Total forward additions $\rho(\rho + 1)/2$, so ρ registers serve them all.

The semiclassical transform [**?**] is the same trick as §21: every controlled rotation in the inverse QFT is controlled by a qubit about to be measured, so measure it first and make the control classical. Both scalar registers collapse to *one recycled qubit*. This is the assumption behind both papers reporting only the cost of the point arithmetic.

The windowed addition itself, with the three lookups and the two in-place multiplications that replace §35's two divisions:

```
def windowed_point_add(m, addr, x2, y2, points, p, iters=None):
    """[1128] Algorithm 1.  `points[i]` is the window entry for address i.
    ..."""
    ctx, n = m.ctx, len(x2)
    w = len(addr)
    assert len(points) == 1 << w, "the window must be full"
    assert points[0].inf, "entry 0 must be the point at infinity"

    ...

    one = m.anc(1, "one")
    ctx.x(one[0])
```

```
c = m.anc(1, "c")
MA.is_nonzero(m, addr, c[0])                    # 1

x1, y1 = m.anc(n, "x1"), m.anc(n, "y1")
_lookup(m, one[0], addr, x1, xs)                # 2
_lookup(m, one[0], addr, y1, ys)
MA.modsub(m, x1, x2, p)                         # 3
MA.modsub(m, y1, y2, p)                         # 4
_lookup(m, one[0], addr, x1, xs)                # 5
_lookup(m, one[0], addr, y1, ys)

E.inplace_div(m, x2, y2, p, iters)              # 6  y2 <- y2 / x2

xx = m.anc(n, "3x")
_lookup(m, one[0], addr, xx, x3s)               # 7
MA.modadd(m, xx, x2, p)                         # 8
_lookup(m, one[0], addr, xx, x3s)               # 9
m.free(xx)

_csub_square(m, c[0], y2, x2, p)                # 10 if c: x2 -= y2^2
E.inplace_mul(m, x2, y2, p, iters)              # 11 y2 <- y2 * x2
MA.cmodneg(m, c[0], x2, p)                      # 12 if c: x2 <- -x2
...
```

	point additions			zig-zag registers			garbage qubits	
n	none	w=11	w=16	none	w=11	w=16	none	w=16
192	386	36	26	28	8	7	16 128	4 032
256	514	48	34	32	10	8	24 576	6 144
384	770	70	50	39	12	10	44 928	11 520
521	1 044	96	66	46	14	11	71 898	17 193

At $n = 256$: 514 additions become 34, a 15× reduction, and the registers holding projective garbage fall from 32 to 8. The two compose — windowing cuts how many additions there are, zig-zag cuts how many of their outputs are live at once — and together they take the garbage from 24 576 qubits to 6 144.

37 Everything at once

The subsections above each hold one thing fixed. This one does not: it stacks them, on the operation that actually matters — one point addition — and then on the whole circuit.

Two different kinds of number appear below, and they should not be read against each other. The point-addition table and the keybox are *component comparisons*: each construction is built and measured on its own, with the arithmetic options that construction names. The whole-circuit table at the end is a single *reference* build: `ecdlp_circuit(..., oracle="arith")` with its defaults, which are the affine addition of Alg. 3 with the reference multiplier and the [29] Kaliski inversion — no windowing, no dialog, no projective coordinates, none of the Level 1–4 optimisations. It is there to show the size of a complete, simulated, correct circuit, not the best one the components above could assemble. (`ec_shor` does accept the optimised backends, `arithmetic="windowed"`, `multiplier=`, `inversion=`, and the test suite runs them end to end; they are not what that row measures.)

curve	construction	Toffoli	qubits	depth	vs. affine
Alg. 3, Alg. 4 are [30]'s; Alg. 1 is [31]'s.					
$p = 7$	affine, in place (Alg. 3)	2 756	64	11 886	—
	Jacobian, out of place (Alg. 4)	1 544	75	8 511	44%
$p = 11$	affine, in place (Alg. 3)	4 769	83	21 023	—
	Jacobian, out of place (Alg. 4)	2 642	98	15 126	45%
$p = 7$	windowed w=2, dialog mult. (Alg. 1)	1 523	49	6 907	45%

Three constructions of the same map, and they stack differently. The affine addition is in place and leaves nothing behind, and pays for that with an inversion. The Jacobian addition removes the inversion entirely and is 45% cheaper, but leaves $3n$ qubits of input point as garbage — which is only affordable once windowing and zig-zag have cut how many such inputs are live at once. The windowed addition is cheapest of the three at 45% below affine, and it gets there differently again: not by changing coordinates, but by replacing invert-then-multiply with a single Euclidean dialog, and by amortising three table lookups over a whole window of additions.

How the savings compose — and where they do not

Read down the stack for one point addition at $n = 256$, taking each factor from the measurements above:

- *Gate-level, and only partly independent.* The Gidney adder ($\approx 2\times$ on the inner additions) and the temporary AND ($2.50\times$ on T) are *not* two savings: they are one saving counted at two levels. The adder wins by choosing a gate whose target starts clean, and the temporary-AND accounting is exactly what makes that gate's uncompute free. Multiplying the two would double-count. Copy-then-add controls are a smaller further factor than the table suggests: the $2.0\times$ there bundles the control style together with the switch to Gidney's adder inside it. Hold the adder fixed and the control style alone is worth $1.33\times$ (96 against 128 at $n = 32$, CDKM on both sides) — it buys parallelism and depth more than gates. Pseudo-Mersenne reduction ($6.2\times$ on doublings) is the one that cleanly composes, because it acts on a different part of the circuit again: the reductions rather than the additions.
- *Structural, and they replace rather than compose.* The dialog (54% off the multiplicative step) and projective coordinates (45% off the addition) are two different answers to the same problem — the inversion — so you take one, not both. Every published construction picks a side.
- *Count-level, and these do multiply the whole thing.* Windowing divides the number of point additions by 15 at $n = 256$; zig-zag divides the live projective garbage by 4; the semiclassical transform deletes both counting registers. None of them touches an arithmetic circuit, which is precisely why they compose cleanly with everything above.

The last group dominates. An optimisation that halves a point addition is worth half as much as one that runs fifteen times fewer of them, which is why both papers spend their first section on windowing and their last on gate counts.

And the whole circuit — the reference-affine arithmetic oracle, not a stack of the optimisations above — on the toy curve $y^2 = x^3 + 5x + 4 \bmod 7$ with $\mathrm{ord}(G) = 5$:

ECDLP circuit	qubits	Toffoli	gates	depth
real arithmetic, two counting registers	69	16 488	105 339	70 967
permutation oracle, two counting registers	12	—	603	282
permutation oracle, semiclassical QFT	7	—	619	287

69 qubits and 105 339 gates to compute a discrete logarithm in a group of order five. That ratio is the honest summary of this part: the arithmetic is enormous, and the counting-level tricks are the only reason cryptographic sizes are discussed at all. Note the last two rows — the semiclassical transform takes 12 qubits to 7 by recycling one control qubit, which is precisely why both papers report only the point arithmetic and treat the scalar registers as free.

37.1 Space at cryptographic size

The one place where a projection is worth making, because it is what the papers' headline qubit counts are built from. The dialog record dominates the space of [31]'s construction. Every column here is the `ec_cost` closed form, not a circuit measurement, and the "Fig. 1 packing" column prices the standalone packing of §36.6, which the measured multiplier does not use:

n	record, raw	with Fig. 1 packing	per n	with register sharing
256	802	669	$2.613n$	579
384	1 180	984	$2.562n$	859
521	1 582	1 319	$2.532n$	1 157

The middle column converges to the paper's $2.355n$ from above — the gap is the $O(\sqrt{n})$ safety margin on the iteration count, which is a constant fraction at these sizes and vanishes asymptotically. Register sharing is available as an opt-in layout: it shrinks u and v and reuses their cleared wires. Its overflow probability depends on the padding and input distribution; the basis-state simulator detects overflow rather than truncating. The implementation keeps fixed five-bit record packing, so these measured circuit widths do not claim the paper's tighter $2.12n + O(\sqrt{n})$ projection.

The paper's own closed form for the total — its asymptotic $2.355n$ record plus $2n$ for the pair of modular integers the reconstruction holds, i.e. $4.355n + O(\sqrt{n})$ — gives 1151 qubits at $n = 256$ against the paper's 1192, and 1407 against 1446 for the gate-optimised variant. That is the paper's formula evaluated, *not* the table above summed: the table's middle column carries its $O(\sqrt{n})$ margin explicitly and would give a larger figure. Neither is a check on the other.

The full-algorithm count of $2^{26.11}$ Toffolis is likewise the paper's equation (1) evaluated at the paper's own per-addition figure — restated for scale, not independently confirmed. Nothing in this package measures a 256-bit point addition.

38 Verifying it

A wrong ECDLP circuit does not return a wrong answer. It returns a *flat histogram* — and since the classical post-processing verifies its own candidates against the curve, it will happily print the correct k from pure noise. Testing the output is therefore worth almost nothing; three other things are worth a great deal.

Exact simulation at full width. Every arithmetic circuit here is a permutation of basis states, so pushing one basis state through in $O(\text{gates})$ time is exact — and scales to hundreds of qubits where a statevector dies at thirty. Ancillas are checked back to $|0\rangle$ *at the instruction where they are*

freed, not merely at the end, which is what catches a circuit that computes the right value and leaves a dirty flag behind. This is sound only for circuits whose phases carry no information; the sole phase and Hadamard gates outside the AND gadget are in the state preparation and the Fourier transform, which are run on Aer instead.

```
def simulate(qc, init=None, checks=(), strict=True):
    """Push one basis state through ‘qc‘ and return the final bit assignment.

    ..."""
    idx = {q: i for i, q in enumerate(qc.qubits)}
    bits = [0] * qc.num_qubits
    if init:
        for k, v in init.items():
            bits[idx[k] if k in idx else k] = int(v) & 1

    ...
        name = op.name
        w = [idx[q] for q in qargs]

        if name in _SKIP or name in _DIAGONAL:
            return
        if name == "x":
            bits[w[0]] ^= 1
            return
        if name == "swap":
            bits[w[0]], bits[w[1]] = bits[w[1]], bits[w[0]]
            return
        if name == "ecand":
            if strict and bits[w[2]] != 0:
                raise SimError("AND target was not |0>: temporary AND invalid")
            bits[w[2]] = bits[w[0]] & bits[w[1]]
            return
        if name == "ecand_dg":
            if strict and bits[w[2]] != (bits[w[0]] & bits[w[1]]):
                raise SimError("AND-dagger target did not hold a AND b")
            bits[w[2]] = 0
            ...
```

Statevector cross-checks. The fast simulator only sees basis states, and a bug in it would hide from every test that uses it. So the small circuits are also evolved on genuine superpositions and checked for exact fidelity *and* for the absence of leaked amplitude — the statement that ancillas are unentangled, which basis-state testing cannot see. A stray diagonal gate would appear here as a phase, and phases are exactly what Shor depends on.

The listings, too. Every listing in this part is extracted from `shor_qiskit/ec_*.py` by AST at build time, not copied by hand, and the whole part is regenerated and diffed against the document by `bench/ec_check_tex.py`. So a listing cannot drift from the function it claims to show: change the code and the document either follows or the check fails. Where a listing is shortened, the elision is visible as a bare `...` — nothing is silently trimmed.

Property-based tests over random curves. Random prime, random non-singular (a, b), random points and scalars. That is where curve-shape assumptions hide: $a = 0$, $b = 0$, curves with a point of order two, primes where 2 is or is not a square. The harness shrinks failures — staying inside each field's declared domain, and re-checking its own result before reporting it — and it has been validated against injected bugs, where it isolates the exact offending modulus.

what	result
classical models (Kaliski, Montgomery, dialog, Jacobian)	exact
adders, comparators, modular arithmetic	exhaustive $p \leq 61$, random to $n = 128$
Kaliski inversion and division	exhaustive; records and counter clean
affine point addition	every point pair, both control branches
projective point addition	every point pair; every Z under `SHOR_EC_FULL`
in-place multiplication (dialog)	every (x, y) for $p \leq 31$ ($p \leq 61$ full)
ECDLP end to end	every logarithm on two toy curves

38.1 The distribution, and the control that makes it mean something

On $y^2 = x^3 + 5x + 4 \bmod 7$ with $G = (0, 5)$ of order 5, and $Q = [2]G$, the measured peaks sit at $(j_1, j_2) \in \{(0, 0), (6, 5), (2, 3), (3, 6), (5, 2)\}$ carrying 20%, 10%, 10%, 10%, 10% of the shots. Rounding by $a_i = \text{round}(j_i \cdot 5/8)$ turns those into $(a_1, a_2) = (1, 2), (2, 4), (3, 1), (4, 3)$ — and every one of them satisfies $a_2 = 2a_1 \bmod 5$, which is (6) with $k = 2$.

```
def ecdlp_postprocess(counts, order, bits, search=1):
    """Recover k from measured (j1, j2) pairs of the two ECDLP registers.

    `counts` maps (j1, j2) -> number of shots.  Returns [(k, weight)] sorted by
    weight, heaviest first.
    ..."""
    if isinstance(counts, dict):
        items = list(counts.items())
    else:
        items = [(pair, 1) for pair in counts]

    votes, q = {}, 1 << bits
    for (j1, j2), c in items:
        for da in range(-search, search + 1):
            for db in range(-search, search + 1):
                a1 = (round(j1 * order / q) + da) % order
                a2 = (round(j2 * order / q) + db) % order
                if gcd(a1, order) != 1:
                    continue
                k = a2 * pow(a1, -1, order) % order
                w = c / (1 + abs(da) + abs(db)) ** 2
                votes[k] = votes.get(k, 0.0) + w
    return sorted(votes.items(), key=lambda kv: -kv[1])
```

The control that catches a broken circuit

The correct k must be the *top* candidate, not merely *a* candidate that verifies. On both toy curves it is, for every logarithm, holding at least 30% of the vote against a uniform 20% and 7.7% respectively. And feeding the same post-processing a flat, random histogram returns a top share of 21.2% against a uniform 20% — no peak, no signal.

That control exists because an early version of this post-processing did not have it. Ekerå's limited search [34] widens the rounding when 2^m is not a multiple of r, and with the radius set too generously it enumerated the entire group: every k "verified", on every input, including noise. The bug was invisible in the pass/fail of an end-to-end test and obvious the moment the vote shares were printed next to the uniform baseline.

39 What is not here

Named rather than glossed:

- **Composing Gidney's dirty-ancilla adder** [35] into the modular arithmetic. The standalone dynamic adder is implemented, but the surrounding arithmetic still requires unitary inverse emission and does not yet use it.
- **The Ed25519 extension** of [30] Sec 4.4, in extended Edwards coordinates. The curve model here is short Weierstrass only.
- **Physical-level estimates** — surface codes, QLDPC, wall-clock runtime. This part stops at the logical layer, as Parts II–VI do.

The Montgomery implementation now exposes both layouts from [30]: the *depth-optimised* form allocates fresh accumulator outputs per carry-save addition, while the *qubit-optimised* form reuses the accumulator in place. The former uses more qubits and is shallower; both are checked against the same Montgomery reference by the tests.

Where this part sits

The affine construction is [28] as improved by [29], with [30]'s modifications: 2017–2020 with a 2026 finish. The projective addition and the carry-save Montgomery multiplier are [30]. The Euclidean dialog, the approximate and pseudo-Mersenne arithmetic, and the windowed addition are [31]. What is deliberately absent is the line that leaves this architecture entirely — the residue-number-system constructions that cut qubit counts by changing what a number *is*, rather than how it is added.

The honest limit of what runs on a laptop: the circuit with real arithmetic is verified on every basis state, which for a permutation is the whole truth, but at 69 qubits for a seven-element field it cannot be run in superposition. The end-to-end demonstration uses a permutation oracle of identical circuit shape. Both halves are tested; the join between them is an argument, not a simulation.

References

[1] P. W. Shor, *Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer*, SIAM Review **41**(2):303–332, 1999 (SIAM J. Comput. **26**(5):1484–1509, 1997). arXiv:quant-ph/9508027.

[2] A. Yu. Kitaev, *Quantum measurements and the Abelian stabilizer problem*, 1995. arXiv:quant-ph/9511026.

[3] R. Cleve, A. Ekert, C. Macchiavello, and M. Mosca, *Quantum algorithms revisited*, Proc. R. Soc. Lond. A **454**:339–354, 1998. arXiv:quant-ph/9708016.

[4] V. Vedral, A. Barenco, and A. Ekert, *Quantum networks for elementary arithmetic operations*, Phys. Rev. A **54**:147–153, 1996. arXiv:quant-ph/9511018.

[5] D. Beckman, A. N. Chari, S. Devabhaktuni, and J. Preskill, *Efficient networks for quantum factoring*, Phys. Rev. A **54**:1034–1063, 1996. arXiv:quant-ph/9602016.

[6] C. Zalka, *Fast versions of Shor's quantum factoring algorithm*, 1998. arXiv:quant-ph/9806084.

[7] T. G. Draper, *Addition on a quantum computer*, 2000. arXiv:quant-ph/0008033.

[8] S. Beauregard, *Circuit for Shor's algorithm using* $2n + 3$ *qubits*, Quantum Inf. Comput. **3**(2):175–185, 2003. arXiv:quant-ph/0205095.

[9] S. A. Cuccaro, T. G. Draper, S. A. Kutin, and D. P. Moulton, *A new quantum ripple-carry addition circuit*, 2004. arXiv:quant-ph/0410184.

[10] A. Pavlidis and D. Gizopoulos, *Fast quantum modular exponentiation architecture for Shor's factoring algorithm*, Quantum Inf. Comput. **14**(7–8):649–682, 2014. arXiv:1207.0511.

[11] A. Barenco, A. Ekert, K.-A. Suominen, and P. Törmä, *Approximate quantum Fourier transform and decoherence*, Phys. Rev. A **54**:139–146, 1996. arXiv:quant-ph/9601018.

[12] D. Cheung, *Improved bounds for the approximate QFT*, 2004. arXiv:quant-ph/0403071.

[13] R. B. Griffiths and C.-S. Niu, *Semiclassical Fourier transform for quantum computation*, Phys. Rev. Lett. **76**:3228–3231, 1996. arXiv:quant-ph/9511007.

[14] M. Mosca and A. Ekert, *The hidden subgroup problem and eigenvalue estimation on a quantum computer*, Proc. QCQC'98, LNCS 1509, 1999. arXiv:quant-ph/9903071.

[15] S. Parker and M. B. Plenio, *Efficient factorization with a single pure qubit and* $\log N$ *mixed qubits*, Phys. Rev. Lett. **85**:3049–3052, 2000. arXiv:quant-ph/0001066.

[16] C. Gidney, *Halving the cost of quantum addition*, Quantum **2**:74, 2018. arXiv:1709.06648.

[17] R. Babbush, C. Gidney, D. W. Berry, N. Wiebe, J. McClean, A. Paler, A. Fowler, and H. Neven, *Encoding electronic spectra in quantum circuits with linear* T *complexity*, Phys. Rev. X **8**:041015, 2018. arXiv:1805.03662.

[18] C. Zalka, *Shor's algorithm with fewer (pure) qubits*, 2006. arXiv:quant-ph/0601097. — the coset representation of modular integers.

[19] C. Gidney and M. Ekerå, *How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits*, Quantum **5**:433, 2021. arXiv:1905.09749.

[20] C. Gidney, *Approximate encoded permutations and piecewise quantum adders*, 2019. arXiv:1905.08488.

[21] C. Gidney, *Windowed quantum arithmetic*, 2019. arXiv:1905.07682.

[22] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary Ed., Cambridge University Press, 2010.

[23] L. M. K. Vandersypen, M. Steffen, G. Breyta, C. S. Yannoni, M. H. Sherwood, and I. L. Chuang, *Experimental realization of Shor's quantum factoring algorithm using nuclear magnetic resonance*, Nature **414**:883–887, 2001.

[24] *Building Shor's algorithm from scratch: factoring 15*, Jupyter notebook (Qiskit), after [7, 8], Nielsen–Chuang [22], and the Qiskit community tutorials.

[25] B. Assel, *qiskit-shor: full implementation of Shor's factoring algorithm with the Qiskit SDK*, GitHub repository, 2025–2026. https://github.com/benjamin-assel/qiskit-shor.

[26] B. Assel, *Where are we with Shor's algorithm?*, Towards Data Science, 2025. https://towardsdatascience.com/where-are-we-with-shors-algorithm/.

[27] J. Proos and C. Zalka, *Shor's discrete logarithm quantum algorithm for elliptic curves*, Quantum Inf. Comput. **3**(4):317–344, 2003.

[28] M. Roetteler, M. Naehrig, K. M. Svore, and K. Lauter, *Quantum resource estimates for computing elliptic curve discrete logarithms*, ASIACRYPT 2017, LNCS 10625, pp. 241–270.

[29] T. Häner, S. Jaques, M. Naehrig, M. Roetteler, and M. Soeken, *Improved quantum circuits for elliptic curve discrete logarithms*, PQCrypto 2020, LNCS 12100, pp. 425–444.

[30] H. Kim, K. Jang, S. Wang, V. Srivastava, A. Baksi, G. Song, H. Seo, and A. Chattopadhyay, *New quantum circuits for ECDLP: breaking prime elliptic curve cryptography*, IACR ePrint 2026/106. https://eprint.iacr.org/2026/106.

[31] A. Schrottenloher, *Optimized point addition circuits for elliptic curve discrete logarithms*, IACR ePrint 2026/1128. https://eprint.iacr.org/2026/1128.

[32] R. Babbush, A. Zalcman, C. Gidney, M. Broughton, T. Khattar, H. Neven, T. Bergamaschi, J. Drake, and D. Boneh, *Securing elliptic curve cryptocurrencies against quantum vulnerabilities: resource estimates and mitigations*, arXiv:2603.28846, 2026.

[33] T. Khattar, N. Shutty, C. Gidney, A. Zalcman, N. Yosri, D. Maslov, R. Babbush, and S. P. Jordan, *Verifiable quantum advantage via optimized DQI circuits*, arXiv:2510.10967, 2025.

[34] M. Ekerå, *Revisiting Shor's quantum algorithm for computing general discrete logarithms*, arXiv:1905.09084, 2019.

[35] C. Gidney, *A classical-quantum adder with constant workspace and linear gates*, arXiv:2507.23079, 2025.

[36] M. Grassl, B. Langenberg, M. Roetteler, and R. Steinwandt, *Applying Grover's algorithm to AES: quantum resource estimates*, PQCrypto 2016, LNCS 9606, pp. 29–43.

[37] Classiq, *Elliptic curve discrete logarithm* (tutorial notebook), 2025–2026. https://docs.classiq.io/explore/algorithms/number_theory_and_cryptography/elliptic_curves/elliptic_curve_discrete_log.